INTEGRATED LOCATION-ROUTING-AND-SCHEDULING PROBLEMS: MODELS AND ALGORITHMS

by

Zeliha Akça

Dissertation Submitted to
Department of Industrial and Systems Engineering
for the
Doctor of Philosophy Degree
in
Industrial and Systems Engineering

Advisor
Prof. Theodore K. Ralphs

Lehigh University

2009

Contents

Contents ii

List of Tables v

List of Figures vii

1 Integrated Location Routing and Scheduling Problems (LRSP) 1

1.1 Introduction . . . 2
1.2 Motivation . . . 2
1.3 Literature Review . . . 4
1.3.1 Facility Location . . . 4
1.3.2 Vehicle Routing . . . 5
1.3.3 Integrated Facility Location and Routing . . . 9
1.3.4 Vehicle Scheduling . . . 10
1.3.5 Vehicle Routing and Scheduling . . . 11
1.3.6 Integrated Location, Routing and Scheduling . . . 12
1.3.7 Summary of Literature . . . 14
1.4 Problem Definition and Formulations . . . 15
1.4.1 Graph-Based Formulation of LRSP . . . 15
1.4.2 Set-Partitioning-Based Formulation of LRSP . . . 18
1.5 Strengthening the Set-Partitioning-Based Formulation . . . 20
1.6 Dantzig Wolfe Decomposition (DWD) of G-LRSP . . . 25
1.6.1 DWD in General . . . 25
1.6.2 DWD of G-LRSP . . . 27
1.7 Comparing G-LRSP and SPP-LRSP . . . 34
1.8 Complexity of the LRSP . . . 36

2 Branch and Price Algorithm for the LRSP 38

2.1 Introduction . . . 39
2.2 Description of Solution Methodology . . . 39

2.3 Column Generation 43
2.3.1 Formulating the Pricing Problem 43
2.3.2 Elementary Shortest Path Problem With Resource Constraints (ESPPRC) . 44
2.3.3 Underlying Network Structure For Modified ESPPRC 45
2.4 Solving the Pricing Problem Exactly 47
2.4.1 1p-ESPPRC: One Phase Pricing Problem 48
2.4.2 2p-ESPPRC: Two-Phase Pricing Problem 52
2.4.3 Bidirectional Pricing Problem 53
2.5 SPPRC: Solving the Relaxation of Pricing Problem 53
2.6 Solving the Pricing Problem Heuristically 54
2.6.1 ESPPRC-LL: ESPPRC with Label Limit 55
2.6.2 ESPPRC-CS: ESPPRC for a Subset of Customers 55
2.7 Branching Rules 56
2.8 Terminating The Branch-and-Price Tree 59
2.9 Overall Branch and Price Algorithm 62
2.9.1 Initialization 63
2.9.2 Branch Selection 66
2.9.3 Primal Feasible Solution 67
2.9.4 Solving Pricing Problem and Algorithm Overview 67

3 Cut Generation In Branch and Price Algorithm 69
3.1 Introduction 70
3.2 Overview of Cut Generation in Branch And Price Algorithm 70
3.3 Cutting Planes Based on SPP-LRSP 72
3.3.1 Relaxations of the SPP-LRSP 73
3.3.2 Projection of the SPP-LRSP 76
3.4 Disjunctive Cuts for LRSP 82
3.4.1 Implementation 1 86
3.4.2 Implementation 2 87
3.4.3 Implementation 3 90
3.4.4 Implementation 4 92
3.5 Cutting Planes Based on Arc Based Formulations 92

4 Computational Experiments 100
4.1 Introduction 101
4.2 Description of the Instances 101
4.3 Computational Experiments For Chapter 1 104
4.3.1 Comparing the LP Relaxations of SPP-LRSP and G-LRSP 104

4.3.2 Comparing Alternative Set Partitioning Formulations 105
4.4 Computational Experiments For Chapter 2 107
4.4.1 Performance of Exact Pricing Algorithms: 1p-ESPPRC and 2p-ESPPRC . 108
4.4.2 Effect of Adding Heuristic Pricing Algorithms: ESPPRC-LL and ESPPRC-CS 109
4.4.3 Performance of Primal Feasible Heuristics 112
4.4.4 Performance of Initializing Branch & Price Algorithm Using H-BP (E-BP vs E-BP-2S) 116
4.4.5 Results of Branch & Price Algorithm for Generated Instances 117
4.4.6 Performance of Branch and Price Algorithm for Instances from Literature . 123
4.5 Computational Experiments For Chapter 3 125
4.5.1 Lifted Cover Inequalities and GAP Cuts 125
4.5.2 Disjunctive Cuts 127

5 Case Study: LRP 131

6 Case Study: Effect of Scheduling Constraint: LRSP vs LRP-DC 132
6.1 Effect of Scheduling Constraint in the LRSP 133

A Solution Details 137
A.1 Details of the Optimal Solutions in 25-customer Instances 137
A.2 Details of the Optimal or Best Solutions in 40-customer Instances 142

List of Tables

4.1 Location & Demand Data for LRSP instances generated from Cordeau et al. (1997) 102
4.2 Facility Locations for LRSP instances 102
4.3 Summary of Parameters 103
4.4 LP Relaxation Bounds of G-LRSP and SPP-LRSP: 25 Customer Instances 105
4.5 LP Relaxation Bounds of G-LRSP and SPP-LRSP: 30 Customer Instances 106
4.6 Comparison of LP relaxation bounds with constraints (1.24), (1.25) and (1.18) . . . 107
4.7 Performance of 1p-ESPPRC, 1p-ESPPRC-NwD, 2p-ESSPRC 109
4.8 Performance improvement with 2p-ESPPRC-LL and 2p-ESSPRC-CS 111
4.9 Performance improvement with 2p-ESSPRC-CS in BP algorithm for 40 customers 113
4.10 Performance of I-Heur, BB, H-BP 114
4.11 I-Heur, BB, H-BP Upper Bounds vs. LP Relaxations and Optimal IPs 115
4.12 Performance of BB at Root Node of E-BP 115
4.13 E-BP vs E-BP-2S: Effect of H-BP, 25-Customer Instances 117
4.14 E-BP vs E-BP-2S: Effect of H-BP 118
4.15 Results for instances with 25 customers and 5 facilities 120
4.16 More Solution Details for 25-customer instances 120
4.17 Results for instances with 40 customers and 5 facilities 122
4.18 Solution details for instances with 40 customers and 5 facilities 124
4.19 More Solution Details for 40-customer Instances 124
4.20 Comparison of Lin et al. (2002) results and branch-and-price results 125
4.21 Cover Inequalities and GAP Cuts for 25-customer Instances 126
4.22 Cover Inequalities and GAP Cuts for 25-customer Instances 127
4.23 Disjunctive Cutting Planes: Implementation 1 and 2 128
4.24 Disjunctive Cutting Planes: Implementation 3 and 4 128
4.25 Time in DISJCUTGEN procedure for implementations 1, 2, 3 and 4 129

6.1 Vehicle Capacity values used in the instances 133
6.2 Results for instances with 25 customer 5 facilities - LRPDC 134
6.3 Results for instances with 25 customer 5 facilities - LRPDC 135

6.4 Solution details for LRPDC and LRSP 136

List of Figures

1.1 Example of pairings 19

2.1 Branch-and-Price Tree 42
2.2 Example of a network and a path from source to sink (dashed line) 46

Chapter 1

Integrated Location Routing and Scheduling Problems (LRSP)

1.1 Introduction

Logistics, planning and controlling the efficiency of the flow and storage activities in the system, is one of the most important functions of a company. The design of a distribution system begins with the questions of where to locate the facilities and how to allocate customers to the selected facilities. Furthermore, determining how to distribute goods to customers from these facilities and how to use vehicles and/or distribution resources efficiently are important decisions that arise in the design of logistics systems.

In this research, we focus on a class of problems that integrates the decisions of facility location, vehicle routing, and route assignment and seeks to minimize the total cost. We refer to such problems as location routing and scheduling problems (LRSPs). The LRSP generalizes and subsumes several well-studied problem classes, such as the location and routing problems (LRP), the vehicle routing problem (VRP), the multi-depot VRP (MDVRP), and the VRP with multiple trips (VRPMT).

Given a set of candidate facility locations and a set of customer locations, the objective of the LRSP is to select a subset of the facilities, construct a set of delivery routes, and assign routes to vehicles in such a way as to minimize total cost subject to satisfaction of the system's constraints. The total cost may include both fixed costs and operating costs of facilities and vehicles.

The LRSP arises in the context of many delivery problems, such as gas, oil, food, beverage, bill and newspaper delivery. In practical applications of the LRS problem, various constraints need to be considered. For example, facilities and vehicles may have capacities, the length of routes may be restricted with time limits, service of customers may include both deliveries and pick ups, service to customers might be restricted to occur within time windows, different types of vehicles might be used, customers might have priorities, and some data such as demand of customers or travel times might be uncertain. In this research, we consider an LRSP with capacity constraints on the facilities and on the vehicles, as well as time constraints on the vehicles. We assume that all data is deterministic, and the problem is either a pick-up or delivery problem.

1.2 Motivation

The motivation behind the integration of three decisions, facility location, vehicle routing and vehicle scheduling is to improve the efficiency of the solution and yield more accurate and cost-effective solutions. Early in the literature (Webb (1968) and Salhi and Rand (1989)), it has been realized that the location decision and the routing decision are interdependent when customers demand less-than-truckload quantities. Therefore, if it is possible to serve customers through routes making multiple stops, the assumption of single-customer routes in location models will not accurately capture the transportation cost. Gas and oil delivery systems, distribution of goods to retailers, and newspaper and bill delivery systems are examples of such systems that require routes with multiple stops.

Location and routing models seek to minimize total cost by simultaneously selecting a set of facilities and constructing a set of delivery routes that satisfy the specified system constraints. LRPs, however, implicitly assume that each vehicle covers exactly one route. This may potentially overestimate the number of vehicles required and the associated distribution cost. In many cases, it is possible to serve multiple routes with a single vehicle, in which case the decision of assigning routes to vehicles becomes interdependent with the location and routing decisions. Eilon (1977) reports that fixed costs are equal to the eighty percent of total vehicle costs. Therefore, models integrating the scheduling decision and considering the fixed costs of the vehicles may yield more cost efficient solutions. Furthermore, some systems require different types of vehicles, which may have different costs than the multi-purpose vehicles. In such cases, the number of customized vehicles is an important decision for the company. Similarly, if the service to the customer requires a trained worker who is more expensive than a regular driver, then the company will want to consider training costs, which are also fixed costs.

Why capturing the costs more accurately is important? Major components of logistics cost are warehousing costs including inventory carrying costs and transportation costs. Based on estimates for the U.S. in 2002, transportation costs were $577 billion, and inventory carrying costs and warehousing costs were $298 billion. The total logistics costs were $910 billion, which was equivalent to 8.7% of the U.S. gross domestic product in 2002 (Mac (2005)). Undeniably, logistics costs are very important for the U.S. economy as well as for a company. Since the contribution of logistics activities to the GDP is large, improvements in the efficiency of the logistics activities will contribute to the growth of the U.S. GDP. Even a small amount of improvement in distribution systems can result in a substantial amount of savings.

Another motivation to consider scheduling decisions together with the location and routing decisions is to be able include more system constraints to the model. In distribution systems in which the company has a constant fleet size, the number of vehicles should be considered in the model. To be able to cover all customers, it might be necessary to use a vehicle for more than one route. Therefore, the decision of scheduling of vehicles must be considered with the decisions of location and routing. In some systems, there are time constraints for the customers or the vehicles. Delivery of some items such as perishable foods, newspapers and bills are time sensitive. In addition, the overtime for the workers might be expensive, hence the company is not willing to pay overtime driver hours. Therefore, while designing the model, time limit for the vehicle routes must be considered.

1.3 Literature Review

The LRSP integrates three sets of decisions, facility location, vehicle routing and vehicle scheduling. In this chapter, we briefly review the literature for each component problem as well as the literature for related integrated problems.

1.3.1 Facility Location

The first component of the LRS problem is where to locate the facilities and how to allocate the customers to these facilities. Determining where to locate facilities is an important problem that arises in the design of distribution networks. ReVelle and Eiselt (2005) and Klose and Drexl (2005) provide recent surveys of the various types of location problems, formulations and solution algorithms. The facility location literature encompasses a wide variety of problems, which can be categorized in a variety of ways. Daskin (1995) provides a taxonomy of facility location problems and describes methods categorizing location problems. Location problems can be grouped as planar problems, network problems or discrete problems based on the allowable location of the facilities and the customers. Problems in which customers and facilities can be located anywhere in the plane are planar location problems. Problems in which the customers and the candidate locations are restricted to a discrete set are discrete location problems. Network location problems are a special case of discrete location problems in which the candidate facilities and the demand locations are restricted to the nodes of a network and travel is only possible via the arcs of network.

Most relevant to our study of the LRS problem are discrete location problems. Labbé and Louveaux (1997) provide a survey of the literature related to discrete facility location problems including application areas, solution methods and model extensions. Early work on discrete location problems started with Hakimi (1964, 1965) who introduced the p-median problem on a network, which is known to be NP-hard (Kariv and Hakimi (1979)). The objective is to locate p facilities on a network with demand nodes so as to minimize the total weighted distance between the customers and the closest facilities to them. ReVelle and Swain (1970) were the first to formulate the p-median problem as a 0-1 integer program. Since then researchers have developed both exact methods (e.g., Galvao (1993), Bilde and Krarup (1977)) and heuristic methods (e.g., Maranzana (1964), Teitz and Bart (1968), Rosing and ReVelle (1997)) for the p-median problem. Depending on the difficulty of the instances, optimal or near optimal solutions for p-median problems are reported for instances up to 3000 customer nodes.

By removing the constraint that p locations must be selected and by adding a fixed cost for each candidate location, we obtain a simple plant location problem from the p-median problem. Also called the uncapacitated facility location problem (UFLP), the problem has been well-studied in the literature (Cornuejols et al. (1990)). The objective of the problem is to select a subset of the candidate facilities and to assign customers to the selected facilities so as to minimize the sum of fixed and

operating costs. The operating cost in the UFLP is the transportation cost calculated assuming individual customer routes that originate at a facility, serve one customer and return to the same facility. The problem is NP-hard (Cornuejols et al. (1990)). A number of researchers have focused on polyhedral analysis of the UFLP. Although a complete description of the UFLP polytope is not known, Guignard (1980), Cornuejols and Thizy (1982) and Cho et al. (1983 a,b) provide partial descriptions. Others have focused on developing algorithms for the UFLP, for example, branch-and-bound based algorithms by Erlenkotter (1978) and Körkel (1989), a Lagrangian relaxation algorithm by Cornuejols et al. (1977) and a dual simplex algorithm by Simao and Thizy (1989). The best results for the problem are reported by Körkel (1989) using an approach that extends the dual-ascent heuristic procedure developed by Erlenkotter (1978) (ReVelle and Eiselt (2005)).

By adding capacity constraints on the facilities, we obtain the capacitated facility location problem (CFLP) from the UFLP. Leung and Magnanti (1989) and Aardal et al. (1995) investigate the polyhedral structure of the CFLP and introduce facet defining inequalities for the problem, which lead to cutting plane algorithms (Aardal (1998)) for solving the problem. By using an extended formulation (with other valid inequalities) of the CFLP, Cornuejols and Thizy (1982) find and compare lower bounds for the CFLP by relaxing different constraints within a Lagrangian relaxation and decomposition algorithm. Other papers presenting various solution algorithms for the CFLP are Baker (1986), Roy (1986), Pirkul (1987) and Beasley (1988).

In the CFLP, the demand of a customer can be supplied by more than one facility. However, in some applications, there may be a single source requirement for a customer, which means that the total demand of a customer must be supplied by exactly one facility. When the single source requirement is added to the CFLP, it is called the capacitated facility location problem with single sourcing (CFLPSS). The CFLPSS is more difficult than the CFLP since, while the CFLP is a mixed integer model, the CFLPSS is a pure integer model with same number of variables and constraints. In addition, for a given set of open facilities, the CFLP solves a transportation problem to find the assignment of customers to the facilities, but the CFLPSS solves a generalized assignment problem, which is also an NP-hard problem. In the literature, most of the studies use heuristics, especially Lagrangian based heuristics, such as Klincewicz and Luss (1986) and Sridharan (1993). Delmaire et al. (1997) design several metaheuristics and Ahuja et al. (2004) develop a neighborhood search algorithm for the problem. Holmberg et al. (1999) develop an exact branch-and-bound algorithm that uses a Lagrangian based heuristic and repeated matching heuristic that solves successive matching problems. They can solve problems up to 200 customers and 30 facilities.

1.3.2 Vehicle Routing

The second component of the LRS problem is how to design the vehicle routes. The classical vehicle routing problem (VRP) finds a set of minimum cost vehicle routes, each of which starts at the depot, visits some customer locations and returns to the depot. There is a large body of

literature related to the VRP that describes the many problem variations and solution approaches. The VRP has many variations depending on the characteristics of the vehicles, the facilities and the customers. For example, the vehicles may be identical or different with respect to capacity limits; the problem may be concerned with delivery only, pick up only or both delivery and pick up; the vehicles might be restricted to serve each customer in a given time window and/or serve according to precedence relations of the customers. In addition, the problem may involve a single facility or multiple facilities. Christofides (1985) surveys the different types of formulations used to model the VRP and the exact and heuristic algorithms designed for the problem prior to 1985. In a recent study, Toth and Vigo (2002) give a detailed description of the variants of the VRP and their formulations and they discuss the solution methods that are applied to these variants. Letchford and González (2006) review the different formulations and derive relationships between these formulations. Using projection, they compare the LP relaxation bounds and show that some valid inequalities in the two index space are implied by extra flow variables or set partitioning variables.

To formulate the VRP as an integer program, three different approaches are used in literature. One approach is to define integer variables for each arc or edge in the model representing the number of times the arc or edge is traveled. This type of model, which is called a vehicle flow model, can be divided into two groups: two index formulations and three index formulations (Laporte and Nobert (1987)). In a two index formulation, a flow variable has two indices representing the origin and the destination of the arc and in a three index formulation, each variable has an additional index representing the vehicle traveling the arc. Vehicle flow models, especially two index formulations, are very common in the VRP literature. **ZE: For capacitated VRP (CVRP), Letchford and González (2006) investigate the relationship between two index and three index vehicle flow models and present some theoretical results. They prove that the three index formulation strengthened with certain classes of valid inequalities gives the same lower bound with a two index formulation with simpler constraints. Therefore, the LP relaxation bound based on the three index formulation is weaker than the LP relaxation bound based on the two index formulation.** In addition to the constraints to form feasible routes, the vehicle flow model includes generalized subtour elimination constraints, the number of which grows exponentially with the number of nodes in the problem. There is one subtour elimination constraint for each subset of customers, which forces a lower bound for the number of vehicles exiting the subset based on the minimum number of vehicles necessary to serve the customers in the subset. The exponential number of constraints is handled by considering a relaxation of the model that includes only some of the constraints or by using a branch-and-cut algorithm. Miller, Tucker, and Zemlin (1960) introduce a polynomial number of constraints that eliminate subtours, but the LP relaxation of the model with these new constraints is weaker than the model with generalized subtour elimination constraints. Desrochers and Laporte (1991) strengthen the Miller-Tucker-Zemlin subtour elimination constraints by lifting them. They define stronger valid inequalities that can be adapted to variations of the VRP, such as the capacitated VRP (CVRP), the distance constrained VRP (DVRP) and the VRP with time

windows (VRPTW).

A second modeling approach, developed by Garvin et al. (1957), is called a commodity flow model. In this approach, an additional variable for each arc is added to the variables in the vehicle flow model. These additional variables represent the amount of demand that flows along the edges traveled by the vehicles. Letchford and González (2006) analyze three different commodity flow models: one commodity, two commodity and multi commodity flow models. In two commodity flow model, in addition to the flow on each arc representing the vehicle load, there is a reverse flow equal to the vehicle residual capacity. In multi commodity flow model, demand of each customer represents one commodity and flow of each commodity is denoted by different variables on each arc. Gouveia (1995) and Letchford and González (2006) state some theoretical results about the relationship between commodity flow model and two index vehicle flow model. No exact solution algorithms for this approach are reported in the literature. Based on the computational experiments, Gavish and Graves (1979) report large gaps between the LP solutions and the optimal integer solutions when the number of customers are greater than 15.

A third approach, proposed by Balinski and Quandt (1964), is a set partitioning formulation, in which there is one variable corresponding to each feasible vehicle route. The model has possibly exponentially large number of binary variables representing whether a specific route is selected or not. Since the feasibility of each route is considered in the process of constructing the set of routes to be included in the model, various routing constraints, such as vehicle capacity, route length restrictions and time windows for the customer nodes, can be included in the model without changing the formulation. Because of the exponentially large number of variables, it is usually solved by a column generation algorithm. The LP relaxation of the model is generally tight (Toth and Vigo (2002)). Letchford and González (2006) theoretically compare the strength of the set partitioning formulation with the two index vehicle flow formulation and prove that the set partitioning variables imply some of the valid inequalities in the two index space.

In reviewing the solution algorithms for the VRP, we highlight those for deterministic, capacitated VRPs. In a capacitated VRP (CVRP), the total demand of a route must be less than the capacity of the vehicle. Both exact methods and heuristic methods have been developed to solve the CVRP. Laporte and Nobert (1987) survey the exact algorithms for solving the VRP. They give a complete analysis of branch-and-bound algorithms, which are the most common way to solve the CVRP. They also describe dynamic programming and integer programming approaches that use different formulations of the problem. In addition to branch-and-bound procedures, Laporte (1992) describes set partitioning formulations, column generation procedures and heuristic approaches (e.g. Clark and Wright, tabu search and sweep algorithm) for the problem.

Branch-and-cut algorithms are very successful in finding optimal solutions for large instances of the symmetric TSP, which is a problem closely related to the CVRP. This success has motivated the development of branch-and-cut algorithms for vehicle flow models of the CVRP. Toth and Vigo

(2002) describe how to apply a branch-and-cut algorithm to the VRP and the CVRP. Branch-and-cut algorithms incorporate the results of polyhedral studies of the CVRP polytope, valid inequalities (e.g. Laporte and Nobert (1981b) and Laporte et al. (1985)) and facets of the polytope. The first papers published about the facets of the CVRP polytope were by Araque, Hall, and Magnanti (1990) and Araque (1990), who assumed unit demand for customers, and by Cornuejols and Harche (1993), who extended the research to general demand. One of the main difficulties with branch-and-cut algorithms is that the separation procedure for the cuts might be NP-hard. To overcome this difficulty, researchers seek polynomial time algorithms for exact separation (e.g., McCormick et al. (2002) and Blasum and Hochstättler (2000)) and heuristic separation algorithms (e.g., Augerat et al. (1999), Ralphs et al. (2003)).

Set covering-based formulations are used successfully by Cullen et al. (1981) to solve the VRP heuristically and by Agarwal et al. (1989), Bixby (1999), Hadjiconstantinou et al. (1995) and Desrochers et al. (1992) to solve the VRP exactly. The first step for solving a set partitioning-based formulation is to solve the LP relaxation of the model using column generation. The objective of the column generation subproblem (pricing problem) is to construct feasible vehicle routes which include each customer at most once (elementary routes) and satisfy some other constraints required for the feasibility of a route such as vehicle capacity. Since the complexity of the problem of finding elementary routes is NP-hard, it is common to relax this property and solve the set partitioning formulation with non elementary routes.

ZE: Read Toth sections and fix the following paragraph. There seems to be something wrong

Agarwal et al. (1989) use a branch-and-bound algorithm, Desrochers et al. (1992) use a branch-and-bound algorithm and dynamic programming, Bixby (1999) uses a cutting plane algorithm and Hadjiconstantinou et al. (1995) solve the dual of column generation problem using a dynamic programming heuristic to get the LP relaxation of the problem. For many of the problems, the LP relaxation solution is very close to integer solution. However, even if the LP relaxation is very strong, some branching may be required to obtain an integer solution. Both cutting plane algorithms (see Toth and Vigo (2002) for details) and branch-and-price algorithms (see Desrochers et al. (1992)) have been used.

ZE: ? combine cutting plane and column generation approach by extending the set partitioning formulation for CVRP with the two index vehicle flow variables and constraints based on these variables. They design a branch, cut and price algorithm which seems to be very successful compared to the reported results in the literature.

A generalization of the VRP, which is also a special case of our LRS problem, is the Multi-Depot Vehicle Routing Problem (MDVRP). The MDVRP constructs a set of vehicle routes with minimum cost in which each customer is visited by exactly one vehicle and each vehicle route starts and ends at the same depot. Exact solution algorithms are very rare for MDVRP; Laporte et al. (1984) and Laporte et al. (1988) are the only two papers that report exact solution algorithms for the MDVRP. Both solution methods are based on branch-and-bound algorithms. Many papers develop heuristic

algorithms based on construction and improvement techniques (e.g., Tillman (1969), Tillman and Cain (1972), Gillett and Johnson (1976) and Chao et al. (1993)) and tabu search (e.g., Cordeau et al. (1997) and Renaud et al. (1996)).

1.3.3 Integrated Facility Location and Routing

In most facility location problems, the assumption is that customers are served on individual routes or that customers travel to the facilities. For situations in which customers receive service from routes making multiple stops, the assumption will not accurately capture the transportation cost. In order to accurately represent the cost of serving customers, we need to know the order of the customers on the route. Hence, the location and routing problems must be solved simultaneously. As early as 1968, Webb (1968) realized that using straight line trips to each customer versus using multiple stop tours resulted in different solutions. Salhi and Rand (1989) also investigated the effect of ignoring the routing of vehicles while deciding the locations of facilities. They show that solving the location and routing problems independently does not always result in the best solution in terms of cost. Thus, although solving the combined problem may be more difficult than solving the individual ones, it may result in a more cost efficient solution.

Laporte (1988) reviews the work on deterministic location routing problems; he describes the different formulations of the problem, the solution methods used and the computational results published up to 1988. A more recent paper by Min et al. (1998) classifies both deterministic and stochastic problems in the LRP literature with respect to problem characteristics and solution methods. They group the LRP papers according to characteristics such as single or multistage, deterministic or stochastic, single or multiple facilities and vehicles, and uncapacitated or capacitated vehicles and facilities. Because the LRP combines two NP-hard problems, researchers have developed a wide variety of both heuristic and exact algorithms for the LRP. Most of the heuristic algorithms tend to divide the problem into subproblems. Heuristic algorithms handle these subproblems sequentially or iteratively. Based on Min (1996), for problems with large facility fixed costs and capacitated vehicles, sequential approaches perform computationally better. Some examples of sequential heuristic algorithms are developed by Perl and Daskin (1985), Hansen et al. (1994) and Barreto et al. (2007). Perl and Daskin (1985) develop a heuristic for a three index flow formulation of an LRP with capacitated distribution centers and vehicles. They decompose the problem into three subproblems: a multi-depot vehicle dispatch problem, a warehouse location-allocation problem and a multi-depot routing allocation problem. They develop an algorithm that solves the problems sequentially but considers the dependency between the problems. They solve the multi-depot vehicle dispatch problem and the multi-depot routing allocation problem heuristically and the location-allocation problem exactly. Hansen et al. (1994) introduce flow variables and flow constraints to the model in Perl and Daskin (1985) to get an MILP and solve the new model with a heuristic that is a modification of the heuristic developed by Perl and Daskin (1985). Barreto

et al. (2007) develop a sequential heuristic algorithm for problems with capacitated facilities and capacitated vehicles. In their heuristic, the first step is to solve routing subproblem. They group customers using clustering techniques and design TSP solutions. Then, by assuming each group of customers as a single customer, single source capacitated location routing problem is solved. Srivastava (1993) presents 3 iterative heuristics called SAV1, SAV2 and CLUST and compares the results with a classical sequential heuristic in which facility locations are determined first and vehicle routes are designed based on open facilities. SAV1 heuristic starts with all depots being open and using a multi depot vehicle routing heuristic calculates approximate routing costs for the facilities. At each step a facility is closed until the problem becomes infeasible. SAV2 heuristic works similar to SAV1 except initially all depots are closed and at each iteration one depot is opened. CLUST heuristic generates groups of customers using a minimum spanning tree and a density search clustering technique. Then a depot is selected for each cluster considering the distance between cluster centroid and the depot.

ZE: And finding LB in Albareda Sambola

Exact algorithms for the LRP have been developed mostly for two index vehicle flow formulations. Laporte and Nobert (1981a) solve a single depot model with a constraint relaxation method and a branch-and-bound algorithm. They report solving problems with 50 customer locations. Laporte et al. (1983) solve a multi-depot model using a constraint relaxation method and Gomory cutting planes to satisfy integrality. They can solve problems with at most 40 customer sites. Laporte et al. (1986) apply a branch-and-cut algorithm to a multi-depot LRP model with vehicle capacity constraints. They use subtour elimination constraints and chain barring constraints that guarantee that each route starts and ends at same facility. The authors report computational results for problems with 20 customer locations and 8 depots. Belenguer et al. (2006) provided a two-index formulation of the LRP with capacitated facilities and capacitated vehicles and presented a set of valid inequalities for the problem. They developed two branch-and-cut algorithms based on different formulations of the problem. They reported that they could solve instances with up to 32 customers to optimality in less than 70 CPU seconds and could provide good lower bounds for the rest of the instances, which had up to 134 customers. Berger (1997) develops a set partitioning-based model for an LRP with route-length constraints and uncapacitated vehicles and facilities. **?** extend the work to develop an exact branch-and-price algorithm in which they solve the pricing problem as an elementary shortest path problem with one resource constraint. They report computational results for problems with 100 customers and various distance constraints.

1.3.4 Vehicle Scheduling

The third component of the LRS problem is how to assign routes to vehicles, which is known as the vehicle scheduling or loading problem. The (multi-depot) vehicle scheduling problem ((MD)VSP) seeks a least-cost assignment of routes (or trips), each with a starting and ending time, to vehicles

such that each route is assigned to one vehicle, each vehicle starts a route after the starting time and finishes before the ending time and each vehicle starts from a depot and returns to same depot. In some variations, there are constraints such as depot capacity and upper and lower bounds on the number of vehicles at each depot. The MDVSP is a special case of the LRS problem; given a set of selected facilities and a set of vehicle routes connected to these facilities such that each customer is served by one vehicle, the LRS problem reduces to a MDVSP. Carraresi and Gallo (1984) describe how to convert a single depot VSP to an assignment problem or a minimum cost network flow model and therefore they conclude that the single depot VSP is easy to solve. However, Bertossi et al. (1987) show that if there are more than 2 depots, the MDVSP is NP-hard.

Both heuristic and exact solution methods are proposed for the MDVSP. Bodin and Golden (1981) propose two heuristics based on decomposing the problem into single depot VSPs. Bertossi et al. (1987) model the MDVSP by defining an independent problem for each depot and additional constraints to cover each trip exactly once. They relax the additional constraints using a Lagrangian relaxation algorithm and then solve independent VSPs. Dell'Amico et al. (1993) propose a heuristic method that guarantees a solution with the minimum number of vehicles. They solve shortest path problems on a graph that includes one node for each route and arcs between two nodes that can be assigned to the same vehicle. A path on the graph represents a vehicle duty.

Several exact algorithms based on different formulations of the problem are proposed. Carpaneto et al. (1989) use a branch-and-bound algorithm based on additive lower bounds. Forbes et al. (1994) develop an algorithm using a dual simplex method to solve their three-index integer programming model, which is a multi-commodity network flow problem. Ribeiro and Soumis (1994) and Bianco et al. (1994) use set partitioning-based formulations for which they generate columns and apply a branch-and-bound algorithm to get an integer solution. Fischetti et al. (2001) design a branch-and-cut algorithm by defining valid inequalities of the MDVSP.

1.3.5 Vehicle Routing and Scheduling

In the VRP, there is one-to-one relationship between vehicles and routes, which means that one route is assigned to each vehicle. In a combined vehicle routing and scheduling problem (VRSP), scheduling constraints are added to the VRP constraints to change this one-to-one relationship between vehicles and routes. Therefore, the VRSP seek to determine the set of vehicle routes as well as the assignment of these routes to the available vehicles. In the literature, this version of the VRSP is sometimes called the VRP with multiple trips (VRPMT). Only a few papers in the literature address the VRPMT and they only describe heuristic approaches.

Taillard et al. (1996) use a three-step algorithm to find good feasible solutions for the VRSP. In the first step, they generate some number of VRP solutions using a tabu search algorithm. In the second step, they select a subset of these routes using an enumerative algorithm and, in the third step, they use a bin packing heuristic to combine the routes in such a way that they will not violate

the time constraint and assign them to vehicles. The authors generate test problems with 50 to 199 customers. Feasible solutions are provided for 82 instances out of 104 instances. They report that the cost of the feasible solutions are on the average within 1.2% of the VRP solution obtained by Rochat and Taillard (1995).

Brandao and Mercer (1997) use a tabu search algorithm and the GENI algorithm (developed by Gendreau et al. (1992) to solve TSP) to solve the VRPMT with time window constraints. They design a three-phase algorithm in which the solution is improved in each phase. The input for the first phase is an initial set of routes constructed using VRP constraints. The first phase checks each route in the initial set for the time window infeasibility and, if there is an infeasible route, the route is divided into smaller routes that satisfy the time windows. The second phase constructs a solution with smaller cost by searching the set of routes. Similar to the second phase, the third phase also seeks a solution with lower cost but the solution space is restricted to the feasible space during the search. They evaluate their algorithm by comparing their solutions with manual solutions in a case study. They report on average 20% better solutions. In addition, Brandao and Mercer (1998) solve the instances created by Taillard et al. (1996). They can find feasible solutions for 89 instances out of 104 instances.

Petch and Salhi (2004) develop a multi-stage heuristic algorithm in which they create vehicle routes and then combine the routes using bin packing heuristics. Olivera and Viera (2004) solve the VRPMT using an adaptive memory algorithm that is an enhancement of tabu search. The basic idea is to construct new solutions using the components of the previously generated solutions. They initialize the memory by generating some feasible solutions to the VRP, which they use to construct feasible assignments of routes to vehicles. Then, they use tabu search to improve the constructed solution. This heuristic differs from the others in that it constructs feasible vehicle assignments in each iteration, while the others only construct the assignment at the final stage. They can find feasible solutions for 95 instances out of 104 Taillard et al. (1996)'s instances. The feasible solutions are on the average within 1.6% of the best known VRP solutions. In infeasible solutions, in which overtime is used, the solutions provided by Olivera and Viera (2004) are better than the solutions provided by other authors.

1.3.6 Integrated Location, Routing and Scheduling

The decisions of location and routing are interdependent for the cases in which customers are served by routes with multiple stops. In such cases, solving the combined LRP may result in a more cost efficient solution. We can go one step further and say that most LRP papers assume that there is a one-to-one relationship between routes and vehicles. However, it may be that the vehicle capacity restricts the route but the total available time does not. In this case, a vehicle can serve more than one route, and thus, the decision of assigning of routes to vehicles should be optimized together with the decisions of facility location and vehicle routing. Integrating these three decisions gives

rise to a class of problems called *integrated location routing and scheduling* (LRS) problems.

As far as we know, the papers by Lin et al. (2002) and Lin and Kwok (2005) are the only LRS papers to appear in the literature. Lin et al. (2002) describe an LRS problem, which arises in the delivery of telephone bills, in which they seek to locate facilities and to route and assign vehicles so as to minimize cost. To solve the problem, they develop a heuristic that divides the problem into three phases, facility location, routing and loading, and that iterates through the phases. The location phase determines the set of facilities via enumeration. For a given set of opened facilities, the routing phase constructs vehicle routes using the Clarke and Wright algorithm (Clarke and Wright (1964)) followed by an insertion heuristic and more extensive metaheuristics. The loading phase optimally assigns routes to vehicles considering the time limits. The authors report the development of a branch-and-bound algorithm to solve the exact problem to evaluate their heuristics but they do not provide a formulation. They test their algorithm on five instances with 10-12 customers and 4 potential depots, but the branch-and-bound algorithm can solve only three of the five instances to optimality within the specified time limit. For these instances, they report that their heuristic algorithm can find optimal or good solutions quickly. In addition, they solve a real life instance with 27 customers and 4 depots using their heuristic algorithm and compare the results with the management's manual solution.

Lin and Kwok (2005) extend the study by Lin et al. (2002) to solve a multi-objective LRS problem that considers workload balance in terms of total vehicle time and total vehicle load as well as total cost. They use the same three-phase heuristic algorithm designed by Lin et al. (2002). Since they consider multi-objective problems, for each feasible solution in the algorithm, they keep three performance measures, the total cost, the difference between the maximum and the minimum vehicle time and the difference between maximum and minimum vehicle load. The feasible solutions are evaluated based on these three performance measures and only non-dominated solutions are kept. In addition, they design a tabu search (TS) algorithm for the routing phase of the algorithm and compare its performance with the simulated annealing (SA) algorithm applied in Lin et al. (2002). While the TS algorithm and the SA algorithm have similar performance in some instances, the SA algorithm is better than the TS algorithm when the number of demand nodes in a route increases. Furthermore, they compare the performance of the algorithm in two approaches: assigning vehicles to routes after the routing phase (sequential approach) and assigning vehicles while constructing routes (multiple approach). They conclude that the performance of the approaches depends on the difficulty of the instance. The multiple approach is better than the sequential approach when the solution includes routes with more demand nodes. They test their algorithm with two sets of real data, one with 27 customers and one with 89 customers, and a set of simulated data including instances with 100 and 200 customers.

1.3.7 Summary of Literature

The LRS problem integrates three NP-hard problems: facility location problem, vehicle routing problem and vehicle scheduling problem. Solving these three problems interdependently and simultaneously results in more realistic and cost efficient solutions. However, the combined LRS problem is more difficult than its components. In the literature, there are only two papers investigating the LRS problem. These papers only develop heuristic algorithms, they do not provide a formulation of the problem, an exact algorithm to find the optimal solution and a lower bound for the optimal solution.

The LRS problem is a generalization of other NP-hard problems such as LRPs, MDVRPs, VRPMTs. There are only few papers solving those problems exactly. For the multi-depot LRP, the optimal solutions are reported for instances with uncapacitated vehicles and up to 40 customers and instances with capacitated vehicles and up to 20 customers. For the MDVRP, computational results for exact algorithms are provided for instances with up to 50 customers and 8 depots (with symmetric distance matrix) and for instances with up to 80 customers and 3 depots (with asymmetric distance matrix). For the VRPMT, no exact algorithm is provided in the literature. Currently, we can solve instances up to 40 customers and 5 facilities.

1.4 Problem Definition and Formulations

In this section, we provide a formal description of the LRSP considered in this research and present two formulations of the problem.

We consider an LRSP with capacity constraints on the facilities and on the vehicles, as well as time constraints on the vehicles. In particular, given a set of candidate facility locations and a set of customer locations, we seek a solution in which

i. each customer is visited exactly once,

ii. each route starts and ends at the same facility,

iii. the total demand of the customers assigned to a route is at most the vehicle capacity,

iv. the total working time of a vehicle is no more than the time limit, and

v. the total demand of the customers assigned to a facility does not exceed the capacity of the facility.

We develop models for the delivery form of the LRSP and assume that there is no service time at the customers. The models, however, can be adapted easily for pickup problems and for nonzero service times. We assume that the demand information and distance matrix are known and not dynamic. We assume that each facility has enough vehicles to serve the allocated customers. Simply, we assume that the number of vehicles at each facility is equal to the number of customers. And we consider one class of vehicles, all having the same characteristics and working under the same conditions.

At the heart of the LRSP is a network flow structure, deriving from the routing of customer demands, that is constrained through the vehicle and facility capacities. Hence, we consider both *arc-based* and *path-based* formulations for the LRSP, as each has been widely applied for related problems. In Section 1.4.1, we present a three-index commodity flow formulation of the LRSP. In Section 1.4.2, we present a set-partitioning-based formulation of the problem.

1.4.1 Graph-Based Formulation of LRSP

For the related problems of LRP and VRP, the literature contains two main types of graph-based models, vehicle flow and commodity flow. In both formulations, integer variables represent the number of times a vehicle traverses a given arc or edge in an underlying graph. In commodity flow formulations, an additional variable representing the number of units of a given commodity transported along the arc is present. In general, vehicle flow models contain an exponential number of subtour elimination constraints, whereas commodity flow models, with the help of additional flow variables, use a polynomial number of constraints to eliminate subtours. For the LRP and the VRP,

two-index vehicle flow formulations are preferred in solution algorithms based on branch-and-cut since these formulations include a smaller number of variables and their LP relaxations yield better bounds. However, it is not possible to formulate the LRSP using variables with only two indices because of the time constraint for the vehicles. In this context, we require a three-index commodity flow model.

To formulate the model, we introduce the following notation.

Sets

$$
\begin{aligned}
I &= \text{set of customer locations,} \\
J &= \text{set of candidate facility locations,} \\
N &= I \cup J = \text{set of all nodes,} \\
A &= (J \times I) \cup (I \times I) = \text{set of arcs,} \\
H_j &= \text{set of vehicles located at facility } j, \forall j \in J, \text{ such that } |H_j| = |I|, \\
H &= \bigcup_{j \in J} H_j = \text{set of all vehicles such that } |H| = |I||J|.
\end{aligned}
$$

Parameters

$$
\begin{aligned}
C_j^F &= \text{daily equivalent fixed cost of opening facility } j, \forall j \in J, \\
C^O &= \text{vehicle operating cost per unit travel time,} \\
C^V &= \text{daily equivalent fixed cost of a vehicle (including the driver cost),} \\
D_i &= \text{demand of customer } i, \forall i \in I, \\
L_j^F &= \text{capacity of facility } j, \forall j \in J, \\
L^V &= \text{capacity of a vehicle,} \\
L^T &= \text{time limit for a vehicle and the driver, and} \\
T_{ij} &= \text{travel time between locations } i \text{ and } j, \forall (i,j) \in A.
\end{aligned}
$$

Decision Variables

$$
\begin{aligned}
x_{ikh} &= \begin{cases} 1 & \text{if vehicle } h \text{ travels on arc } (i,k), \forall h \in H, (i,k) \in A, \\ 0 & \text{otherwise,} \end{cases} \\
y_{ikh} &= \text{flow on arc } (i,k) \text{ carried by vehicle } h, \forall (i,k) \in A, h \in H, \\
t_j &= \begin{cases} 1 & \text{if facility } j \text{ is selected, } \forall j \in J, \\ 0 & \text{otherwise,} \end{cases} \\
v_h &= \begin{cases} 1 & \text{if vehicle } h \text{ is used, } \forall h \in H, \text{ and} \\ 0 & \text{otherwise.} \end{cases}
\end{aligned}
$$

$$
\begin{aligned}
\textbf{(G-LRSP)}\ \text{Minimize} \quad & \sum_{j\in J} C_j^F t_j + C^V \sum_{h\in H} v_h + C^O \sum_{h\in H} \sum_{(i,k)\in A} T_{ik} x_{ikh} && (1.1)\\
\text{s.t.} \quad & \sum_{h\in H} \sum_{k\in N} x_{ikh} = 1 \qquad \forall i \in I, && (1.2)\\
& \sum_{k\in N} x_{ikh} - \sum_{k\in N} x_{kih} = 0 \qquad \forall i \in N, h \in H, && (1.3)\\
& \sum_{h\in H_j} \sum_{k\in I} y_{jkh} - L_j^F t_j \le 0 \qquad \forall j \in J, && (1.4)\\
& y_{ikh} - L^V x_{ikh} \le 0 \qquad \forall (i,k) \in A, h \in H, && (1.5)\\
& \sum_{k\in N} y_{ikh} - \sum_{k\in N} y_{kih} + D_i \sum_{k\in N} x_{ikh} = 0 \qquad \forall i \in I, h \in H, && (1.6)\\
& \sum_{(i,k)\in A} T_{ik} x_{ikh} - L^T v_h \le 0 \qquad \forall h \in H, && (1.7)\\
& x_{jkh} = 0 \qquad \forall j \in J, k \in N, h \in H_t, t \in J\backslash\{j\}, && (1.8)\\
& x_{ikh} \in \{0,1\} \quad \forall (i,k) \in A, h \in H, && (1.9)\\
& y_{ikh} \ge 0 \qquad \forall (i,k) \in A, h \in H, && (1.10)\\
& t_j \in \{0,1\} \quad \forall j \in J, \text{ and} && (1.11)\\
& v_h \in \{0,1\} \quad \forall h \in H. && (1.12)
\end{aligned}
$$

The objective function (1.1) states that the total cost, which includes the fixed cost of the selected facilities, the fixed cost of the vehicles (including the driver cost), and the operating cost of the vehicles, should be minimized. Constraints (1.2) specify that exactly one vehicle must travel from customer node i to some other node. Constraints (1.3) require that a vehicle should enter and leave a node an equal number of times. For customer nodes, constraints (1.2) and (1.3) ensure that if a vehicle enters a customer node, it will leave this node and therefore that each customer node is served exactly once. For facility nodes, the number of visits by any vehicle can exceed one, since a vehicle can cover multiple routes. Constraints (1.4) ensure that the total outbound flows to the customer nodes from each facility do not exceed its capacity. Constraints (1.5) are vehicle capacity constraints and define the relationship between the binary variable x_{ikh} and the flow variable y_{ikh}. Each constraint ensures that flow between any pair of nodes cannot exceed the capacity of a vehicle. Constraints (1.6) require conservation of flow at each customer node; these constraints must be modified for pick-up problems. Constraints (1.5) and (1.6) together ensure that no route violates the vehicle capacity. The combined sets of constraints (1.2), (1.3), and (1.6) ensure that only valid traveling salesman tours that include a facility are formed, as in Nambiar et al. (1981). Constraints (1.7) limit the total time of a vehicle's schedule to the time limit. In this research, we assume that the set of vehicles in two different facilities are disjoint and a vehicle located at facility j can only

visit customer nodes and the facility j during its trip. Constraints (1.8) restrict travel on the arcs originating from a facility to vehicles located at that facility. Constraints (1.9), (1.10), (1.11), and (1.12) are the integrality and non-negativity requirements on the variables.

1.4.2 Set-Partitioning-Based Formulation of LRSP

In this section, we introduce a set-partitioning-based formulation of the LRSP. First, we define the column set constructed to formulate the LRSP.

Pairing Definition

To define the column set, the set-partitioning-based formulation uses the idea of a *pairing*, which we adapt from the pairing concept in the crew scheduling literature (e.g., Desrosiers et al. (1991), Vance et al. (1997), and Anbil et al. (1998)). In the crew scheduling literature, a pairing is a sequence of duty periods and represents a possible schedule for a single crew. In the LRSP context, a pairing is a sequence of routes that represents a possible (feasible) schedule for a single vehicle.

A pairing is feasible if

i. each route included in the pairing starts and ends at the same facility,

ii. for each route in the pairing, the total demand of the customers assigned to the route is at most the vehicle capacity,

iii. the total travel time of the pairing is at most the vehicle time limit,

iv. each customer included in the pairing must be visited only once.

By adding constraint (iv), we prevent the generation of pairings that visit some customer(s) multiple times since such pairings are set to zero in an integer feasible solution of an LRSP.

Figure 1.1 shows two possible pairing constructions. Pairing 1 includes routes 1, 2 and 3; pairing 2 includes routes 4 and 5. Each pairing specifies the routes covered by a vehicle and has an associated cost, which is defined as the sum of the costs of the component routes plus a fixed cost for the vehicle. Since the cost of a multiple-stop route depends on the sequence of nodes visited, the pairing information implicitly includes sequencing information for each route. The cost of a pairing, however, is not affected by the ordering of the routes in the absence of time windows and thus is arbitrary. We will take this fact into consideration to improve the solution algorithms for the subproblem and will not generate symmetric pairings corresponding to different orderings of the same routes.

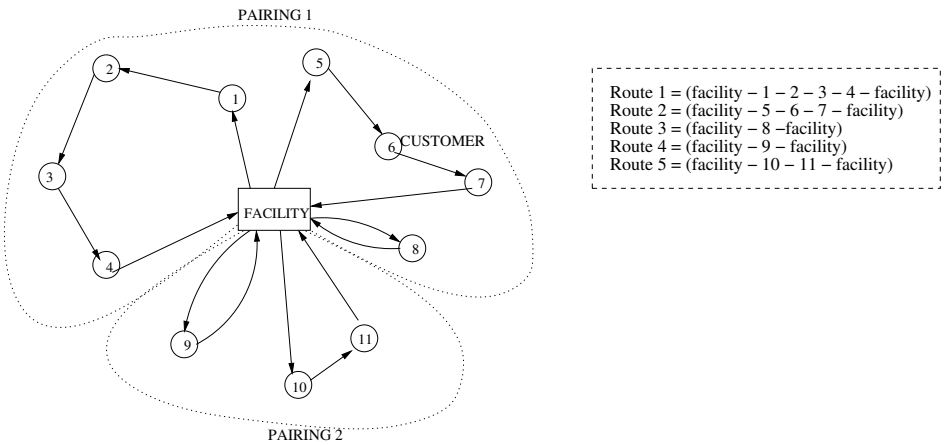

Figure 1.1: Example of pairings

Formulation

Given a set of candidate facility locations and the set of feasible pairings that can be constructed for each candidate facility, the objective of the LRSP is to select a subset of the facilities and an associated subset of pairings in such a way as to minimize the total cost subject to the following constraints:

i. each customer must be covered by exactly one pairing,

ii. the total demand of the customers assigned to a facility must not exceed the capacity of the facility, and

iii. vehicles cannot be associated with a facility unless it is open.

To present the formulation, we define additional parameters and decision variables. Associated with each candidate facility $j \in J$, we let P_j be the set of all feasible pairings for facility j.

Parameters

$$a_{ip} = \begin{cases} 1 & \text{if customer node } i \text{ is in pairing } p \text{ of facility } j, \forall i \in I, p \in P_j, j \in J, \\ 0 & \text{otherwise, and} \end{cases}$$

$$C_p = \text{cost (vehicle fixed cost plus operating cost) of pairing } p \text{ of facility } j, \forall p \in P_j, j \in J.$$

Decision Variables

$$t_j = \begin{cases} 1 & \text{if facility } j \text{ is selected, } \forall j \in J, \\ 0 & \text{otherwise,} \end{cases}$$

$$z_p = \begin{cases} 1 & \text{if pairing } p \text{ is selected for facility } j, \forall p \in P_j, j \in J, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

$$\textbf{(SPP-LRSP)} \text{ Minimize} \quad \sum_{j \in J} C_j^F t_j + \sum_{j \in J} \sum_{p \in P_j} C_p z_p \tag{1.13}$$

$$\text{subject to} \quad \sum_{j \in J} \sum_{p \in P_j} a_{ip} z_p = 1 \qquad \forall i \in I, \tag{1.14}$$

$$\sum_{p \in P_j} \sum_{i \in I} a_{ip} D_i z_p - L_j^F t_j \leq 0 \qquad \forall j \in J, \tag{1.15}$$

$$t_j \in \{0,1\} \quad \forall j \in J, \text{ and} \tag{1.16}$$

$$z_p \in \{0,1\} \quad \forall p \in P_j, j \in J. \tag{1.17}$$

The objective function (1.13) seeks to minimize the total cost, which includes the fixed cost of the selected facilities and the cost of the selected pairings. We note that the cost C_p of a pairing includes both the vehicle fixed cost and variable costs. Constraints (1.14) guarantee that each customer location is served by exactly one pairing. Constraints (1.15) ensure that the total demand of the selected pairings does not exceed the facility capacity. Constraints (1.16) and (1.17) are standard binary restrictions on the variables.

The structure of the SPP-LRS is very similar to the CFLP with single sourcing (CFLPSS). In the CFLPSS, only pairings with one customer are considered and assigned to capacitated facilities. However, in the LRSP, pairings with multiple customers must be taken into account.

1.5 Strengthening the Set-Partitioning-Based Formulation

In this section, we discuss how we can strengthen the set-partitioning-based formulation SPP-LRSP by adding valid inequalities. First, we introduce a lower bound on the total number of facilities required in any integer feasible solution. Let R denote a lower bound on the minimum number of required facilities. Then we can add to SPP-LRSP a constraint to force the number of selected facilities to be at least R:

$$\sum_{j \in J} t_j \geq R. \tag{1.18}$$

For any instance, we can calculate a valid value for R from the demands of the customers and the capacities of the facilities. There are two possible cases:

Case 1. If all of the facilities have the same capacity value, $L_j^F = k \; \forall j \in J$, then

$$R = \left\lceil \frac{\sum_{i \in I} D_i}{k} \right\rceil.$$

Case 2. If at least one facility capacity differs, then a lower bound on the number of required facilities can be calculated using a greedy algorithm: compute the total demand and assign it to the facilities in non-increasing order of their capacities. Let $n = |J|$ and let $(j_1, j_2, .., j_n)$ be an ordering

of the facilities such that $L^F_{j_1} \geq L^F_{j_2} \geq ... \geq L^F_{j_n}$. Then,

$$R = \text{argmin}_{\{l=1..n\}} \left(\sum_{s=1}^{l} L^F_{j_s} \geq \sum_{i \in I} D_i \right).$$

Constraint (1.18) is not necessary in the formulation of the LRSP. However, as a simple lower bound on the number of required facilities, it may eliminate some solutions in which a fractional number of facilities is used, thereby improving the LP relaxation bound. The following proposition can be stated:

Proposition 1.1 *Chvátal-Gomory rank (CG rank) of constraint (1.18) is 1, if* $\left(\frac{\sum_{i \in I} D_i}{k}\right)$ *is fractional in case 1 and* $\left(\frac{\sum_{i \in I} D_i}{L^F_{j_1}}\right)$ *is fractional or strictly less than* R *in case 2.*

Proof Recall that the CG rank of any original inequality and any inequality dominated by a non-negative linear combination of the original inequalities is 0. An inequality which can be generated as a CG inequality based on rank 0 inequalities but is not rank 0 itself has CG rank 1.

Case 1: $L^F_j = k \; \forall j \in J$.

To prove the proposition, we compute the summation of constraints (1.15) over all facilities $j \in J$:

$$\sum_{j \in J} \sum_{p \in P_j} \sum_{i \in I} a_{ip} D_i z_p \quad \leq \quad k \sum_{j \in J} t_j \tag{1.19}$$

Then, if we substitute constraint (1.14) in (1.19) and divide by k, we get:

$$\sum_{j \in J} t_j \quad \geq \quad \frac{\sum_{i \in I} D_i}{k} \tag{1.20}$$

The t_j variables are binary, so $\sum_{j \in J} t_j$ must be integer. Therefore, if we let $R = \left\lceil \frac{\sum_{i \in I} D_i}{k} \right\rceil$, then constraint (1.18) must be valid. If the right hand side of constraint (1.20) is fractional, the constraint (1.18) has CG rank 1. Otherwise, its rank is 0, since it can be defined as the non-negative linear combination of the original inequalities.

Case 2: The facilities have different capacity values.

To generalize this statement for the problems in which the facility capacities might be different in value, we reorder the facilities as $t_{[1]}, t_{[2]}, .., t_{[J]}$ such that $L^F_{[1]} \geq L^F_{[2]} \geq .. \geq L^F_{[J]}$. Then, inequality (1.19) becomes

$$\sum_{j \in J} \sum_{p \in P_j} \sum_{i \in I} a_{ip} D_i z_p \leq \sum_{j \in J} L^F_{[j]} t_{[j]} \tag{1.21}$$

Again, if we substitute in (1.21) using constraint (1.14) and then divide each side by $L^F_{[1]}$, we get:

$$t_{[1]} + \frac{L^F_{[2]}}{L^F_{[1]}} t_{[2]} + .. + \frac{L^F_{[J]}}{L^F_{[1]}} t_{[J]} \geq \frac{\sum_{i \in I} D_i}{L^F_{[1]}} \tag{1.22}$$

If we let

$$R_2 = \left\lceil \frac{\sum_{i \in I} D_i}{L^F_{[1]}} \right\rceil$$

then, by rounding up the coefficients of inequality (1.22), we get:

$$\sum_{j \in J} t_j \geq R_2 \tag{1.23}$$

The value of R_2 is calculated based on the assumption that all facilities have the same capacity, which is the largest capacity value. However, in the calculation procedure of R for problems that have different capacity values (case 2), it is considered that the second facility might not have a capacity as large as the first facility, the third facility might not have a capacity as large as the second facility and so on. Therefore, $R \geq R_2$ and the constraint (1.18) dominates constraint (1.23). If $\left(\frac{\sum_{i \in I} D_i}{L^F_{[1]}} \right)$ is fractional or $R_2 < R$, then the CG rank of constraint (1.18) is 1. □

Due to the similarity in structure between the SPP-LRSP and the CFLP, we might consider adding the set of constraints

$$z_p \leq \quad t_j \quad \forall j \in J, \forall p \in P_j \tag{1.24}$$

In the CFLP, constraints similar to inequality (1.24) are facets of the CFL polytope if the demands of the customers in the constraints are less than the capacity of the facilities (Leung and Magnanti (1989)). In addition, Daskin (1995) remarks that constraints in the CFLP similar to (1.24) considerably improve the LP bound.

Constraints (1.24) ensure that a pairing of a facility can be selected only if that facility is selected. Although constraints (1.24) are implied by constraints (1.15), constraints (1.24) may improve the LP relaxation of the model as in the case of CFLP. However, since there is potentially an exponential number of such constraints, we would like to find a smaller set of constraints to add.

Proposition 1.2 *To strengthen the LP relaxation bound, we can replace constraints (1.24) with*

$$\sum_{p \in P_j} a_{ip} z_p \ \leq \ t_j \ \ \forall j \in J \text{ and } i \in I \tag{1.25}$$

and constraints (1.25) have a CG rank of 1.

Proof Constraints (1.25) are valid for the integer program. In any feasible solution, each demand node $i \in I$ is assigned to exactly one facility, so $\sum_{p \in P_j} a_{ip} z_p$ is 1 for one facility and 0 for all others. For any $j \in J$ such that $\sum_{p \in P_j} a_{ip} z_p$ is 1, t_j will be 1, which corresponds to pairings being selected only for open facilities.

Also, when the coefficients of the variables in inequality (1.25) and constraint (1.24) are compared, it is easy to see that inequality (1.25) dominates constraint (1.24). For example, if we take $z_{p'} \leq t_{j'}$ from constraint set (1.24) for $j = j'$ and $p = p'$, we can compare it with a constraint in set (1.25) for which $j = j'$ and $i = i'$ such that $a_{i'p'} = 1$. That is $z_{p'} + \sum_{p \in P_{j'} \setminus \{p'\}} a_{i'p} z_p \leq t_{j'}$. Because of the z_p terms other than $z_{p'}$ on the left side of the constraint, $\sum_{p \in P_{j'}} a_{i'p} z_p \leq t_{j'}$ dominates $z_{p'} \leq t_{j'}$.

Now we show the rank of the inequality $\sum_{p \in P_j} a_{ip} z_p \leq T_j$ for an arbitrary i and j. For notational convenience, suppose $i = 1$ and $j = 1$.

Let $D(k) = \sum_{i \in I} a_{ik} D_i$ be the total demand of a pairing $k \in P_1$. From constraint sets (1.14) ($\leq$ part of the constraints) and (1.15), we can write:

$$\sum_{\{p \in P_1 | a_{1p}=1\}} z_p + \sum_{\{j \in J \setminus \{1\}\}} \sum_{\{p \in P_j | a_{1p}=1\}} z_p \leq 1 \tag{1.26}$$

$$\sum_{\{p \in P_1 | a_{1p}=1\}} D(p) z_p + \sum_{\{p \in P_1 | a_{1p}=0\}} D(p) z_p - L_1^F t_1 \leq 0 \tag{1.27}$$

Let $D_{min} = min_{\{p \in P_1 | a_{1p}=1\}} \{D(p)\}$. Multiply inequality (1.26) with $(L_1^F - D_{min})$ and add the resulting inequality to the inequality (1.27). Then after dividing the obtained inequality by L_1^F round it to floor:

$$\begin{aligned} \sum_{\{p \in P_1 | a_{1p}=1\}} \left\lfloor \frac{L_1^F - D_{min} + D(p)}{L_1^F} \right\rfloor z_p \; &+ \sum_{\{j \in J/\{1\}\}} \sum_{\{p \in P_j | a_{1p}=1\}} \left\lfloor \frac{L_1^F - D_{min}}{L_1^F} \right\rfloor z_p \\ &+ \sum_{\{p \in P_1 | a_{1p}=0\}} \left\lfloor \frac{D(p)}{L_1^F} \right\rfloor z_p - t_1 \\ &\leq \left\lfloor \frac{L_1^F - D_{min}}{L_1^F} \right\rfloor \end{aligned} \tag{1.28}$$

Also, we simply assume that $D(p) \leq L_1^F \;\; \forall p \in P_1$ since otherwise we can set $z_p = 0$ and remove the variable from the constraint. This is true, because if the total demand of the pairing is greater than the capacity of the facility, this pairing cannot be chosen in a feasible solution for this facility. In inequality (1.28), the coefficients in the first summation are in the range $[1, 2)$, the coefficients in the second summation are in $[0, 1)$, and the coefficients in the third summation are in the range $[0, 1)$ if $D(p) < L_1^F$ for all $p \in P_1$. The right hand side is in the range $[0, 1)$. Thus when we round down the coefficients, we get $\sum_{p \in P_1} a_{1p} z_p \leq t_1$. Since we have obtained inequality (1.25)

by using the original constraints and the CG inequality procedure, the CG rank of the inequality is 1. Also, this inequality is not dominated by any linear combination of the original constraints, thus its CG rank cannot be 0. If $D(p) = L_1^F$ for some $p \in P_1$ in the third summation, when we round down, the coefficient for this variable in (1.28) is 1. Thus, we can add the variables z_p for all $p \in P_1$ and satisfying $D(p) = L_1^F$ and $a_{1p} = 0$ to inequality (1.25) to get a stronger constraint. But if we do not add these variables, the constraint (1.25) will still be valid. $\square$

1.6 Dantzig Wolfe Decomposition (DWD) of G-LRSP

In this section, we prove that the SPP-LRSP can be obtained by applying Dantzig Wolfe decomposition (DWD) to the graph-based formulation of the LRSP (G-LRSP).

1.6.1 DWD in General

The DWD method is used in integer programming to improve the LP relaxation bound. The method reformulates the original problem in such a way that the reformulation yields a tighter bound for the optimal objective function value. To explain the steps of the DWD method in generality, suppose we have an integer program, P that can be partitioned into two sets of constraints.

$$
\begin{aligned}
& \text{Minimize} && c^T x \\
\text{(P)} \quad & \text{subject to} && A^1 x \geq d^1, && (1.29)\\
& && A^2 x \geq d^2, && (1.30)\\
& && x \geq 0, \text{ and integer} && (1.31)
\end{aligned}
$$

where $A^1 \in \mathbb{Q}^{m\times n}$, and $A^2 \in \mathbb{Q}^{l\times n}$ are constraint matrices and $c \in \mathbb{Q}^n$, $d^1 \in \mathbb{Q}^m$ and $d^2 \in \mathbb{Q}^l$ are rational vectors. By relaxing the integrality constraint on x, an LP relaxation bound is obtained. The idea of DWD method is to get possibly a better LP relaxation bound by optimizing the intersection of constraints (1.29) with the convex hull of (1.30) and (1.31). Let $X^2 = \{x \in \mathbb{Z}^n : A^2 x \geq d^2, x \geq 0\}$ be the set of integer points defined by systems (1.30) and (1.31), and $Conv(X^2)$ be the convex hull of X^2. Assuming that the $Conv(X^2)$ is bounded, we can express $Conv(X^2)$ using the convex combination of the extreme points of X^2:

$$Conv(X^2) = \{x \in \mathbb{R}^n | x = \sum_{q\in E} \mathrm{x}^q \theta_q, \sum_{q\in E} \theta_q = 1, \theta \in \mathbb{R}_+^{|E|}\} \tag{1.32}$$

where $\{\mathrm{x}^q\}_{q\in E}$ is the set of extreme points of X^2. Here, we assume that $Conv(X^2)$ is bounded and pursue the review of DWD method under this assumption since this is the case in LRSP. For unbounded case, the definition of $Conv(X^2)$ in (1.32) can be updated by considering the extreme rays of X^2.

ZE: We can use decomposable subproblem as an example, like LRSP?

Then, LP relaxation of P can be reformulated using (1.32):

$$
\begin{array}{lll}
 & \text{Minimize} & c(\sum_{q\in E} \mathrm{x}^q\theta_q) \\
(\mathrm{P_M}) & \text{subject to} & A^1(\sum_{q\in E} \mathrm{x}^q\theta_q) \;\geq\; d^1, \\
 & & \sum_{q\in E} \theta_q \;=\; 1, \\
 & & \theta \;\in\; \mathbb{R}_+^{|E|}.
\end{array}
$$

If we rearrange,

$$
\begin{array}{lll}
 & \text{Minimize} & \sum_{q\in E} (c\mathrm{x}^q)\theta_q \\
(\mathrm{P_M}) & \text{subject to} & \sum_{q\in E} (A^1\mathrm{x}^q)\theta_q \;\geq\; d^1, \qquad (\pi) \qquad (1.33) \\
 & & \sum_{q\in E} \theta_q \;=\; 1, \qquad (\mu) \qquad (1.34) \\
 & & \theta \;\in\; \mathbb{R}_+^{|E|}. \qquad (1.35)
\end{array}
$$

$\mathrm{P_M}$ is called *master* problem. The bound obtained by solving $\mathrm{P_M}$ is typically tighter than the bound obtained by solving the LP relaxation of P. If system (1.30) has integrality property, i.e., all extreme points of system (1.30) are integer, then the LP relaxation value of P is equal to the objective function value of $\mathrm{P_M}$. Otherwise, solution of $\mathrm{P_M}$ yields a better lower bound to the optimal solution value of P.

To find the solution of $\mathrm{P_M}$, we have to explore the set of extreme points, $\{\mathrm{x}^q\}_{q\in E}$. One way of doing that is to enumerate the whole set. However, in most cases enumeration of the whole set is not possible because of the size of set E. Other way of finding the solution of $\mathrm{P_M}$ is to use *column generation algorithm*. Column generation is an approach for solving LPs for which we want to avoid explicitly enumerating all columns. By solving a sequence of linear programs and dynamically generating columns eligible to enter the basis, such an algorithm implicitly considers all columns but generates only those that may improve the objective function. The basic idea is to determine a subset of the variables that includes those that are nonzero in some optimal solution. By solving the LP including only this subset of columns rather than all possible columns, we can obtain the optimal LP solution.

In each iteration of the column generation algorithm, a restricted version of $\mathrm{P_M}$ (RMP) that includes only a subset of columns is solved. The column generation algorithm feeds the dual information from the solution of the RMP to an optimization model called *pricing* problem (or *subproblem*). The pricing problem searches columns in set X^2 with an objective function equal to the

reduced cost of a variable in $\mathrm{P_M}$. The pricing problem can be formulated as follows:

$$\begin{aligned} \text{Minimize} \quad & z_{rc} = (c - \pi A^1)x - \mu \\ \text{subject to} \quad & A^2 x \geq d^2 \\ & x \geq 0, \text{ and integer} \end{aligned}$$

where π is the dual vector corresponds to system (1.33) and μ is the dual variable for constraint (1.34). The objective of the pricing problem is to either identify new columns to add to the RMP or prove optimality of the current solution. From the theory of linear programming, we know that if every variable has a non-negative reduced cost, then the solution is optimal. If $z_{rc} < 0$ for a given (π, μ), then a column in $\{\mathrm{x}^q\}_{q \in E}$ is found and can be added to the RMP. Then, the RMP is reoptimized. The algorithm continues until $z_{rc} \geq 0$ for a given dual vector.

With column generation algorithm, we can find a lower bound for the optimal integer solution value for P. To find the optimal integer solution using model $\mathrm{P_M}$, we have to force the integrality of variables. This can be done by adding integrality restriction to variable θ in $\mathrm{P_M}$.

ZE: Mention branch and price and say will be explained how to solve integer master problem?

1.6.2 DWD of G-LRSP

Back to the G-LRSP, to find the DWD of the problem, we decompose the problem into two subsystems: master problem and subproblem. The *master problem* is defined by constraints (1.2) and (1.4), while the *subproblem* is defined by constraints (1.3), (1.5), (1.6), (1.7), (1.8), (1.9), (1.10), (1.11), and (1.12). Remember that the vehicle index set, H is equal to the union of the vehicle indices located at each facility, i.e., $H = \cup_{j \in J} H_j$. Thus, all constraints of the subproblem are indexed by the set J and it is therefore immediate that the subproblem decomposes by facility.

For each candidate facility $j \in J$, the set of integer solutions of the decomposed subproblem can be represented by the set of vectors defined as

$$\begin{aligned} \{ \ (x,t,v)^j \in \mathbb{Z}^{(|N| \times |N| \times |H_j|) \times \{0,1\} \times (|H_j|)}, (y)^j \in \mathbb{R}^{(|N| \times |N| \times |H_j|)} \mid \ (x,y,t,v)^j \text{ satisfies} \\ (1.3)_j, (1.5)_j, (1.6)_j, (1.7)_j, (1.8)_j, (1.9)_j, (1.10)_j, (1.11)_j, (1.12)_j \}, \end{aligned} \tag{1.36}$$

which is described only by those constraints associated with facility j and specifies values only for those variables associated with vehicles assigned to facility j. Hence, the index j for the constraints represents the set of constraints associated with facility j. System (1.36) is bounded for all facilities. We let E be a set indexing the extreme points of all of the above sets, with E_j the indices for vectors associated with facility j only, so that $E = \cup_{j \in J} E_j$. For a facility j and an index $q \in E_j$, the corresponding member $(\mathrm{x}^q, \mathrm{y}^q, \mathrm{t}^q, \mathrm{v}^q)$ of the above set is then a vector with the

following interpretation:

$$
\begin{aligned}
\mathrm{x}^q_{ikh} &= \begin{cases} 1 & \text{if vehicle } h \text{ travels on arc } (i,k) \text{ in solution } q, \forall (i,k) \in A, \text{ and } h \in H_j \\ 0 & \text{otherwise} \end{cases} \\
\mathrm{y}^q_{ikh} &= \text{Load of vehicle } h \text{ on arc } (i,k) \text{ in solution } q, \forall (i,k) \in A, \text{ and } h \in H_j \\
\mathrm{t}^q &= \begin{cases} 1 & \text{if facility } j \text{ is open in solution } q \\ 0 & \text{otherwise} \end{cases} \\
\mathrm{v}^q_h &= \begin{cases} 1 & \text{if vehicle } h \text{ is used in solution } q, \forall h \in H_j \\ 0 & \text{otherwise} \end{cases}
\end{aligned}
$$

Note that the variable t indicating whether the facility is open does not appear in any of the linear constraints of the subproblem and can hence be set to either 0 or 1 without affecting feasibility.

Following the DWD methodology, we can reformulate master problem using the convex combination of the extreme points of the subproblem. Because the subproblem decomposes as described above, solutions to the original problem can be seen as vectors obtained by "recomposing" convex combinations of the members of E_j for each $j \in J$. In other words, any solution (x, y, t, v) to the master problem can be written as:

$$
\begin{aligned}
x_{ikh} &= \sum_{q \in E} \mathrm{x}^q_{ikh} \theta_q && \forall (i,k) \in A, h \in H, && (1.37) \\
y_{ikh} &= \sum_{q \in E} \mathrm{y}^q_{ikh} \theta_q && \forall (i,k) \in A, h \in H, && (1.38) \\
t_j &= \sum_{q \in E_j} \mathrm{t}^q \theta_q && \forall j \in J, && (1.39) \\
v_h &= \sum_{q \in E} v^q_h \theta_q && \forall h \in H, && (1.40) \\
\sum_{q \in E_j} \theta_q &= 1 && \forall j \in J, && (1.41) \\
\theta_q &\geq 0 && \forall q \in E. && (1.42)
\end{aligned}
$$

ZE: change it to E_j?

Using (1.37) - (1.42), we can then formulate the master problem as:

$$
\begin{aligned}
\textbf{(MDW)} \text{ Minimize } & \sum_{q \in E} \tilde{C}_q \theta_q \\
\text{s.t.} \quad & \sum_{q \in E} b_{iq} \theta_q = 1 \quad \forall i \in I, && (1.43)\\
& -\sum_{q \in E_j} \sum_{i \in I} b_{iq} D_i \theta_q + L_j^F \sum_{q \in E_j} \mathrm{t}^q \theta_q \geq 0 \quad \forall j \in J, && (1.44)\\
& \sum_{q \in E_j} \theta_q = 1 \quad \forall j \in J, && (1.45)\\
& \theta_q \geq 0 \quad \forall q \in E, && (1.46)
\end{aligned}
$$

where

$$
\begin{aligned}
b_{iq} &= \sum_{h \in H} \sum_{k \in N} \mathrm{x}^q_{kih}, && \forall i \in I, q \in E,\\
\tilde{C}_q &= C_q^F \mathrm{t}^q + C^V \sum_{h \in H} \mathrm{v}^q_h + C^O \sum_{i \in N} \sum_{k \in N} \sum_{h \in H} T_{ik} \mathrm{x}^q_{ikh}, && \forall q \in E,
\end{aligned}
$$

where $C_q^F = C_j^F$ when $q \in E_j$ for $j \in J$. Here, b_{iq} can be interpreted as the number of times customer i is visited in solution q, and $\tilde{C}_q$ is the cost of solution q (including facility fixed cost and vehicle fixed costs) for all $q \in E$.

To solve MDW, we can use column generation algorithm. Let $(\pi^T, \mu^T, \alpha^T)^T$ be a dual vector such that π is the dual variable associated with constraint (1.43), μ is the dual variable associated with (1.44), and α is the dual variable associated with (1.45). The reduced cost of a column $q \in E_j$ for some $j \in J$ corresponds to variable θ_q is

$$
\hat{C}_q = \tilde{C}_q - \sum_{i \in I} b_{iq} \pi_i + \left(\sum_{i \in I} b_{iq} D_i - L_j^F \mathrm{t}^q \right) \mu_j - \alpha_j \tag{1.47}
$$

Using the definitions of $\tilde{C}$, we can rewrite (1.47).

$$
\begin{aligned}
\hat{C}_q \;=\; & (C_j^F - L_j^F \mu_j) \mathrm{t}^q + C^V \sum_{h \in H_j} \mathrm{v}^q_h - \\
& \sum_{h \in H_j} \sum_{k \in N} \Big(\sum_{i \in I} (C^O T_{ki} - \pi_i + \mu_j D_i) \mathrm{x}^q_{kih} + C^O T_{kj} \mathrm{x}^q_{kjh} \Big) - \alpha_j
\end{aligned} \tag{1.48}
$$

The column generation algorithm searches for the columns with most negative reduced cost defined by (1.48) over the set of extreme points for facility j defined in (1.36). This problem is called subproblem for facility $j \in J$, and can be formulated explicitly as follows:

$$
\begin{aligned}
\textbf{(SP}_j\textbf{)}\ \text{Minimize} \quad & \hat{f}_j t_j + C^V \sum_{h \in H_j} v_h - \sum_{h \in H_j} \sum_{k \in M} \sum_{i \in M} \hat{c}_{ki} x_{kjh} - \alpha_j \\
& \sum_{k \in M} x_{ikh} - \sum_{k \in M} x_{kih} = 0 \qquad \forall i \in M, h \in H_j, && (1.49)\\
& y_{ikh} - L^V x_{ikh} \leq 0 \qquad \forall i \in M, k \in M, h \in H_j, && (1.50)\\
& \sum_{k \in M} y_{ikh} - \sum_{k \in M} y_{kih} + D_i \sum_{k \in M} x_{ikh} = 0 \qquad \forall i \in I, h \in H_j, && (1.51)\\
& \sum_{i \in M} \sum_{k \in M} T_{ik} x_{ikh} - L^T v_h \leq 0 \qquad \forall h \in H_j, && (1.52)\\
& x_{ikh} \in \{0,1\} \qquad \forall i \in M, k \in M, h \in H_j, && (1.53)\\
& y_{ikh} \geq 0 \qquad \forall i \in M, k \in M, h \in H_j, && (1.54)\\
& t_j \in \{0,1\}, \quad \text{and} && (1.55)\\
& v_h \in \{0,1\} \qquad \forall h \in H_j, && (1.56)
\end{aligned}
$$

where $M = I \cup \{j\}$ and

$$
\begin{aligned}
\hat{f}_j &= C_j^F - L_j^F \mu_j, \\
\hat{c}_{ki} &= \begin{cases} C^O T_{ki} - \pi_i + \mu_j D_i & \text{if } k \in M, \text{and } i \in I, \\ C^O T_{kj} & \text{if } k \in I. \end{cases}
\end{aligned}
$$

The objective of each of the resulting single-facility subproblems SP$_j$ is to generate sets of paths with least-cost assigned to *all vehicles* located at the facility, though without the constraint that customers appear exactly once. Because of this condition, we avoid calling paths as routes for now.

To obtain integer feasible solutions to LRSP using MDW, we have to force integrality restriction for the variable θ. Thus, we add $\theta_q \in \{0,1\}$ for all $q \in E$ to MDW.

Now we prove that solving integer MDW is equal to solving SPP-LRSP. The similar forms of SPP-LRSP and integer MDW should now be evident, but to rigorously show their equivalence, we need to dissect the relationship between set E_j (used to define MDW) and P_j (used to define SPP-LRSP) for a given facility $j \in J$.

A member of set E_j, a feasible solution to SP$_j$, consists of a *collection* of sets of paths (starting at facility j and ending at facility j) assigned to all vehicles located at facility j. It may be possible that some customers are visited multiple times by a single vehicle as well as different vehicles.

As explained in Section 1.4.2, a member of set P_j, on the other hand, is a single set of routes (pairing) that can be assigned to any vehicle at facility j. A customer can only appear at most once in a pairing assigned to a vehicle.

Some members of E_j can be constructed by associating at most $|H_j|$ members of set P_j and some number of empty pairings (zero vectors representing vehicles that are not used) to the vehicles assigned to facility j. The members of E_j that include sets of paths assigned to a single vehicle and visiting some customers multiple times can not be constructed using members of P_j.

Now, by utilizing the integrality requirements from the original problem and carefully eliminating the indices of symmetric solutions from E, we get a much smaller set that we will show is in one-to-one correspondence with collections of members of P_j of cardinality at most $|H_j|$. We proceed as follows:

1. First, to prove the necessity of adding $\theta_q \in \{0, 1\}$ for all $q \in E$ to MDW, we let $\hat{\theta}$ represent a solution to MDW for which the corresponding solution in the original space obtained by applying equations (1.37)-(1.40) is feasible for the original problem G-LRSP. From (1.37) and (1.9), along with the fact that x^q_{ikh} is binary for $q \in E$, we get that $\mathrm{x}^{q_1} = \mathrm{x}^{q_2}$ for all q_1 and q_2 such that $\hat{\theta}_{q_1} > 0$ and $\hat{\theta}_{q_2} > 0$ in the solution to MDW. In other words, all members q of E_j with $\hat{\theta}_q > 0$ must correspond to set of paths visiting exactly the same customers in exactly the same order. Similar results follow for t^q and v^q from (1.39), (1.40), and (1.11), (1.12). Hence, we must in fact have $\hat{\theta}_q = 1$ for exactly one $q \in E_j$ for each $j \in J$ and so $\theta_q \in \{0, 1\}$ for all $q \in E$.

2. Let $q \in E$ a member in which a vehicle visits at least one customer multiple times. Thus, $\mathrm{x}^q_{ik_1h_1} = 1$ and $\mathrm{x}^q_{ik_2h_1} = 1$ for some customer $i \in I$, vehicle $h_1 \in H$ and nodes $k_1, k_2 \in N$ such that $k_1 \neq k_2$. This means that $\sum_{k \in N} \mathrm{x}^q_{ikh_1} > 1$ and therefore $b_{iq} > 1$. Because of constraint (1.43) and the binary restriction for θ_q, a solution to integer MDW for which the corresponding solution in the G-LRSP is feasible, θ_q must be 0. Therefore, we can remove such members from E. In order not to generate these members, we can add the following constraint to SP_j.

$$\sum_{k \in M} x_{ikh} \leq 1 \quad \forall i \in N, h \in H_j \tag{1.57}$$

   When we add constraint (1.57) to SP_j, the feasible solutions are restricted to sets of routes each of which assigned to a vehicle and includes any customer at most once. Therefore, based on the definition of a pairing in Section 1.4.2, we can say that the feasible solutions to augmented SP_j are sets of pairings.

3. As the vehicles associated with a given facility j are identical, a set of pairings can be assigned to vehicles in any arbitrary order. Hence, we obtain different members of E_j that are all equivalent from the standpoint of both feasibility and cost. To eliminate superfluous equivalent members of E_j, we divide the members of set E_j into equivalence classes, where two members of E_j are considered equivalent if the set of customers assigned to the facility

and the partition of that set of customers defined by the pairings (set of routes assigned to a vehicle) are identical. It is clear that any two members of E_j that are equivalent by this definition will have exactly the same impact on both cost and feasibility.

In each equivalence class of E_j there can be up to $|H_j|!$ members depending on the number of vehicles serving to non zero pairings in the solutions. For example, let q_1 and q_2 be two members of E_j and be in the same equivalence class. Then there exists a pair of vehicles h_1 and h_2 such that $\mathrm{x}^{q_1}_{ikh_1} = \mathrm{x}^{q_2}_{ikh_2}$, $\mathrm{x}^{q_1}_{ikh_2} = \mathrm{x}^{q_2}_{ikh_1}$ $\forall i, k \in M$ and at least one of h_1 and h_2 is serving a non zero pairing.

We form equivalence classes from which all but one member may safely be eliminated from E_j.

4. Index q can be removed from E_j if there exist vehicles $h_1, h_2 \in H_j$ such that $\mathrm{x}^q_{ikh_1} = \mathrm{x}^q_{ikh_2}$ $\forall i, k \in M$, where $\mathrm{x}^q_{ikh_1} > 0$ for some $i, k \in M$ (i.e., this is not an empty route). In this case, vehicles h_1 and h_2 define exactly the same sets of routes, which means that $b_{iq} > 1$ for some $i \in I$. Because of constraint (1.43), such a solution must have $\hat{\theta}_q = 0$.

5. From (1.39) and (1.44), we can conclude that if $\hat{\theta}_q = 1$ and we have $\mathrm{x}^q_{ikh} > 0$ for some $i, k \in M$ and $h \in H$, then t^q must be 1. Hence, we can eliminate any solution for which $\mathrm{t}^q = 0$ that does not correspond to a zero solution (i.e., closed facility). All of this allows us to rewrite (1.44) in the form

$$-\sum_{q \in E_j} \sum_{i \in I} b_{iq} D_i \theta_q + L^F_j \sum_{q \in E_j} \theta_q \geq 0 \;\; \forall j \in J. \tag{1.58}$$

6. A member $q \in E_j$ can be removed if there exists a vehicle $h \in H_j$ such that $\mathrm{v}^q_h = 1$ and $\mathrm{x}^q_{ikh} = 0$ for all $i, k \in M$ (meaning that zero vector is assigned to vehicle h even though we pay the fixed cost of that vehicle). In an optimal solution to integer MDW, $\theta_{q'} \geq \theta_q = 0$ where $q, q' \in E_j$, $\mathrm{x}^q = \mathrm{x}^{q'}$, $\mathrm{y}^q = \mathrm{y}^{q'}$, $\mathrm{t}^q = \mathrm{t}^{q'}$, and $\mathrm{v}^q = \mathrm{v}^{q'}$ except that $\mathrm{v}^{q'}_h = 0$ since $\tilde{C}_q \geq \tilde{C}_{q'}$.

7. Finally, we can remove index q corresponding zero column (i.e., $\mathrm{x}^q = 0$, $\mathrm{y}^q = 0$, $\mathrm{v}^q = 0$, and $\mathrm{t}^q = 0$) from E_j. Thus, we can update constraint (1.45):

$$\sum_{q \in E_j} \theta_q \leq 1 \quad \forall j \in J, \tag{1.59}$$

If we restrict set E_j according to the rules described above and call the restricted set $\bar{E}_j$ for each $j \in J$, then we can finally conclude the following.

Proposition 1.3 *There is a one-to-one correspondence between nonempty subsets of P_j with cardinality less than $|H_j|$ and members of $\bar{E}_j$.*

Proof The proof follows easily from the definition of sets P_j and $\bar{E}_j$, and the restriction rules.

□

By replacing set E_j with $\bar{E}_j$ for all $j \in J$ in MDW, as well as replacing (1.44) with (1.58) and (1.45) with (1.59), and adding the constraint $\theta_q \in \{0, 1\}$ for all $q \in E$, we obtain a new (equivalent) formulation MDW'. Then, we finally have the equivalence of SPP-LRSP and integer MDW as follows.

Proposition 1.4 *There is a one-to-one correspondence between solutions to SPP-LRSP and solutions to MDW' such that corresponding solutions also have the same objective function value. Thus, SPP-LRSP, MDW', and integer MDW are all equivalent.*

Proof First, we construct a one-to-one mapping to establish the correspondence between solution sets of SPP-LRSP and MDW'. We define mapping $\Psi : (t, z) \to \theta$. Let $(\bar{t}, \bar{z})$ be a feasible solution to SPP-LRSP where $\bar{t}_j = 1$ if facility j is open, $\bar{z}_p = 1$ if pairing p is selected and assigned to a vehicle for all $p \in P_j$ and $j \in J$, then $\bar{\theta} = \Psi(\bar{t}, \bar{z})$ be the corresponding solution in MDW' where $\bar{\theta}_q = 1$ if solution q is selected for all $q \in \bar{E}_j$, $j \in J$.

Since we can decompose variables in both formulations by facility, we can decompose mapping Ψ by facility and define mapping Ψ_j indexed by facility. $\Psi = (\Psi_j)_{j \in J}$ and $\bar{\theta}_{\bar{E}_j} = \Psi_j(\bar{t}_j, \bar{z}_{P_j})$ where notation x_S represents a vector composed of the components of vector x indexed with set S. Let $\bar{J} = \{j \in J | \bar{t}_j = 1\}$, $\bar{P}_j = \{p \in P_j | \bar{z}_p = 1\}$ $\forall j \in \bar{J}$ and $\bar{P}_j = \emptyset$ $\forall j \in J \backslash \bar{J}$ (because of constraint (1.15)). For some facility $j \in J$, we have three types of solutions, which changes the definition of $\bar{\theta}_{\bar{E}_j} = \Psi_j(\bar{t}_j, \bar{z}_{P_j})$.

i. Zero solution, $\bar{t}_j = 0$ and $\bar{P}_j = \emptyset$, thus $\bar{z}_p = 0$ for all $p \in P_j$. Then, $\Psi_j(\bar{t}, \bar{z}_{P_j}) = \bar{\theta}_{\bar{E}_j} = 0$, i.e., $\bar{\theta}_q = 0$ for all $q \in \bar{E}_j$ (since we removed zero solution from $\bar{E}_j$ in reduction rule 7).

ii. Nonzero but empty solution, $\bar{t}_j = 1$ and $\bar{P}_j = \emptyset$. Then, $\bar{\theta}_1 = 1$ such that $\mathrm{x}^1 = 0$, $\mathrm{y}^1 = 0$, $\mathrm{v}^1 = 0$ and $\mathrm{t}^1 = \bar{t}_j = 1$, and $\bar{\theta}_q = 0$ for all $q \in \bar{E}_j$ and $q \neq 1$.

iii. $\bar{t}_j = 1$ and $\bar{P}_j \neq \emptyset$. For each facility we can choose at most $|I| = |H_j|$ pairings, i.e., $\sum_{p \in P_j} \bar{z}_p = |\bar{P}_j| \leq |H_j|$. Based on Proposition 1.3, there is one member $k \in \bar{E}_j$ (in which $|\bar{P}_j|$ vehicles assigned one pairing from set $\bar{P}_j$, $|H_j| - |\bar{P}_j|$ vehicles assigned zero solution, and $\mathrm{t}_j^k = 1$) corresponds to subset $\bar{P}_j$. Thus, the corresponding solution in MDW' is $\bar{\theta}_k = 1$ and $\bar{\theta}_q = 0$ for all $q \in \bar{E}_j$ and $q \neq k$. We use notation $k \rightleftharpoons \bar{P}_j$ to define this one-to-one correspondence between member k and pairing set $\bar{P}_j$ defined in Proposition 1.3.

It follows that Ψ_j is one-to-one from construction, therefore Ψ is also one-to-one. It is clear that we can easily construct the one-to-one reverse mapping, $(\bar{t}_j, \bar{z}_{P_j}) = \Psi_j^{-1}(\bar{\theta}_{\bar{E}_j})$ in a similar way.

From the construction of Ψ_j for each solution type, we can write the following relations between two formulations' variables and parameters.

$$t_j = \sum_{q \in \bar{E}_j} \theta_q \qquad \forall j \in J, \tag{1.60}$$

$$b_{iq} = \sum_{p \in S: q \rightleftharpoons S} a_{ip} \qquad \forall i \in I, \forall q \in \bar{E}_j, \tag{1.61}$$

$$\tilde{C}_q = C_j^F + \sum_{p \in S: q \rightleftharpoons S} C_p \quad \forall q \in \bar{E}_j, \forall j \in J \tag{1.62}$$

If $(\bar{t}, \bar{z})$ is feasible, then $\bar{\theta} = \Psi(\bar{t}, \bar{z})$ is also feasible. This can be seen from the similarity between the structures of SPP-LRSP and MDW' and from (1.60) and (1.61). Finally, the equivalence of objective function values follows from (1.62).

□

Proposition 1.4 proved that applying Dantzig Wolfe decomposition to G-LRSP, is equal to solving SPP-LRSP.

1.7 Comparing G-LRSP and SPP-LRSP

G-LRSP is a three-index commodity flow formulation. For the related problems of LRP and VRP, the literature contains two main types of graph-based models, vehicle flow and commodity flow. In both formulations, integer variables represent the number of times a vehicle traverses a given arc or edge in an underlying graph. In commodity flow formulations, an additional variable representing the number of units of a given commodity transported along the arc is present. In general, vehicle flow models contain an exponential number of subtour elimination constraints, whereas commodity flow models, with the help of additional flow variables, use a polynomial number of constraints to eliminate subtours. For the LRP and the VRP, two-index vehicle flow formulations are preferred in solution algorithms based on branch-and-cut since these formulations include a smaller number of variables and their LP relaxations yield better bounds. However, it is not possible to formulate the LRSP using variables with only two indices because of the time constraint for the vehicles. In this context, we require a three-index commodity flow model. Because of the facility and vehicle capacity restrictions, we do not know a priori the number of vehicles required at each facility, though the number is clearly at most $|I|$. Thus, this formulation can require up to O($|I|^4$) binary variables and O($|I|^4$) constraints, which can be as many as 6 million for an instance with 50 customers.

On the other hand, SPP-LRSP includes O($|I|$) number of constraints while the number of variables depends on the size of the pairing sets (i.e., $|\bigcup_{j \in J} P_j|$), which can be exponentially large.

However, most of these columns are equal to zero in an optimal solution. Therefore, intelligent enumeration techniques which may use some logical conditions, information from good solutions, and information from experiences can be developed to avoid generating all possible columns. In addition, the disadvantage of having extremely large number of variables can be overcome by using column generation algorithm that explores all columns implicitly and generates only those that may improve the objective function.

Observe that the three-index formulation G-LRSP exhibits a high degree of symmetry under the assumption that the vehicle fleet assigned to each facility is homogeneous. This is due to the fact that the assignment of pairings to a specific vehicle is essentially arbitrary, i.e., the cost of a given solution to G-LRSP is invariant under permutation of the indices assigned to specific vehicles. In addition, from the definition of the variables in formulation G-LRSP, symmetric distance matrix results in solutions with same cost but including routes constructed in different directions. A solution algorithm such as a branch-and-bound or a branch-and-cut based on G-LRSP may perform poorly because of the inherent symmetry in the formulation.

The SPP-LRSP formulation eliminates the symmetry that arises in G-LRSP because of the identical vehicles. In SPP-LRSP, we enumerate one set of feasible pairings that can be assigned to any vehicle located at a facility instead of duplicating this set for each vehicle located at a facility. Furthermore, while generating the pairings using a subproblem, depending on the solution procedure, we can use undirected links between nodes in case of symmetric distance matrix. Therefore, we can remove direction in the routes.

The lower bound obtained by solving the LP relaxation of the SPP-LRSP is stronger than the bound obtained by solving the LP relaxation of the G-LRSP. This can be demonstrated both theoretically and empirically. Let z_{LP}^{G} be the LP relaxation of formulation G-LRSP and z_{LP}^{SPP} be the LP relaxation of formulation SPP-LRSP. In Proposition 1.4 we proved that MDW' obtained by applying Dantzig Wolfe Decomposition to G-LRSP is equivalent to SPP-LRSP. Therefore using DWD methodology we can theoretically compare the LP relaxation bounds of G-LRSP and SPP-LRSP. Let $z_{LP}^{MDW'}$ be the LP relaxation of formulation MDW'. Then, we have the following result.

Proposition 1.5 $z_{LP}^{G} \leq z_{LP}^{MDW'} = z_{LP}^{SPP}$.

Proof z_{LP}^{G} is obtained by optimizing the objective function (1.1) on the polyhedron defined by constraints (1.2) - (1.8) (G-LRSP without the integrality constraints).

On the other hand, $z_{LP}^{MDW'}$ is obtained by optimizing the objective function (1.1) on the polyhedron defined by the intersection of constraints (1.2) and (1.4) with the convex hull of of the solutions that are constructed with composing the points generated by solving subproblem (1.36) for all facilities, which is the convex hull defined by (1.3), (1.5), (1.6), (1.7), (1.8), (1.9), (1.10), (1.11), and (1.12).

The proof follows from the fact that the subproblems do not have the integrality property, i.e., the solution is unchanged when the integrality restriction is removed. The polyhedron defined by

the constraints (1.3), (1.5), (1.6), (1.7), (1.8) is greater or equal to the convex hull of the same set of constraints. □

The relation in Proposition 1.5 can also be demonstrated emprically. In Section **ZE: REFER COMPUTATIONAL RES. FOR LP RELAX.** , we present computational results that compare the LP relaxation bounds obtained by solving SPP-LRSP and G-LRSP.

Finally, while we are searching for cutting planes valid for the model, we can see some differences between formulation G-LRSP and formulation SPP-LRSP. In formulation G-LRSP, the variables indexed with vehicle h which enables us to search cutting planes associated with either a single vehicle or a set of vehicles. We can multiply a cut generated for a single vehicle h for all vehicles in set H. This can prevent jumps between the symmetrical solutions because of the homogeneous vehicle fleet. However, in formulation SPP-LRSP, a pairing variable for a facility is associated with any vehicle located at the facility, thus we can not differentiate the vehicles. We can only use cutting planes associated with all vehicles located at a facility. These cutting planes have the same coefficient for all pairings (representing a single vehicle) located at a facility. Solution process for SPP-LRSP becomes more complicated if cutting planes only including a subset of pairings, and possibly with different coefficients for the variables representing pairings are added. Therefore, the structure of the cutting planes searched is more restrictive for SPP-LRSP. However, one advantage of SPP-LRSP is that some cutting planes valid for formulation G-LRSP are already implied by formulation SPP-LRSP. This is evident from the Proposition 1.5 and the definition of the columns of SPP-LRSP. This will further be investigated in Section **ZE: REFER CUTS SECTION** .

1.8 Complexity of the LRSP

The LRSP is NP-hard. To show this, we consider an instance of the LRSP in which there is at most one vehicle at each facility, there are no facility and vehicle fixed costs, and the vehicle time limit is unrestricted. Such a special case is equivalent to an instance of the MDVRP, which was shown to be NP-hard by Lenstra and Kan (1981), and Bodin et al. (1983). The MDVRP aims to find a minimum-cost set of vehicle routes, each one starting from a depot and terminating at the same depot. Each customer must be served exactly once and there are specified lower and upper bounds for the number of capacitated vehicles at each depot. In the created LRSP instance, location decision and facility costs are eliminated by setting the fixed cost of facilities to zero, and scheduling part is eliminated by setting the time limit to infinity. Solution of this instance results a set of routes with minimum total operating cost. This solution can easily be translated to a solution for the MDVRP. In the solution of the MDVRP, the allocation of the customers to the facilities, the vehicle routes and the total cost are the same as in the solution of the LRSP. Since we have shown that the MDVRP is a special case of the LRSP and the MDVRP is NP-hard, we can conclude that the LRSP is also

1.8. COMPLEXITY OF THE LRSP

NP-hard.

Chapter 2

Branch and Price Algorithm for the LRSP

2.1 Introduction

In this chapter, we describe our solution methodology for LRSP. Although it would be possible to develop a branch-and-bound or a branch-and-cut algorithm for G-LRSP we suspect that such an approach would not yield positive results, given the inherent symmetry in the formulation, the weak LP relaxation bound and the known difficulty associated with solving related problems. Instead, we use the set-partitioning-based formulation which is similar to those that have been successful for other hard combinatorial problems.

As we have shown that adding both constraints (1.18) and (1.25) to SPP-LRSP strengthen the formulation, to develop our solution methodology, we use an augmented SPP-LRSP formulation, denoted AUG-SPP-LRSP, which is comprised of the objective function (1.13) and constraints (1.14), (1.15), (1.16), (1.17), (1.18), and (1.25).

In AUG-SPP-LRSP, each column corresponds to a feasible pairing. For medium- and large-sized instances, the number of feasible pairings can be extremely large, making it unlikely that we can efficiently solve instances that explicitly include all feasible pairings. Instead, we have developed a branch-and-price algorithm in which we dynamically generate only a subset of the feasible pairings at each node of the tree.

2.2 Description of Solution Methodology

Branch-and-price that combine column generation with branch-and-bound algorithms has become a standard approach for solving large-scale integer programming problems, especially those formulated using set-partitioning-based models. Barnhart et al. (1998) discuss the general methodology and the many challenges in developing a branch-and-price algorithm. The algorithm has been applied to a variety of problems, including various vehicle routing problems (Fukasawa et al. (2006)), location routing problems (Berger et al. (2007)) and airline crew scheduling problems (Vance et al. (1997)), all of which are relevant in the context of the LRSP.

ZE: Need more details about the literature?

The basic idea of a branch-and-price algorithm is to solve an appropriate LP at each node of the branch-and-bound tree using column generation. Column generation is an approach for solving LPs for which we want to avoid explicitly enumerating all columns. The basic idea is to determine a subset of the variables that includes those that are nonzero in some optimal solution. By solving the LP including only this subset of columns rather than all possible columns, we can obtain the optimal LP solution.

Column generation algorithms have been applied to a wide variety of large scale optimization problems, such as vehicle routing, crew scheduling, cutting stock, lot sizing and machine scheduling. Lübbecke and Desrosiers (2005) list other problems to which column generation has been applied.

Here, we explain the terminology and the main steps of the column generation algorithm. In each iteration of the column generation procedure, we work with a restricted version of the LP relaxation; this *restricted master problem (RMP)* includes only a subset of the variables. We solve the RMP and use the associated *dual variable information* to formulate the *pricing problem* or *subproblem*. The objective of the pricing problem is to either identify new columns to add to the RMP or prove optimality of the current solution. For given dual variables, the objective function of the pricing problem is equal to the reduced cost function of a variable that we want to generate. The constraints of the pricing problem define the set of feasible columns corresponds to variables. From the theory of linear programming, we know that if every variable has a non-negative reduced cost, then the solution is optimal. If the pricing problem identifies at least one column with negative cost, then we add one or more columns to the RMP and reoptimize. Otherwise, the current solution is optimal for the full problem.

A column generation algorithm (COLGEN) for solving the LP relaxation of a formulation includes the following four steps.

COLGEN:

0. Generate an initial set of columns that contains a feasible solution. Build a RMP.
1. Solve the RMP and obtain dual variable information.
2. Using the dual variables, form a pricing problem.
3. Solve the pricing problem.
   i. If the solution specifies a column with negative reduced cost, add the new column to the RMP and go to step 1.
   ii. Otherwise, the solution found in step 1 is an optimal solution for the LP relaxation. Stop.

ZE: Mention challenges of COLGEN? One challenge of applying COLGEN is to solve the pricing problem which is generally an integer programming model. One call to algorithm COLGEN may require many calls to the oracle that solves pricing problem. Therefore, an efficient solution method for pricing problem is crucial for the efficiency of the whole algorithm. In order to improve the performance of COLGEN, one may prefer solving relaxations of the pricing problem which are easier to solve. However, replacing real pricing problem with a relaxed model yields a solution value which is a lower approximation (for minimization problems) of the real LP relaxation solution. Another approach is to solve pricing problem heuristically. Indeed, step 3 of COLGEN does not require to find a column with minimum cost (i.e., optimal solution to pricing problem) in intermediate iterations. However, to terminate the COLGEN in step 3 (ii) and conclude that the LP relaxation is optimal, one must find a column with minimum cost at final iteration of COLGEN, thus

one must solve the pricing problem to optimality at least once. Another challenge about COLGEN is that executing step 1 may become time consuming as the number of columns in RMP increases and column management may become a crucial part of the algorithm. In column generation algorithms, it is also very common to iterate between steps 1 and 3 for a large number of iterations before termination. When there are alternative dual optimal solutions, the algorithm jumps from one primal LP solution to another without an improvement in the solution value.

After describing column generation algorithm, we can list the steps of a branch-and-price algorithm which are similar to steps of a branch-and-bound. The difference is that columns must be generated using COLGEN throughout the tree because we might not have all possible columns in the current node and branching rules and fixing of variables may change the values of the dual variables. We use notation RMP_2 = COLGEN(RMP_1) to state that RMP_2 is obtained after applying COLGEN to RMP_1. Figure **ZE: 2.1 shows a schematic representation of the algorithm. Update the figure** **BRANCH-PRICE**

0. Initialize a feasible RMP. Let the upper bound (UB) be infinity and set of unprocessed nodes be empty.

   Root Node:

1. Call COLGEN(RMP) and let RMP = COLGEN(RMP).

   i. If the solution to RMP is integer, then STOP. The optimal integer solution is equal to RMP solution.

   ii. Otherwise, find a disjunction to eliminate the current fractional solution and create two new nodes corresponding to the disjunction. Update set of unprocessed nodes with these two new nodes.

2. Let lower bound equal to solution of RMP.

3. Call an oracle to find a primal feasible solution. If the solution is better than UB, update UB.

   Node Selection & Branching:

4. If the set of unprocessed nodes is non empty, select one node to process, otherwise STOP. Return UB as an optimal integer solution.

   Non Root Node:

5. Modify RMP and the pricing problem in COLGEN according to the disjunctions along the path in the tree from the root node to the selected node.

6. Mark the node as processed. Call COLGEN(RMP) with modified RMP and the pricing problem and let RMP = COLGEN(RMP).

i. If the solution is infeasible, then fathom the node. Return to Step 4.

ii. If the solution to RMP is integer then fathom the node. And if it is better than UB, update UB. Return to Step 4.

iii. If the solution value is greater than the UB, then fathom the node. Return to Step 4.

iv. Otherwise, find a disjunction to eliminate the current fractional solution and create two new nodes corresponding to disjunction. Update set of unprocessed nodes with these two new nodes. Update lower bound based on the solution value.

7. Call an oracle to find a primal feasible solution. If the solution is better than UB, update UB. Return to Step 4.

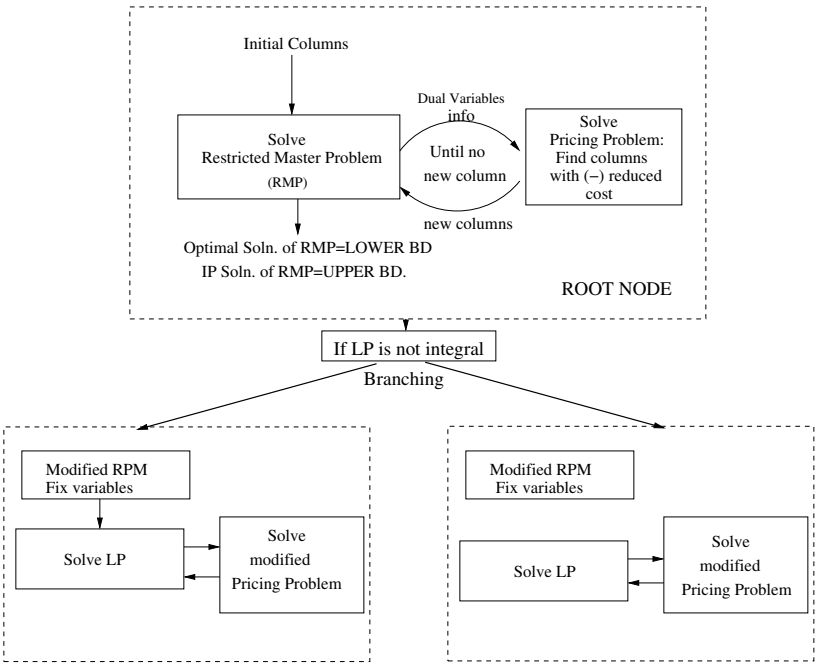

Figure 2.1: Branch-and-Price Tree

Besides the challenges in column generation algorithm, the main challenge in developing a branch-and-price algorithm is developing effective branching rules that do not destroy the structure of the pricing problem. At non root nodes of the tree, the RMP is modified based on the branching rules applied along the path and so is pricing problem. Therefore, applied branching rules must not complicate the pricing problem and can easily be incorporated in the solution algorithm for pricing problem. In the following sections, we describe the components of the branch-and-price algorithm that we developed for the LRSP. **ZE: In Section ??, we describe the column generation algorithm for solving the LPs at the nodes of the branch-and-price tree. In Section ??, we describe the branching rules. In Section ??, we present implementation details of the algorithm.**

2.3 Column Generation

To begin the column generation procedure, we need to construct an initial RMP for AUG-SPP-LRSP that includes all of the facility location variables but only a subset of the pairing variables. We solve the RMP and use the associated dual variable information to formulate the pricing problem. In the context of the AUG-SPP-LRSP, the objective of the pricing problem is to determine if there are feasible pairings (based on the definition and constraints in Section 1.4.2) with negative reduced cost that can be added to the RMP. If the pricing problem identifies at least one pairing with negative reduced cost, then we add one or more columns to the RMP and reoptimize. Otherwise, the current solution is the optimal solution for the full model.

2.3.1 Formulating the Pricing Problem

As explained in Section 1.4.2, set of pairings for each facility are independent, thus we decompose the pricing problem into a set of independent pricing problems, one for each facility. To write the objective function of pricing problem for facility j, we consider formulation AUG-SPP-LRSP and obtain an expression for the reduced cost of a pairing. Let π_i be the dual variable associated with constraint (1.14) for customer i, μ_j be the dual variable associated with constraint (1.15) for facility j, and σ_{ji} be the dual variable for linking constraint (1.25) for facility j and customer i. For the variable z_p for $p \in P_j$, the reduced cost $\hat{C}_p$ can be written as

$$\hat{C}_p = C_p - \sum_{i \in I} a_{ip}\pi_i + \sum_{i \in I} a_{ip} D_i \mu_j + \sum_{i \in I} a_{ip}\sigma_{ji}, \tag{2.1}$$

where C_p is the cost of pairing p. Then, we can express the pricing problem for facility j as the problem of minimizing $\hat{C}_p$ subject to the following constraints defining feasible pairings.

i. Each route included in the pairing starts and ends at the same facility,

ii. for each route in the pairing, the total demand of the customers assigned to the route is at most the vehicle capacity,

iii. the total travel time of the pairing is at most the vehicle time limit,

iv. each customer included in the pairing must be visited only once.

We can formulate this problem for facility j as follows:

$$\textbf{SP Minimize} \quad C^V + \sum_{i \in M} \sum_{k \in M} \hat{c}_{ik} x_{ik} \tag{2.2}$$

$$\sum_{k \in M} x_{ik} \leq 1 \qquad \forall i \in I \tag{2.3}$$

$$\sum_{k \in M} x_{ik} - \sum_{k \in M} x_{ki} = 0 \qquad \forall i \in M, \tag{2.4}$$

$$y_{ik} - L^V x_{ik} \leq 0 \qquad \forall i \in M, k \in M, \tag{2.5}$$

$$\sum_{k \in M} y_{ik} - \sum_{k \in M} y_{ki} + D_i \sum_{k \in M} x_{ik} = 0 \qquad \forall i \in I, \tag{2.6}$$

$$\sum_{i \in M} \sum_{k \in M} T_{ik} x_{ik} \leq L^T, \tag{2.7}$$

$$x_{ik} \in \{0, 1\} \quad \forall i \in M, k \in M, \tag{2.8}$$

$$y_{ik} \geq 0 \qquad \forall i \in M, k \in M, \tag{2.9}$$

where $M = I \cup \{j\}$, $x_{ik} = 1$ if link (i, k) is used in solution, and 0 otherwise, y_{ik} is equal to the amount of flow on link (i, k), and $\hat{c}_{ik}$ is the cost of traveling link (i, k). We refer $\hat{c}_{ik}$ as the *reduced cost of link* (i, k) and calculate as follows based on the reduced cost function 2.1.

$$\hat{c}_{ik} = \begin{cases} C^O T_{ik} - \pi_k + D_k \mu_j + \sigma_{jk} & \text{if } i \in M, \text{and } k \in I, \\ C^O T_{ij} & \text{if } i \in I. \end{cases} \tag{2.10}$$

To write the objective function (2.2), recall that in equation (2.1), the cost C_p is the sum of the fixed cost of a vehicle and the variable costs of the routes that are included in the pairing. The variable cost of a route is equal to the sum of the costs of arcs in the route. We can include part $-\sum_{i \in I} a_{ip} \pi_i + \sum_{i \in I} a_{ip} D_i \mu_j + \sum_{i \in I} a_{ip} \sigma_{ji}$ to the objective function by transferring the values π_i, $D_i \mu_j$ and σ_{ji} associated with customer i as costs to the incoming arcs of customer node i. Therefore, in the definition of $\hat{c}_{ik}$ in (2.10), operating cost of arcs into customer nodes are modified with corresponding dual variables. With this modification, the reduced cost of a pairing can be computed as the sum of the reduced costs of the arcs in the pairing.

2.3.2 Elementary Shortest Path Problem With Resource Constraints (ESPPRC)

The pricing problem (SP) is a mixed integer programming model. On the other hand, it can be defined as an elementary shortest path problem with resource constraints (ESPPRC) on a modified network. In an ESPPRC, the objective is to find a shortest path between the source node and the sink node such that each other node is visited at most once and the resource constraints are not violated.

To properly represent our pricing problem as an ESPPRC, we need to make one adjustment; while in an ordinary elementary shortest path problem, each node can be visited at most once, in

our case, because we are looking for a set of routes, we allow the sink node to be visited more than once. In our modified ESPPRC, the sink node represents the end of one route and maybe the beginning of another. In an ordinary ESPPRC, a path from the source to the sink corresponds to one route, however, a solution of our modified ESPPRC may correspond to multiple routes. Another reason for multiple visits to the sink is to reset the remaining capacity of the vehicle before the new route. In a pairing, the vehicle starts a route with zero truck load and picks up the required demand from all of the customers on the route. The vehicle returns to the depot after the last customer in the route and resets its capacity to zero load for the next route. In the pricing problem, the truck capacity is one of the resources. Whenever the path visits the sink in the modified network, the vehicle capacity resource consumption is updated and set to zero load. In the next section, we describe a network for ESPPRC to solve pricing problem (SP).

2.3.3 Underlying Network Structure For Modified ESPPRC

To define the pricing problem for facility j, we construct a network with $|I| + 2$ nodes, $|I|$ nodes for the customers, one for facility j as a source node and a copy of facility j as a sink node. Let A^j be the arc set of the network for facility j, s represent the source node, si represent the sink node. Then A^j consists of four sets of arcs:

$$A^j_{s,c} = (\{s\} \times I) \quad = \text{set of arcs from the source to customer nodes} \tag{2.11}$$

$$A^j_{c,si} = (I \times \{si\}) \quad = \text{set of arcs from customer nodes to the sink} \tag{2.12}$$

$$A^j_{si,c} = (\{si\} \times I) \quad = \text{set of arcs from the sink to customer nodes} \tag{2.13}$$

$$A^j_{c,c} = (I \times I) \quad = \text{set of arcs between pairs of customer nodes} \tag{2.14}$$

Note that there are no arcs from customer nodes back to the source but there are arcs from the sink to customer nodes, this enables us to generate pairings including multiple routes. In a path from source in this network, the arcs from the sink to customer nodes represent the beginning of a new route, whereas the arcs from the customer nodes to the sink represent the end of a route. We let each customer node has a demand equal to its demand in the LRSP instance. Demand of the source is zero while demand of the sink is equal to minus vehicle capacity $-L^V$. Vehicle capacity is a resource constraint in the problem and total demand must be less than the vehicle capacity on a path from source to any node in the network. And in the real problem, total demand of each route must be less than the vehicle capacity. Since the arcs from the sink to customer nodes represent the beginning of a new route, the vehicle load, total demand at the beginning must be reset to zero at sink node. By setting total demand to maximum of zero and total demand at any node, and letting the demand of sink node be minus vehicle capacity, a new route starts with zero total demand.

To generate pairings that can be served within time limit of a vehicle, we let the distance between each pair of nodes be equal to the travel time between these nodes given in the LRSP instance. Time

limit is the other resource constraint in the problem and total length of any path in the network must not exceed the time limit. Finally, to obtain that the cost of a path from source to sink in the constructed network is equal to the reduced cost of the corresponding pairing, we use the following cost function based on the objective function of (SP).

$$c'_{ik} = \begin{cases} c_{ik} - \pi_k + D_k\mu_j + \sigma_{jk} & \forall (i,k) \in (A^j_{s,c} \cup A^j_{si,c} \cup A^j_{c,c}), \\ c_{ij} & \forall (i,k) \in A^j_{c,si}, \end{cases} \tag{2.15}$$

where $c_{ik} = C^O T_{ik}$ be the operating cost of link (i,k) in LRSP instance.

A path from the source to the sink on the constructed network corresponds to a pairing, if (i) it visits any customer node at most once, (ii) total demand from source to any node in the path does not exceed the vehicle capacity, and (iii) the total length of the path does not exceed the time limit. These conditions are satisfied if the path is a feasible solution to ESPPRC on this network. And total cost of the path plus the vehicle fixed cost C^V equals to the reduced cost of the pairing for a given dual vector. If the path visits the sink more than once, then it corresponds to a pairing that includes more than one route. Figure 2.2 shows an example network and a path on the network. In the example, the path corresponds to pairing (facility$_j$ - 3 - facility$_j$ - 1 - 2 - facility$_j$) which includes two routes.

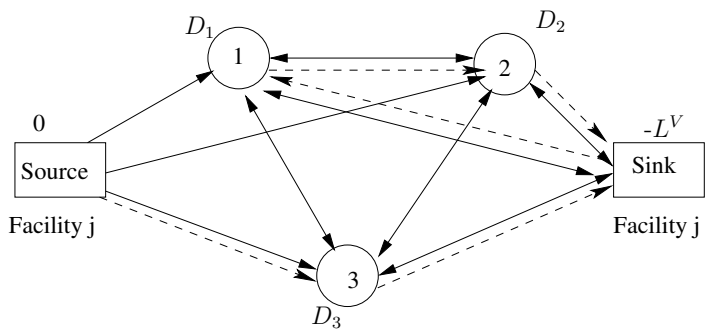

Figure 2.2: Example of a network and a path from source to sink (dashed line)

By using the example in Figure 2.2 we can easily illustrate that the total cost of the path is equal to the reduced cost of the corresponding pairing. Using the cost function defined in (2.15), the total cost of the path in the example plus the fixed cost is:

$$\begin{aligned} \text{Total cost of the path} + C^V &= c'_{j,3} + c'_{3,j} + c'_{j,1} + c'_{1,2} + c'_{2,j} + C^V \\ &= (c_{j,3} - \pi_3 + D_3\mu_j + \sigma_{j,3}) + c_{3,j} + (c_{j,1} - \pi_1 + D_1\mu_j + \sigma_{j,1}) + \\ &+ (c_{1,2} - \pi_2 + D_2\mu_j + \sigma_{j,2}) + c_{2,j} + C^V \end{aligned} \tag{2.16}$$

Grouping terms differently and noting that the column vector representing this pairing is $a_p = [111]$ and the cost of a pairing is the sum of the arc costs plus the vehicle fixed cost, we obtain the

following expression, which is just the reduced cost of z_p as calculated via equation (2.1).

$$\begin{aligned}(2.16) \quad &= VF + (c_{j,3} + c_{3,j} + c_{j,1} + c_{1,2} + c_{2,j}) \\ &- (\pi_3 + \pi_1 + \pi_2) + \mu_j(D_3 + D_1 + D_2) + (\sigma_{j,3} + \sigma_{j,1} + \sigma_{j,2}) \\ &= C_p - \sum_{i \in I} a_{ip}\pi_i + \sum_{i \in I} a_{ip} D_i \mu_j + \sum_{i \in I} a_{ip}\sigma_{ji} = \hat{C}_p\end{aligned}$$

Therefore, the reduced cost of a pairing in the original problem is equal to the cost of a path in the modified network.

In the following section, we describe approaches for solving the pricing problem defined on the constructed network.

2.4 Solving the Pricing Problem Exactly

The pricing problem is formulated as an ESPPRC. If the underlying network is acyclic, then the "elementary" property of the paths can be ignored, since than solution of a shortest path problem with resource constraints (SPPRC) is equal to the solution of ESPPRC. Desrochers (1988) developed a label correcting algorithm for SPPRC that is based on dynamic programming and that has pseudo polynomial complexity depending on size of the resource windows and the number of resources. The algorithm has been shown to be successful when there are tight resource constraints, but it becomes more time consuming as the number of resources increases. As the algorithm builds partial paths, it assigns a label to each indicating the resource consumption. To eliminate the problem of an exponentially increasing number of labels, the algorithm applies dominance rules to delete dominated labels.

If the network is not acyclic, which is generally the case in pricing problems because of the negative arc costs, "elementary" property of the paths must be considered. Beasley and Christofides (1989) describe how to find elementary paths in a graph by adding an extra binary resource for each node showing whether the node is visited or not. However, this increases the number of resources and possible combinations exponentially. In fact, ESPPRC is proved to be NP-Hard (Dror (1994)). Feillet et al. (2004) adapt Desrochers's label correcting algorithm to find elementary paths and strengthen the algorithm by developing additional domination rules for the labels. We have adapted Feillet et al. (2004)'s label correcting algorithm to our pricing problem.

We have developed two solution approaches. One approach is to solve one ESPPRC on the network described in Section (2.3.3). An alternative approach is to extend the first approach to a two-phase pricing problem in which each phase is also formulated as a network problem and solved as an ESPPRC.

2.4.1 1p-ESPPRC: One Phase Pricing Problem

In this approach we extend Feillet et al. (2004) algorithm to solve one ESPPRC on the constructed network. In the Feillet et al. (2004) algorithm, each path from the source to a node in the network is assigned a label, which is a vector of the reduced cost of that (partial) path, the resources consumed (vehicle load and elapsed time), the customer nodes included in the path, and the customer nodes that cannot be visited due to the resource constraints. Customers are considered *unreachable* if they have already been visited in the path or if they cannot be served by the vehicle without violating a resource constraint. The algorithm fans out from the source node, repeatedly examining the nodes of the graph. Each time it examines a node, the algorithm extends each of the paths to each possible successor node, continuing until no further extensions are feasible.

To explain the algorithm more specifically, we let l_r^i be a label corresponds to path r from source to node i. We state that vector $l_r^i = (R_c^i, R_v^i, R_t^i, R_n^i, V_1^i, .., V_{|I|}^i)$ keeps the information about path r where

$$
\begin{aligned}
R_c^i &= \text{reduced cost of the path,} \\
R_v^i &= \text{total demand of the path,} \\
R_t^i &= \text{time consumption of the path,} \\
R_n^i &= \text{total number of unreachable customer nodes,} \\
V_k^i &= \left\{ \begin{array}{ll} -2 & \text{if customer } k \text{ is temporarily unreachable for} \\ & \text{the path} \\ -1 & \text{if customer } k \text{ is unreachable for the path} \\ & \text{because of the resource limits,} \\ 0 & \text{if customer } k \text{ is not in the path and it is} \\ & \text{reachable for the path with the resources,} \\ s > 0 & \text{if customer } k \text{ is the } s^{th} \text{ node in the path,} \end{array} \right\} \forall k \in I.
\end{aligned}
$$

Note that the dimension of the vector V is $|I|$, there is one member for each customer node in the network. In label l_r^i, if customer k is visited in the path, V_k^i keeps the place of the customer in the path and remarks that customer k is unreachable for this label because of the elementary property. Based on Feillet et al. (2004)'s algorithm, if customer k is not visited in the path r, V_k^i is set to either 0 or -1. Even though we have two resource constraints (vehicle capacity and time limit) in the problem, we only check the time limit constraint to mark a customer as "unreachable", i.e., to set $V_k^i = -1$. Since if vehicle load exceeds vehicle capacity by adding customer k to the path, that does not mean that we can not add customer k to this path in the future. It may be possible that a path extended from path r stops at sink node and would have enough capacity to visit customer k in further extensions. In 1p-ESPPRC, we extended Feillet et al. (2004)'s algorithm by adding a new domination rule which will be explained later in this section. For this extension, if

customer k satisfy the conditions in the domination rule, we set V_k^i to -2 meaning that the customer is *temporarily unreachable* and this condition may change further in the path. We do not need an element in V for the source, because every path starts from the source and can never return to it. Similarly, we do not need an element for the sink, because the sink must always be reachable for a path. Extending a path to the sink is always resource feasible: (1) since demand of the sink is minus vehicle capacity, thus total demand would be less than the vehicle capacity and (2) while constructing label r to a non sink node i ($l_r^i = (R_c^i, R_v^i, R_t^i, R_n^i, V)$), the time limit constraint is checked to ensure that the path is further extendable to the sink, i.e., $R_t^i + T_{i,si} \leq L^T$; otherwise it is not created since every path must reach to the sink at the end.

The label correcting algorithm for solving the ESPPRC includes three main parts:

1p-ESPPRC:

1. **Initialization step**
   (a) To begin the algorithm, a label list is created for each node to keep the generated labels (partial paths to the node).
   (b) All label lists except the list for the source node are initially empty. The source has one label which is a zero vector, i.e., resource consumptions and the reduced cost are zero, and all customers are reachable.
2. **Extension step**
   (a) For each node, the label list is searched and all unprocessed labels are extended to all of the possible successors of the node. Unprocessed means that the label was not extended in previous iterations. The label l_r^i for node i can be extended to node k if
      i. reachable: $k = si$ or $V_k^i = 0$, and
      ii. resource feasible: $R_v^i + D_k \leq L^V$ and $R_t^i + T_{i,k} + T_{k,si} \leq L^T$.
   (b) If it is feasible to extend l_r^i to node k, a new label is created for node k by updating the resource consumptions. Let l_p^k be the label extended from l_r^i, then

$$
\begin{aligned}
R_c^k &= R_c^i + c'_{i,k}, \\
R_v^k &= \max\{R_v^i + D_k, 0\}, \\
R_t^k &= R_t^i + T_{i,k}, \\
V_g^k &= \left.\begin{cases} N & \text{if } g = k \text{ and } N = \# \text{ of nodes in the path}, \\ -1 & \text{if } V_g^i = 0 \text{ and } R_t^k + T_{k,g} + T_{g,si} > L^T, \\ V_g^i & \text{if } V_g^i \neq 0, \\ 0 & \text{otherwise}, \end{cases}\right\} \forall g \in I, \\
R_n^k &= \sum_{g \in I : V_g^i \neq 0} 1.
\end{aligned}
$$

(c) Mark the labels that are extended as processed.

3. **Elimination step**

(a) After extending unprocessed labels of a node to its successors, and constructing new labels for the successors, combine these new labels with their previously created label lists. Let node k be a node with new labels. To combine the label list of i with the list of new labels created for i, compare each label with the others in the list and only keep the *non-dominated labels*. let l_1^i and l_2^i be two labels for node i. Label l_1^i dominates label l_2^i if all of the following conditions are satisfied:

i. the current travel time and the vehicle load of l_1^i are less than or equal to those of l_2^i,

ii. the reduced cost of l_1^i is less than or equal to the reduced cost of l_2^i, and

iii. the number of total unreachable customer nodes of l_1^i are less than or equal to those of l_2^i, and for all nodes $g \in I$, g is reachable only in label l_1^i or g is unreachable in both labels l_1^i and l_2^i.

All dominated labels are deleted from the label lists of the nodes.

(b) If label list is changed for any node after domination algorithm is applied i.e., new labels are added, then the algorithm returns to the extension step. When all the labels are extended and no new labels are added to the lists of nodes, the algorithm terminates.

Additional Domination Rule

In addition to Feillet et al. (2004)'s domination rules, we have developed a domination rule for our pricing problem. Solving described 1p-ESPPRC on the constructed network leads to paths from source to sink that correspond to pairings including sets of *ordered* vehicle routes. Since the ordering of the routes within a pairing is arbitrary, we need to eliminate generating multiple paths that are different permutations of a set of routes and correspond to same pairing. For example, a call to 1p-ESPPRC may generate some or all of the following paths:

- (s - 1 - 2 - si - 3 - 4 - si - 5 - 6 - si),
- (s - 1 - 2 - si - 5 - 6 - si - 3 - 4 - si),
- (s - 3 - 4 - si - 1 - 2 - si - 5 - 6 - si),
- (s - 3 - 4 - si - 5 - 6 - si - 1 - 2 - si),
- (s - 5 - 6 - si - 1 - 2 - si - 3 - 4 - si), and
- (s - 5 - 6 - si - 3 - 4 - si - 1 - 4 - si).

However, all six of those paths are equal with respect to total cost and time, and they correspond to the same pairing composed of the following three routes: (facility - 1 - 2 - facility), (facility - 3 - 4 - facility) and (facility - 5 - 7 - facility). To eliminate generation of all possible ordered sets, we introduce a new domination rule in which we force a specific order for routes in the pairing, which decreases the state space of the algorithm significantly. We add the following condition to the algorithm.

Rule The index of the first customer node of each route should be less than the index of the first customer node in the following route.

The modified algorithm only generates paths from source to sink in which the first node of each route is less than the first node of the following route. Thus, the algorithm with this rule generates only path (s - 1 - 2 - si - 3 - 4 - si - 5 - 6 - si).

To apply this rule, we add variable R_f^i to keep the index of the first customer node in the current route in the path to the definition of label l_r^i. We let $l_r^i = (R_c^i, R_v^i, R_t^i, R_n^i, R_f^i, V_1^i, .., V_{|I|}^i)$ be the label corresponds to path r to node i. We do the following changes in 1p-ESPPRC:

- *In extension step of the algorithm*, in part (b), while constructing label l_p^k from l_r^i, we set $R_f^k = k$ if i is the source or sink (k will be the first node in the new route), and $R_f^k = R_f^i$ (still in the same route, thus the first node is same) otherwise. We also need to modify updating vector V^k.

$$
\begin{aligned}
V_g^k &= \left\{ \begin{array}{ll} N & \text{if } g = k \text{ and } N = \# \text{ of nodes in the path,} \\ V_g^i & \text{if } V_g^i \neq 0 \text{ and } V_g^i \neq -2, \\ -1 & \text{if } V_g^i = 0 \text{ or } V_g^i = -2, \text{ and } R_t^k + T_{k,g} + T_{g,si} > L^T, \\ -2 & \text{if } k \text{ is sink, } V_g^i = 0 \text{ and } g < R_f^k, \\ 0 & \text{otherwise,} \end{array} \right\} \forall g \in I, \\
R_n^k &= \sum_{g \in I : V_g^i \neq 0 \text{ or } V_g^i \neq -2} 1.
\end{aligned}
$$

  Here, $V_g^k = -2$ means that node g is *temporarily unreachable* because based on new rule, it can not be the first node in the new route. However, it may become the part of the path during further extensions. Therefore, when we calculate number of unreachable nodes R_n^k, we do not count temporarily unreachable nodes, we treat them same as reachable nodes. In addition, while extending a path to other nodes after the first node in the route, we change the condition temporarily unreachable to reachable by setting V_g^k to 0.

- *In the elimination step of the algorithm*, the steps and the conditions that should be satisfied to decide dominated labels are same. While executing condition (iii) in part (a), if there are temporarily unreachable nodes, we count them as reachable. Only addition to this step is

when both labels dominate each other. In the previous version, we can pick one of l_1^i or l_2^i arbitrarily and delete the other one. However, in this version, if l_1^i dominates l_2^i and l_2^i dominates l_1^i based on conditions (i), (ii) and (iii), we keep the one with smallest R_f^i (first node in the current route).

We have updated our algorithm with this additional rule. At the end of the algorithm, each label at the sink node corresponds to a feasible pairing with an associated reduced cost. The costs of paths at the sink node are updated with the vehicle fixed cost that must be included in the reduced cost of a variable, and any path with negative reduced cost is a candidate pairing to add to the RMP. If the algorithm concludes with no negative reduced cost labels on the sink node, then we can conclude that the current RMP is optimal.

2.4.2 2p-ESPPRC: Two-Phase Pricing Problem

An alternative approach to solve the exact pricing problem is to extend the first approach (1p-ESPPRC) to a two-phase pricing problem: first generate a set of feasible vehicle routes and then combine them to construct a feasible pairing of minimum reduced cost. Each phase is also formulated as a network problem and solved as an ESPPRC.

Since a pairing is a set of routes, we can express the variable part of the reduced cost of a pairing as the sum of the reduced costs of the routes forming the pairing. For there to exist a negative reduced cost pairing composed of more than one route, there must exist negative reduced cost routes. Otherwise, the pairing including only the single route with the lowest reduced cost would be the lowest cost pairing overall. Thus, the basic idea of the two-phase approach is to generate feasible routes first. If there are negative reduced cost routes, then combine these routes to generate pairings with more negative reduced cost. Otherwise, update the reduced cost of the first-phase routes with the fixed cost to obtain a column (a single-route pairing) with negative reduced cost or to conclude that the current solution is optimal.

Phase One: Generating Routes To generate only the feasible routes instead of pairings, we solve a regular ESPPRC problem on the constructed network with $|I| + 2$ nodes. We delete the arcs from the sink to customers and we terminate the paths at sink and do not extend them further from sink. We use algorithm 1p-ESPPRC without the additional domination rule. At the end of the algorithm, each label at the sink node corresponds to a feasible vehicle route with an associated reduced cost.

Let L_S denote the set of feasible routes with negative reduced cost at the sink. If L_S is non-empty, we continue with phase two. Otherwise, we add the fixed cost from expression (2.1) to the cost of the smallest reduced cost route to obtain the reduced cost of that single-route pairing. If this reduced cost is zero or positive, then we can conclude that the current LP relaxation solution is optimal. Otherwise, we have a pairing with negative reduced cost to add to the RMP. Note the implication that, even in the case where there are no *routes* with negative reduced cost, we may still

get a negative reduced cost *pairing* (consisting of a single route) when the fixed cost associated with the vehicle is negative. This can occur after applying certain branching rules (see Section 2.7).

Phase Two: Generating Pairings The objective of phase two is to generate feasible negative reduced cost pairings by combining the routes in set L_S. Two (or more) vehicle routes can be combined to form a feasible pairing if they do not have customer nodes in common and they can both (all) be completed within the vehicle's time limit. Finding a feasible pairing with the minimum reduced cost again corresponds to solving a ESPPRC.

We construct a network with $2 + |L_S|$ nodes: a source node, a sink node and one "intermediate" node to represent each of the vehicle routes in set L_S. The network includes an arc from the source node to each intermediate node, arcs between pairs of intermediate nodes, and an arc from each intermediate node to the sink node. Note that, because the sequence of routes in a pairing is arbitrary, we only include an arc from an intermediate node to another intermediate node with higher index.

Associated with each arc inbound to an intermediate node are two values: the time required to complete the corresponding route and the reduced cost of the route. The time and cost for arcs inbound to the sink are zero. To find a feasible pairing with the minimum reduced cost, we solve an ESPPRC with a single resource constraint (time). As in phase one, we apply the extended label correcting algorithm of Feillet et al. (2004), 1p-ESPPRC. At the end of the algorithm, each label at the sink node corresponds to a feasible pairing with an associated negative reduced cost. The costs of pairings at the sink node are updated with the fixed cost that must be included in the reduced cost of a variable, and any pairing with negative reduced cost is a candidate pairing to add to the RMP.

Since we solve the exact ESPPRC algorithm using 1p-ESPPRC in each phase of the algorithm, the overall pricing algorithm is exact and results in a column with the most negative reduced cost, if any. We refer to this exact pricing algorithm as 2p-ESPPRC.

ZE: Any proposition can be stated?

ZE: Example, 2 figures representing phase 1 and phase 2

2.4.3 Bidirectional Pricing Problem

2.5 SPPRC: Solving the Relaxation of Pricing Problem

One approach to find negative reduced cost pairings is to solve a relaxation of the pricing problem in which we allow customer nodes to be visited more than once, i.e., we relax the constraint (2.3) in the pricing problem formulation in Section 2.3.1. Even though we relax this constraint in the pricing problem and generate columns based on the relaxed model, we can still obtain feasible solutions to the AUG-SPP-LRSP as long as the integrality constraints are satisfied. This is because the constraint (1.14) and the integrality of the variables make sure that only the columns visiting customers at most

once are selected in a feasible solution. To incorporate solutions of relaxed pricing problem into the AUG-SPP-LRS formulation, we change the definition of the parameter a_{ip} to represent the number of visits to customer node i in the pairing p for all $i \in I$, $p \in P_j$ and $j \in J$. Thus, the parameter a_{ip} may take values greater than 1. If we relax other constraints in pricing problem such as the vehicle capacity, time limit constraint or the construction of feasible routes, then these columns in AUG-SPP-LRSP may lead to solutions that does not satisfy the constraints of an LRSP.

Solving a pricing problem in which we allow multiple visits to customers in the constructed network is equal to solving SPPRC. Relaxing elementary property for customers in ESPPRC leads to SPPRC. As noted in section (2.4) the complexity of a labeling algorithm for SPPRC is pseudo-polynomial, which is an advantage over ESPPRC. However, a disadvantage of generating columns from a relaxed pricing problem is that, because one set of constraints is relaxed, the LP relaxation bound may be weakened. That is, when the COLGEN terminates, the LP relaxation solution may contain columns visiting some customers multiple times, in which case the objective function is only a lower bound on the optimal LP relaxation bound obtained using an exact pricing problem. **ZE: In section ??, we demonstrate empirically how the bound of this relaxation compares to the optimal LP relaxation bound.**

To solve SPPRC, we use a generic label correcting algorithm based on Desrochers (1988). The algorithm 1p-ESPPRC described in Section (2.4.1) can be modified to solve SPPRC. In this version, since we do not have elementary property for customers in the path, we do not have to mark the customers as unreachable. Again, we let l_r^i be a label corresponds to path r from source to node i. We state that vector $l_r^i = (R_c^i, R_v^i, R_t^i, V_1^i, .., V_{|I|}^i)$ keeps the information about path r where R_c^i, R_v^i, and R_t^i are as described in Section (2.4.1). We let V_k^i be the number of times customer k is visited in path r. Since visiting a customer is not one of the resources in SPPRC, the algorithm only updates vector V^i and does not compare or check it during the algorithm, thus we have the following modifications to algorithm 1p-ESPPRC to solve a SPPRC:

- While executing *extension step,* in part (a), the condition (i) is deleted, and to decide whether the label l_r^i for node i can be extended to node k, resource feasibility (ii) for the extension is checked. In part (b) of this step, besides R_c^i, R_v^i, and R_t^i, we update V_g^k if $g = k$ and set $V_g^k = V_g^k + 1$.
- In *elimination step* in part (a) condition (iii) is deleted since we do not compare entries of vector V^i to decide domination relation between two labels.

2.6 Solving the Pricing Problem Heuristically

An exact algorithm is guaranteed to generate a column with the minimum reduced cost. The downside of the approach is that it may not be very efficient for large problems. As the number of

customers, vehicle capacity and time limit increase, the state space of the algorithm, i.e., the possible number of partial paths, so the number labels in the label list of each node in the algorithm increases. Thus, processing all these labels and terminating the algorithm take a lot more time considering the complexity of an ESPPRC problem.

Generally it is common in the literature to use heuristics for pricing problem to improve the efficiency of a branch and price algorithm. To find feasible pairings more quickly, here we define **ZE: two** heuristic versions of a labeling algorithm that we use in our solution algorithm. We can use these heuristic modifications for both 1p-ESPPRC and 2p-ESPPRC.

2.6.1 ESPPRC-LL: ESPPRC with Label Limit

This heuristic version is first proposed by Dumitrescu (2002) and restricts the state space of the exact algorithm by directly restricting the number of labels. In particular, during 1p-ESPPRC, or either phase of 2p-ESPPRC, we can limit the maximum number of labels at each node that are stored to be extended, i.e., number of non-processed labels. During the algorithm, as we update the list of labels in elimination step, at the end of the domination algorithm, we order the non-dominated labels in *increasing order of reduced cost* and we keep at most the first *label limit* number of non-processed labels. We refer to these heuristic versions of pricing algorithms as 1p-ESPPRC-LL(n) and 2p-ESPPRC-LL(n), where n is the value of the label limit. For small label limits, the algorithm terminates very quickly. And as n gets larger, the heuristic version approaches to the exact algorithm.

2.6.2 ESPPRC-CS: ESPPRC for a Subset of Customers

This second heuristic restricts the state space by considering just a subset of the customers instead of all of them. In this way, the total number of labels to be generated is restricted in advance.

In each pricing iteration for a facility, we determine *two customers* that have the *lowest reduced costs from the facility*. Let $j \in J$ be the facility index, we choose customers i_1 and i_2 such that $c'_{j,i_1} \leq c'_{j,i_2} \leq c'_{j,i}$ for all $i \in I \backslash \{i_1, i_2\}$. Let $I_n = \{i_1, i_2\}$ be the subset. We initialize the subset with these two customers since we expect to have *at least two routes in a pairing*. Then, by exploring the links from the subset of customers to the remaining customers, we repeatedly add one customer that is connected by a cheapest link to the subset until the size of the subset is equal to the determined *subset size*. Particularly, we let $I_n = I_n \cup \{k\}$ where $k = \text{argmin}_{k \in I \backslash I_n} \{\min_{i \in I_n} \{c'_{i,k}\}\}$ We stop when $|I_n| = n$ where n is the determined subset size.

After we determine I_n, we run exact pricing algorithm with this smaller customer set. We refer to these heuristic versions of pricing algorithms as 1p-ESPPRC-CS(n) and 2p-ESPPRC-CS(n), where n is the subset size. In 2p-ESPPRC-CS(n), we use the selected customers in phase 1 and, state space of phase 2 reduces depending on the reduction in the state space of phase 1.

If a heuristic version of the pricing algorithm identifies columns with negative reduced cost, they are feasible and they can be added to the RMP. However, these columns are not necessarily the

columns with most negative reduced cost for a given dual vector. Therefore, if a heuristic version of the pricing algorithm, do not identify any columns with negative reduced cost, the exact pricing problem must be applied at least once in order to conclude whether or not there exist columns with negative reduced cost, i.e., to conclude that the current LP is optimal or not.

2.7 Branching Rules

An important step in developing a branch-and-price algorithm is devising good branching rules. Branching rules that affect the structure of the pricing problem may make column generation more difficult. Therefore, the main objective is to find disjunctions that eliminate as much of the integer infeasible region as possible but that minimize the increase in difficulty of solving the pricing problem. We apply four branching rules in our branch-and-price algorithm.

Rule 1 **Branching on the facility location variables t:** At any node of the branch-and-price tree, if the corresponding LP solution has at least one fractional facility location variable, then we apply standard variable dichotomy branching for the facility location variables. we select one of the fractional facility location variables, say t_j, as our branching variable.

- In the left branch, we set $t_j = 1$, which forces facility location j to be open. The pricing problem for the corresponding node does not change since t variables are not present in the constraints of the pricing problem, and the pricing problem for each facility is independent from the others.
- In the right branch, we set $t_j = 0$, which forces facility j to be closed. Since the facility is closed, no pairings associated with the facility are allowed in the solution, so there is no need to solve the pricing problem. In addition, we set all z_p variables for facility j to 0.

Rule 2 **Branching on the total number of vehicles at each facility:** By definition, a pairing is associated with a single facility and corresponds to one vehicle. Thus, in any integer solution, the number of vehicles (pairings) used at any facility must be integer. Let w_j be the number of vehicles used at facility j. Then,

$$w_j = \sum_{p \in P_j} z_p \qquad \forall j \in J. \tag{2.17}$$

If we add the equalities (2.17) to the master problem, we can branch explicitly on the w_j variables. Assume that in a fractional solution $w_j = b$, where b is not integer. We create two

new nodes:

$$\begin{aligned} w_j &\leq \lfloor b \rfloor \qquad \text{(in the left branch)} \\ w_j &\geq \lceil b \rceil \qquad \text{(in the right branch)} \end{aligned}$$

Branching on the w_j variables instead of implicitly branching on $\sum_{p \in P_j} z_p$ is an example of explicit constraint branching, an idea developed by Appleget and Wood (2000) to improve the performance of branch-and-bound algorithms for mixed integer linear problems. These new variables make it easier to identify the necessary updates to the pricing problem. Let β_j be the dual variable associated with constraint (2.17) for facility j. Then, the reduced cost of the variable z_p changes to the following:

$$\hat{C}_p = C_p - \sum_{i \in N} a_{ip}\pi_i + \sum_{i \in N} a_{ip} D_i \mu_j + \sum_{i \in N} a_{ip}\sigma_{ji} - \beta_j \tag{2.18}$$

In equation (2.18), we can interpret β_j as a fixed cost for facility j that is independent of pairing p and the included customers. Therefore, we do not need to change our network structure or the pricing problem structure to incorporate β_j. Instead, we can calculate the cost of a path in the modified network in the same way as before and then add the fixed cost $-\beta_j$ to the cost of the solution to calculate the correct reduced cost of the column.

Rule 3 **Branching on assignment of a customer to a specific facility:** At any node of the branch-and-price tree, if the solution is fractional and Rules 1 and 2 are not applicable, then we consider the assignment of a customer to a specific facility. In a fractional solution, a customer may be partially served by two or more pairings assigned to different facilities. In this case, we can create a branching rule to force a customer to be assigned to a specific facility. To do so, for a customer i served by more than one facility and a facility j such that $\sum_{p \in P_j} a_{ip} z_p$ is fractional, we create two branches.

- In the right branch, we force customer node i to be served by facility j by adding the inequality

$$\sum_{p \in P_j} a_{ip} z_p \geq 1 \;\; (\gamma_{ji}) \tag{2.19}$$

  to the RMP. Let γ_{ji} be the dual variable associated with inequality (2.19). To incorporate this into the pricing problem for facility j, we change the cost of any arc entering node i to $c_{ki} - \pi_i + D_i\mu_j + \sigma_{ji} - \gamma_{ji}$ and solve the pricing problem as before. In addition, we can remove customer i from the network for the pricing problems of other facilities.

- In the left branch, we forbid the assignment of customer node i to facility j by adding

to the RMP the constraint

$$\sum_{p \in P_j} a_{ip} z_p \leq 0.$$

To incorporate this restriction into the pricing problem for facility j, we remove node i from the network of facility j. The pricing problems for the other facilities do not change.

Rule 4 Branching on flow between pairs of customer nodes At any node of the branch-and-price tree, if the solution is fractional and Rules 1, 2 and 3 are not applicable, then we may encounter the case where a customer is covered by two or more (fractional) pairings associated with the same facility. Then, we branch on flows on single arcs between pairs of customer nodes using a modified version of the Ryan and Foster (1981) branching rule first suggested by Desrochers and Soumis (1989). In the original Ryan and Foster (1981) branching rule, a pair of set partitioning constraints is selected. In one branch, the rule is to branch on the sum of the variables that satisfy both constraints and, in the other branch, the rule is to branch on the sum of the variables that do not satisfy both constraints simultaneously. In VRP context, set partitioning constraints are associated with customers, and Desrochers and Soumis (1989) modify the Ryan and Foster branching rule by looking only at the variables in which the selected pair of customers are in consecutive order or not. This modification makes it possible to incorporate branching rule to the pricing problem without changing the structure or the solution method.

To apply the modified Ryan and Foster branching rule, we select a pair of customer nodes (n_1, n_2) such that

$$0 < \sum_{j \in J} \sum_{\{p \in P_j | (n_1 \to n_2) \in p\}} Z_{jp} < 1 \tag{2.20}$$

where $\{p \in P_j | (n_1 \to n_2) \in p\}$ is the set of pairings (of facility j) in which n_1 is followed directly by n_2 for some n_1, $n_2 \in I$.

In one branch, we require that the flow on the directed arc between the selected pair of customers is one in all possible solutions. In the other branch, we forbid the flow on the directed arc between the selected pair of customers. We create two new nodes:

$$\sum_{j \in J} \sum_{\{p \in P_j | (n_1 \to n_2) \in p\}} Z_{jp} \geq 1 \quad \text{(in the left branch)}$$

$$\sum_{j \in J} \sum_{\{p \in P_j | (n_1 \to n_2) \in p\}} Z_{jp} \leq 0 \quad \text{(in the right branch)}$$

- The left branch specifies that either node n_1 is followed by node n_2 in a pairing or that neither node appears. To enforce this rule for the pairings to be generated, we update the arcs in the network for the pricing problem. In the pricing problem, all arcs leaving node n_1 and all arcs entering n_2 are deleted except the arc (n_1, n_2). Therefore, if a pairing includes one of the nodes, then it includes the arc (n_1, n_2). To delete an arc in the network, we set the cost of the arc to infinity. In the RMP, to satisfy the branching rule for the previously created pairings, we set to zero the variables corresponding to pairings that include only one of the nodes, n_1 or n_2, or that include both nodes but in which n_2 does not immediately follow n_1.
- The right branch specifies that n_2 cannot follow n_1 in a feasible pairing. The feasible pairings may include n_1 or n_2 only or may include both of them as long as n_2 does not follow n_1 directly. To apply this rule for the pairings to be generated, we simply delete the arc (n_1, n_2) from the network in the pricing problem. In the RMP, we set to zero the previously created pairings in which n_1 is followed by n_2.

These four branching rules guarantee that the branch-and-price algorithm terminates with an optimal solution. We further investigate this statement in the next section.

2.8 Terminating The Branch-and-Price Tree

In a standard branch-and-bound or a branch-and-price algorithm, as in algorithm BRANCH-PRICE, the nodes satisfying one of the following conditions can be fathomed:

1. The LP for the node is proved to be infeasible.
2. The lower bound for the node is larger than the best integer feasible solution.
3. The LP solution of the node is integer.

In our branch-and-price algorithm, there may exist some nodes that satisfy none of the conditions (1) to (3), and even though the LP relaxation is non integer, none of the four branching rules described in Section 2.7 is applicable. Therefore, these nodes satisfy the following:

i. the LP solution is feasible but not integer, and it is not fathomed because of its objective function value,

ii. all the facility location variables are integer (branching rule 1),

iii. the number of vehicles for each facility is integer (branching rule 2),

iv. each customer is assigned to a single facility (branching rule 3), and

v. there are no pairs of customer nodes that satisfy condition (2.20) (branching rule 4) .

We will show that the nodes satisfying these conditions can still be fathomed based on fathoming condition (3). We can construct an integer solution from the the LP solution with the same objective function value, and may update the best integer solution if it is a better upper bound.

In the LP solution vector satisfying conditions (ii) to (v), each open facility is fully utilized and the set of customers is divided into disjoint non-empty subsets each of which is assigned to a single open facility. Therefore, the LP solution vector can be decomposed into smaller solution vectors one for each facility. Each of these vectors corresponds to a solution to a single facility LRSP instance and thus they can be investigated separately. Based on this observation, further in this section we investigate the solution vector associated with a single facility. The statements are true for a solution vector corresponding to any facility. In this solution, the facility variable and the number of vehicles used (total number of pairings selected) are integer, but at least two pairing variables are non zero and fractional. By rearranging the solution values and construction of fractional pairings, we can modify this solution to an integer solution vector associated with the same facility. To explain this, we will show that the fractional pairings which are a part of the LP solution vector satisfying conditions (ii) to (v) have a certain structure.

Let r_1 and r_2 be two routes that can be included in a feasible pairing, and S_1 and S_2 be the set of customers visited by route r_1 and r_2, respectively.

Definition 2.1 Node disjoint: *Paths r_1 and r_2 are node disjoint if they do not have a common customer node, e.g., $S_1 \cap S_2 = \emptyset$.*

Definition 2.2 Identical (Same): *Paths r_1 and r_2 are identical if they both include exactly the same set of customers as well as same set of links between these customers.*

We can represent a pairing in terms the combination of routes included in the pairing. The routes included in a pairing are node disjoint by construction since in a pairing, a customer may be visited at most once. However, in a non-integer solution vector for a facility, at least two pairings are fractional and thus one or more customer nodes may be included in more than one pairing. And, in fact, at least two pairings corresponds to fractional variables must include either same route(s) or different but non-node disjoint routes, or both. We explain this result using an example. Solution 1 is an example of a fractional solution (for a single facility) that includes pairings that share only identical routes, all routes are node disjoint. Solution 2 is an example of a fractional solution that includes pairings that have non-node disjoint routes.

Example Solution 1: Same or node disjoint routes:

$$
\begin{aligned}
z_1 = 0.5 \quad &: \quad \text{facility}_1 - 1 - 2 - \text{facility}_1 \\
z_2 = 0.5 \quad &: \quad \text{facility}_1 - 1 - 2 - \text{facility}_1 - 3 - 4 - 5 - \text{facility}_1 \\
z_3 = 0.5 \quad &: \quad \text{facility}_1 - 6 - 7 - \text{facility}_1 - 3 - 4 - 5 - \text{facility}_1 \\
z_4 = 0.5 \quad &: \quad \text{facility}_1 - 6 - 7 - \text{facility}_1
\end{aligned}
$$

Solution 2: Different and non-disjoint paths:

$$
\begin{aligned}
z_1 = 0.5 \quad &: \quad \text{facility}_1 - 1 - 2 - \text{facility}_1 - 3 - 4 - 5 - \text{facility}_1 \\
z_2 = 0.5 \quad &: \quad \text{facility}_1 - 4 - 6 - 7 - \text{facility}_1 \\
z_3 = 0.5 \quad &: \quad \text{facility}_1 - 5 - \text{facility}_1 - 7 - 1 - 2 - 3 - \text{facility}_1
\end{aligned}
$$

Both solution 1 and 2 are fractional since at least two variables associated with pairings including same customers are selected. However, in solution 1, selected pairings not only include same customers but also include same routes such as route facility$_1$ - 1 - 2 - facility$_1$. On the other hand, in solution 2, pairings do not include same routes but there are at least two pairings (such as pairings 1 and 2) that include non-node disjoint routes.

A fractional solution for a facility can be similar to either solution 1 or solution 2, or it can be combination of solution 1 and solution 2. However, for a solution vector satisfying conditions (i) to (v) we can state the following result.

Lemma 2.1 *In a solution vector satisfies conditions (i) to (v), there exists at least one facility with an associated fractional solution vector. And a fractional solution vector for such facility corresponds to pairings including identical or node disjoint routes (as in solution 1). At least two pairings have at least one identical route. And all non identical routes in the pairings are node disjoint.*

Proof We have noted before that a solution vector satisfying conditions (ii) to (iv), can be decomposed by facility. Since the solution is fractional (condition (i)), there exists at least one facility with a fractional solution vector. Since the solution vector for facility is fractional, there are at least two pairing variables that are fractional (location variable and the total number of vehicles for the facility must be integer because of conditions (ii) and (iii)). Because of fractional pairing variables, there exists at least one customer visited in two fractional pairings.

Let i be a customer included in both pairings z_1 and z_2. Let r_1 be the route visiting customer i in z_1 and r_2 be the route visiting customer i in z_2. Since r_1 and r_2 are both visiting customer i, they must be either the same route or they are not node disjoint. If they are not identical and not node disjoint, then there exists at least one pair of customers k_1 and k_2 such that the link (k_1, k_2) is either

in route r_1 or in r_2, but not in both. However, this contradicts with the satisfaction of condition (v) (branching rule 4). Therefore, r_1 and r_2 must include same set of links, thus they are identical. Therefore, if two fractional pairings visit same customer, they have to include exactly same route(s) including common customers. This concludes that a fractional set of pairings must have at least one route included in at least two pairings, and non identical routes must be node disjoint. □

Lemma 2.2 *In a solution vector satisfies conditions (i) to (v), the LP relaxation value is equal to value of an integer solution.*

Proof Based on Lemma 2.1, if the fractional solution satisfies conditions (i) to (v), the structure of the solution vector for each facility is similar to solution 1, i.e pairings include only identical routes and node disjoint routes. By eliminating identical routes from all pairings but one, we can obtain a set of pairings which includes each route exactly once. This set of pairings corresponds to an integer solution. The objective function value of the new solution is same with the previous fractional solution since both solutions have exactly same set of node disjoint routes and the flow on each node is 1 (each customer included in a route must be served fully by single facility) . □

Therefore, based on Lemma 2.1, by setting the LP solution value equal to the value of of an integer solution, we can fathom a node satisfying conditions (i) to (v) based on fathoming condition (3). This proves the following statement.

Proposition 2.1 *The branching rules listed in Section 2.7 are complete and guarantee that the branch-and-price algorithm provides an optimal solution.*

We conclude this section with the following example of an integer solution that can be obtained from solution 1 which satisfies conditions (i) to (v).

Example Solution 3: Integer solution from solution 1

$$
\begin{aligned}
z_1 = 0 \quad &: \quad \text{facility}_1 - 1 - 2 - \text{facility}_1 \\
z_2 = 1 \quad &: \quad \text{facility}_1 - 1 - 2 - \text{facility}_1 - 3 - 4 - 5 - \text{facility}_1 \\
z_3 = 0 \quad &: \quad \text{facility}_1 - 6 - 7 - \text{facility}_1 - 3 - 4 - 5 - \text{facility}_1 \\
z_4 = 1 \quad &: \quad \text{facility}_1 - 6 - 7 - \text{facility}_1
\end{aligned}
$$

2.9 Overall Branch and Price Algorithm

ZE: Add tailing off property, did some experiments about the duals values, early termination experiments?

In this section, we discuss the overall branch-and-price algorithm and some implementation details of the algorithm. We implemented the algorithm in MINTO 3.1 using CPLEX 9.1 as the LP

solver. MINTO (Savelsbergh and Nemhauser (1996)) is a mixed integer solver that uses a branch-and-bound algorithm with an LP solver. MINTO has built-in applications such as preprocessing, variable and constraint generation and primal heuristics, but it also allows a user to design his/her own application code such as a branch-and-cut and a branch-and-price that is specific to a problem. Within MINTO, we customized several routines and added new ones for our algorithm. We have written routines such as to generate an initial set of columns, to solve the pricing problem and to select a branching variable. In the following, we explain some parts of the algorithm in detail.

2.9.1 Initialization

To initialize the algorithm BRANCH-PRICE, we need to construct an initial RMP that includes location variables and an intial set of pairing variables. In addition, the solution of the initial RMP must be feasible to be able to call COLGEN. In this section, we describe the implementations in order to satisfy these requirements.

Artificial variables

At every node of the branch-and-price tree including the root node, we need to start the column generation algorithm with a feasible RMP or we need to know that the node is infeasible to be able to fathom it. Since the RMP does not contain all of the columns, it is not necessarily true that the real LP at the node is infeasible when the RMP is infeasible. At the root node, it is easy to start with a feasible LP solution, but the RMP might become infeasible with respect to the current set of columns as the branching rules are added. At a non root node, depending on the branching rules applied, the infeasibility may not be easily detected by checking simple conditions. Therefore, it may not be obvious to move a feasible solution or conclude the infeasiblity of the current LP. If the current RMP solution at any node is infeasible, we need to verify that it truly is infeasible so we can fathom the node or we need to find a feasible solution from which to start the column generation algorithm to continue processing the node.

We introduce artificial variables to model SPP-AUG-LRSP to be able to have an initial feasible RMP or obvious infeasibility for any node. We can add the artificial variables at the root node or at any node in which the RMP is infeasible. Currently, we introduce them at the root node and keep them along the tree.

We create an imaginary facility with I pairings, one for each customer. Each pairing (column) visits one customer. Let t_a be equal to 1 if the artificial facility is open, and 0 otherwise. Let $z_{a1}, .., z_{a|I|)}$ be the artificial pairing variables where z_{ak} represents the pairing visiting customer k.

We modify the LP relaxation of AUG-SPP-LRSP with these artificial variables:

$$\text{Minimize} \quad \sum_{j\in J} C_j^F t_j + \sum_{j\in J}\sum_{p\in P_j} C_p z_p + C_a^F t_a + \sum_{i\in I} C_a z_{ai} \tag{2.21}$$

$$\text{subject to} \quad \sum_{j\in J}\sum_{p\in P_j} a_{ip} z_p + z_{ai} = 1 \quad \forall i \in I, \tag{2.22}$$

$$\sum_{p\in P_j}\sum_{i\in I} a_{ip} D_i z_p - L_j^F t_j \leq 0 \quad \forall j \in J, \tag{2.23}$$

$$\sum_{j\in J} t_j \geq R \tag{2.24}$$

$$\sum_{p\in P_j} a_{ip} z_p - t_j \leq 0 \tag{2.25}$$

$$z_{ai} - t_a \leq 0 \quad \forall i \in I \tag{2.26}$$

$$0 \leq t_j \leq 1 \quad \forall j \in J, \tag{2.27}$$

$$0 \leq z_p \leq 1 \quad \forall p \in P_j, j \in J, \tag{2.28}$$

$$0 \leq z_{ai} \leq 1 \quad \forall i \in I, \tag{2.29}$$

$$0 \leq t_a \leq 1, \tag{2.30}$$

where $C_a^F$and C_a are costs of artificial variables. We do not actually need to add variable for artificial facility and constraints (2.26), we do that for the interpretation, and consistency with the model variables. We assign large costs to the artificial variables to discourage them being non zero in the solution, unless it is infeasible. We define $C_a^F = (R+1) \cdot \max_{j\in J}\{C_j^F\}$, and $C_a = 2C^O \sum_{i\in I} t_{1,i} + C^V$ where the fixed cost of the artificial facility is set equal to the minimum number of required facilities plus one times the maximum fixed cost of the facilities to get a larger cost than selecting real facilities. The total operating cost of each artificial pairing is the same with a value equal to the sum of the operating costs of all single customer routes from depot j plus the vehicle fixed cost. We can select depot j arbitrarily.

The model with artificial variables results either feasible or infeasible LP relaxation solution for all possible sets of branching rules. If it is infeasible, we can conclude that the node is infeasible and can be fathomed. If it is feasible, we can start doing column generation. And at the end of algorithm COLGEN, if the solution includes artificial variables, this means that the column generation algorithm searched all possible columns but could not find a column to replace the artificial variables, thus the real LP is infeasible. In this case, because of the artificial variable costs, the RMP solution value is high enough to fathom the node based on the incumbent solution value.

Initial Columns

Artificial variables added to the model can be used to initialize the algorithm. From previous computational experience with branch-and-price algorithms, we expect a reduction in the number of pricing iterations required to solve the LP relaxation when starting from a good set of columns rather than an easily constructed or arbitrary solution. Therefore, besides the artificial variables, we generate feasible columns to construct the initial RMP. However, there is a trade off between the effort (time) required to generate such a solution and the resulting reduction in overall running time.

We have developed an algorithm that combines a facility location heuristic, several well known VRP heuristics (Clark and Wright, nearest neighbor, nearest insertion and sweep heuristics), and a bin-packing heuristic (best fit decreasing) to generate an initial set of pairings. The basic idea of the routine is to sequentially determine the following decisions for a given set of open facilities: (i) allocation of customers to facilities, (ii) design of feasible vehicle routes, and (iii) assignment of routes to vehicles. The total cost of such a sequentially determined solution for a given set of open facilities is calculated. To decide what set of open facilities would yield such a solution of lowest cost, the algorithm starts with all facilities open and iteratively closes the facility whose elimination results in the greatest cost savings until the problem becomes infeasible (this is called the DROP heuristic in the context of facility location problems). We refer to this heuristic algorithm as I-Heur. I-Heur produces a set of columns as well as a feasible solution for the problem. The outline of the algorithm I-Heur which is the combination of functions DROP(J) and TCOST(J) is given below.

I-Heur

- **DROP**(J)

  1. Let $J_o = J$ be the set of open facilities.
  2. For all facilities in set J_o:
     i. Pick $k \in J_o$ and close facility k. Let $J^k = J_o \backslash \{k\}$.
     ii. Call function TCOST(J^k) to calculate the cost of opening all of the facilities in set J^k.
  3. Let $k^* = \text{argmin}_{k \in J_o}\{\text{TCOST}(J^k)\}$.
  4. If TCOST(J^{k^*})$< \infty$, then let $J_o = J_o \backslash \{k^*\}$ and return to step 2. STOP otherwise.
  5. Return set of open facilities, J_o, total cost of the system TCOST(J_o) and set of pairings constructed in function TCOST(J_o).

- **TCOST**(J_t)

  1. Partition customers among set of facilities, J_t. Assign each customer to the nearest facility, if the capacity of the facility is not exceeded with the assignment. Otherwise, assign the customer to the next nearest facility. Let I_j be the set of customers assigned to facility $j \in J_t$.

2. Let COST $= \sum_{j \in J_t} C^F$ (add the fixed cost of facilities).
3. For each facility j in set J_t:
   i. Call function ClarkWright(j, I_j) for VRP instance with customers I_j and facility j. This function constructs the vehicle routes based on Clark and Wright heuristic. Let R_1 be the set of routes generated. Call BestFitD(R_1) to pack the routes into pairings considering time limit (best fit decreasing heuristic for bin packing problems). Let P_1 be the set of pairings generated. Let C_1 be the fixed and operating costs of pairings in set P_1.
   ii. Call function NN(j, I_j) for VRP instance with customers I_j and facility j. This function constructs the vehicle routes based on nearest neighbor heuristic. Let R_2 be the set of routes generated. Call BestFitD(R_2). Let P_2 be the set of pairings generated and C_2 be the costs.
   iii. Call function NI(j, I_j) for VRP instance with customers I_j and facility j. This function constructs the vehicle routes based on nearest insertion heuristic. Let R_3 be the set of routes generated. Call BestFitD(R_3). Let P_3 be the set of pairings generated and C_3 be the costs.
   iv. Call function SW(j, I_j) for VRP instance with customers I_j and facility j. This function constructs the vehicle routes based on sweep heuristic. Let R_4 be the set of routes generated. Call BestFitD(R_4). Let P_4 be the set of pairings generated and C_4 be the costs.
   v. Let COST = COST + min$\{C_1, C_2, C_3, C_4\}$. Let $P_j = P_{argmin\{C_1,C_2,C_3,C_4\}}$.
4. Return COST, total system cost, set of pairings P_j for all $j \in J_t$.

I-Heur returns a set of columns for facilities that are selected to be open. We add these columns to the RMP and initialize the incumbent solution be the cost of the solution obtained from I-Heur. To balance the number of columns added for each facility, for the facilities that are fixed to be closed in the heuristic solution, we determine a set of customers that can be served from the facility within the time limit and use function ClarkWright and BestFitD to determine a set of pairings for those facilities.

2.9.2 Branch Selection

Given a fractional solution at a node of the branch-and-price tree, we apply the branching rules in the order that they are described in Section 2.7. For each branching rule, we choose the most fractional disjunction.

2.9.3 Primal Feasible Solution

Having good primal feasible solutions can reduce the size of the explored tree. We used four ways to obtain primal feasible solutions in our solution algorithms: MINTO's built-in primal heuristic routines, I-Heur, a standard branch-and-bound algorithm, and a heuristic branch-and-price algorithm.

For relatively small problems, since the quality of the solution generated by the I-Heur is sufficiently high, we initialized the upper bound with the solution obtained from I-Heur and used MINTO's primal heuristic routines within the branch-and-price algorithm.

For larger, more difficult problems, we implemented a more powerful procedure to both improve the upper bound and generate an initial set of columns. We run a heuristic branch-and-price algorithm called H-BP, in which we construct an initial branch-and-price tree subject to a time limit, employing only the heuristic pricing algorithms 2p-ESPPRC-LL(n) and 2p-ESPPRC-CS(n). When the time limit is reached (or the algorithm terminates), we have a set of generated columns available to be used and an upper bound for the problem. To improve the quality of the upper bound obtained by H-BP, we applied a standard branch-and-bound algorithm to an integer RMP with the current set of columns at the root node of the H-BP subject to a one-hour time limit. Our computational experiments showed that the quality of the upper bound obtained using H-BP is very high.

2.9.4 Solving Pricing Problem and Algorithm Overview

We have implemented two versions of a branch-and-price algorithm. For small or medium size instances, we ran a *one-stage* solution algorithm that is an exact branch-and-price algorithm initialized with columns generated by I-Heur. We refer to this version as E-BP. For larger instances, we implemented a *two-stage* solution algorithm designed on the idea of "restart". The first stage of the algorithm runs H-BP to get a good upper bound and a good set of columns. The second stage is an exact branch-and-price algorithm that is initialized with the results of H-BP. We refer to this version as E-BP-2S. In our experiments, we used E-BP for instances with 25 customers and E-BP-2S for instances with 40 customers.

Since the steps of the overall pricing algorithm and the parameters used in E-BP and E-BP-2S are different, we separately explain the pricing algorithm in each version.

Pricing Algorithm In E-BP.

At each node of the branch-and-price tree, we employed the following procedure:

1. Set both the subset size and the iteration limit to 12. Execute 2p-ESPPRC-CS(12) for 12 iterations or until the algorithm fails (the algorithm *fails* if it does not generate any negative reduced cost columns).

2. Set the starting label limit value (m) to 50 and the maximum label limit to 300.

3. Call 2p-ESPPRC-LL(m) for each facility. If, during an iteration, no negative reduced cost columns are generated for a particular facility at its current label limit, execute the following procedure:

    i. If the current label limit is less than the maximum, increment the label limit with a multiplier of 2, i.e., let $m = 2 * m$, and solve 2p-ESPPRC. If the pricing problem identifies negative reduced cost columns, keep the label limit value, return to the beginning of step 3 and continue with the pricing problem for the next facility. Otherwise, set the label limit to the maximum.

    ii. If the current label limit is equal to or greater than the maximum, do not solve the pricing problem for this facility during the subsequent pricing iterations for the current node. Return to step 3.

4. When no columns are generated for any facility, the pricing problem for all facilities must be solved using the exact version, 2p-ESPPRC, to prove optimality.

Pricing Algorithm In E-BP-2S.

In E-BP-2S, the algorithm first calls H-BP for a limited time. The steps of the pricing algorithm in H-BP are similar to E-BP except that some parameters are different and the exact pricing 2p-ESPPRC is never used in H-BP. The details follow.

1. Set the subset size to 15 and the iteration limit to 20. Call 2p-ESPPRC-CS(15) for 20 iterations or until the algorithm fails.

2. Set the starting label limit value (m) to 5 and the maximum label limit to be 15.

3. Call 2p-ESPPRC-LL(m) for each facility. If, during an iteration, no negative reduced cost columns are generated for a particular facility at its current label limit, do the following:

    i. If the current label limit is less than the maximum, increment the label limit with a multiplier of 2, i.e., let $m = 2 * m$, and re-solve 2p-ESPPRC-LL(m). If the pricing problem identifies negative reduced cost columns, keep the label limit value, return to the beginning of step 3 and continue with the pricing problem for the next facility. Otherwise, return to step i.

    ii. If the current label limit is equal to or greater than the maximum, skip the pricing problem for this facility during the subsequent pricing iterations for the current node. Return to step 3.

After solving H-BP, we initialize E-BP-2S and start the second stage of E-BP-2S in which the pricing algorithm is the same as E-BP except that it skips step 1, i.e., it does not call 2p-ESPPRC-CS(n).

Chapter 3

Cut Generation In Branch and Price Algorithm

3.1 Introduction

In this chapter, we investigate cutting planes valid for LRSP and that can be incorporated into our branch and price algorithm. Extending a branch and price algorithm to a branch, price and cut algorithm using cutting planes is a promising approach in the effort of improving the overall performance of the solution algorithm. Addition of cutting planes can improve the LP relaxation bounds in the expense of time to search for cuts, to solve larger LP models, and possible performance reduction in the column generation procedure caused by the incorporation of the cuts. Therefore, efficiency of an extended algorithm depends on other factors related with branch and price algorithm besides the strength of the new cutting planes.

The point of this chapter is to derive some valid inequalities, include them to our branch and price algorithm and investigate the performance of the overall algorithm. In order to find cuts that are valid for LRSP, we can explore the set of valid inequalities derived for known polytopes that are relaxations and special cases of LRSP, such as knapsack problem, set partitioning problem, LRP and VRP.

The main contributions of this chapter are to investigate some of the most common cuts that are valid for the related polytopes of LRSP; discuss their validity for LRSP, their integration into the model SPP-LRSP and necessary updates in our branch and price algorithm; and asses the value of these cuts for the solution algorithm. In addition, we explore possible ways of adapting the approach of generating disjunctive cutting planes which are one set of generic cutting planes that are commonly used in branch and cut algorithms. There does not exist a current publication that is regarding disjunctive cuts for branch and price algorithms. This chapter also provides some theoretical results about the strength of the SPP-LRSP by showing that some of the cuts that can be derived using edge based formulation are already implied by the SPP-LRSP.

Next, we review some terminology used in the rest of the chapter and describe necessary updates to our branch and price algorithm to extend it to a branch, price and cut algorithm. Then, the analysis in this chapter can be divided into three parts. First, we study some relaxations and projections of formulation SPP-LRSP in order to obtain valid inequalities. Second, we investigate possible ways to include disjunctive cuts (one of the generic MIP cuts) in a branch and price algorithm. Third, we consider inequalities valid for the relaxations of the edge based formulation of LRSP and discuss their effect to SPP-LRSP formulation.

3.2 Overview of Cut Generation in Branch And Price Algorithm

In this section, we outline the main steps of a branch, price and cut algorithm in general. We use the same terminology as in Section **??** and the description of the branch and price algorithm, BRANCH-PRICE, also given in Section **??**.

First, we summarize some basic terminology from integer programming. An inequality is called

valid inequality if it is satisfied by all points in the convex hull of a problem. A cut is a valid inequality that is generally added to the model later in the solution process and possibly violated by the current LP relaxation solution. One needs to solve the separation problem to find a cut that is violated by a given fractional solution. The aim of adding valid inequalities and cuts to the model is to get a better approximation of the convex hull of the problem. A valid inequality is called a face if it is satisfied with equality by some solutions of the problem. A face is called facet if it is nonempty and its dimension is one less than the dimension of the problem. The most beneficial valid inequalities are the facets of the convex hull. Therefore, people take great interest in finding facet defining inequalities.

In this chapter, we search for valid inequalities and cuts for LRSP and we want to integrate these to our solution algorithm. For ease of the notation, we refer the procedure that generates cuts for a given RMP as CUTGEN(RMP) and we refer it as CUTGEN() if the procedure does not depend on the current LP solution. Both CUTGEN(RMP) and CUTGEN() return a set C including cuts that are generated at the end of the procedures. In order to explain the modifications to the BRANCH-PRICE to include cuts, we classify the generated cutting planes based on their structure.

- *Global Cut:* If the derived inequality is valid for the original problem's convex hull, then it is *global* to the tree, i.e.,it is valid for all nodes of the tree.
- *Local Cut:* Instead, if it is valid for the convex hull of the problem at the specific node, then the cut is *local* to that node, and only valid for that node and its children.
- *Z-free Type Cut:* If an inequality has zero coefficients for all variables that are generated via column generation algorithm, then we refer this cut as *z-free type* cut.
- *With-z type Cut:* If some variables from the column generation have nonzero coefficients in the cut, then we refer it as *with-z type* cut.

The incorporation of a cut to the algorithm depends on the structure of the cut. If a cut is global, then we can add the cut to the RMP in all active nodes of the branch and price tree. Otherwise, the cut is added to the RMP at the current node (and at its children). If we need the LP relaxation solution to generate a violated cut, then the cut is generated after the column generation algorithm (COLGEN) is terminated and the optimal RMP solution is found at the current node. Otherwise, the cut may be added at any time to the current RMP (before or after the COLGEN or any iteration of the COLGEN). In our algorithm, if the cut generation function does not need the LP solution, then we add the valid inequality before we start COLGEN.

Addition of a z-free type cut does not affect the solution algorithm for the pricing problem since it does not affect the column generation algorithm. On the other hand, a with-z type cut affects the column generation algorithm, since it has nonzero coefficients for some variables that are previously generated or may be generated in the future via column generation algorithm. Therefore, we need to

pass the information about the cut to the pricing problem. This may require changes in the solution algorithm for the pricing problem.

We extend the BRANCH-PRICE algorithm to a branch, price and cut algorithm by updating the solution process at each node of the tree as in the following:

A node in BRANCH-PRICE-CUT

0. Let RMP be the RMP at the beginning of the node. For the root node, it is the initial RMP and for a non root node, it is the parent's final RMP updated with the related branching rule.
1. Call CUTGEN(). Let $C \leftarrow$ CUTGEN() where C is the set of cuts.
2. If $C \neq \emptyset$, then update RMP with cuts, i.e., RMP $\leftarrow$ (RMP + C).
3. RMP $\leftarrow$ COLGEN(RMP).
4. If the RMP solution is infeasible or integer, STOP. Otherwise, run CUTGEN(RMP). Let $C \leftarrow$ CUTGEN(RMP).
5. If $C = \emptyset$, STOP processing the node. Otherwise, update RMP with cuts, RMP $\leftarrow$ (RMP + C).
6. If C has any with-z type cuts, then update COLGEN based on the information from with-z type cuts.
7. RMP $\leftarrow$ COLGEN(RMP).

3.3 Cutting Planes Based on SPP-LRSP

In this section, we discuss some valid inequalities derived based on the SPP-LRSP formulation presented in Section 1.4.2. In Chapter 1, we augmented this formulation with two sets of valid inequalities, (1.18) and (1.25), which are shown to be effective computationally in Section 4.3.2. Here, we study the LRSP polytope, defined as follows, to derive more valid inequalities. We used the same notation as in Chapter 1.

$$\mathcal{P}_{\text{SPP}} = \text{conv}\{t \in \{0,1\}^{|J|}, z \in \{0,1\}^{\sum_{j \in J} |P_j|} | t, z \text{ satisfies (1.14), (1.15)}\}. \tag{3.1}$$

This study is two fold: first, we investigate the cutting planes for the relaxations of polytope $\mathcal{P}_{\text{SPP}}$ and second, we investigate the cutting planes for the projections of $\mathcal{P}_{\text{SPP}}$.

3.3.1 Relaxations of the SPP-LRSP

In this section, we investigate two relaxations: knapsack and set partitioning problem. We list most common set of cuts for these problems (from the literature) based on our notation for SPP-LRSP. Even though, we did not implement any of these cuts for our branch and price algorithm, we felt the necessity of reviewing the literature and discussing possible implementation difficulties in the context of this chapter.

Knapsack Polytope In $\mathcal{P}_{\text{SPP}}$, if we relax constraints (1.15) by dropping the variable t, then each constraint defines a single 0/1 knapsack polytope. Since we assume that the column set for each facility, in other words items available for each knapsack in the context of knapsack problems, are disjoint from each other, we consider $|J|$ single knapsack polytopes instead of one multiple knapsack polytope in which the set of items available for the knapsacks are not disjoint. We let $\mathcal{P}^j_{\text{KNP}}$ be the knapsack polytope that corresponds to facility $j \in J$, such that

$$\mathcal{P}^j_{\text{KNP}} = \text{conv}\{z \in \{0,1\}^{|P_j|} | \sum_{p \in P_j} D(p) z_p \leq L^F_j\} \quad \forall j \in J, \tag{3.2}$$

where $D(p)$ is equal to the total demand of customers in pairing p (i.e.,$D(p) = \sum_{i \in I} a_{ip} D_i$). $\mathcal{P}^j_{\text{KNP}}$ for all $j \in J$ is a relaxation of $\mathcal{P}_{\text{SPP}}$. Thus, we can investigate the valid inequalities of $\mathcal{P}^j_{\text{KNP}}$ which are also valid for SPP-LRSP. Balas (1975), Padberg (1980), Crowder et al. (1983) and Weismantel (1997) are some of the papers studying the polyhedral structure of knapsack problems. Here, we evaluate applicability of the following cuts that are investigated in those studies to our algorithm.

One set of well-known nontrivial inequalities that are valid for $\mathcal{P}^j_{\text{KNP}}$ is the *cover inequalities*. Based on our notation and definition of $\mathcal{P}^j_{\text{KNP}}$, a set $S_j \subseteq P_j$ is called a cover if $\sum_{p \in S_j} D(p) > L^F_j$. And if the set S_j is minimal, then the following inequality is valid for $\mathcal{P}^j_{\text{KNP}}$.

$$\sum_{p \in S_j} z_p \leq |S_j| - 1 \tag{3.3}$$

Another set of nontrivial inequalities defined for knapsack polytopes is *(1-k)-configuration inequalities*. Let S_j be a subset of pairings for which the total demand is no more than the capacity (i.e.,$S_j \subseteq P_j$ and $\sum_{p \in S_j} D(p) \leq L^F_j$) and with the property that every one of its subsets is a minimal cover if you add one pairing from the pairings not in S_j. Then, the following inequality called (1-k)-configuration inequality is valid for $\mathcal{P}^j_{\text{KNP}}$.

$$(r - k + 1) z_q + \sum_{p \in S^r_j} z_p \leq r, \tag{3.4}$$

where r is some integer satisfying $k \leq r \leq |S_j|$, and S^r_j is a subset of S_j such that $|S^r_j| = r$.

When $k = |S_j|$, the inequality (3.4) is a cover inequality; therefore cover inequalities are also (1-k)-configuration inequalities.

In a branch and price algorithm, we may separate (3.3) and (3.4) based on the columns currently in the model, and we may add these into the current RMP. However, since these inequalities are with-z type inequalities (with nonzero coefficients for columns that are generated via column generation algorithm), unless we incorporate the information from these generated inequalities into the pricing problem, the inequalities are not helpful; the pricing problem will generate the same or similar columns that are visiting the same set of customers.

Our current solution algorithms for the pricing problem can handle the new information about a single customer or pair of customers with strict conditions of either following each other or not connected to each other in a pairing. In addition, a new information considering a set of customers can be included in the pricing algorithm if the information for each customer in the set is independent of another. These inequalities are constructed considering pairings and all customers included. The order of each customer in a pairing depends on the other customers that are included. Therefore, current pricing algorithms are not equipped to handle the new information from these generated cuts. At this time, it is not clear how to update pricing algorithms efficiently (without making it more difficult to solve) in order to handle the inequalities in the form of (3.3) and (3.4). Therefore we could use neither (3.3) nor (3.4) in our branch and price algorithm.

Next we discuss the set partitioning polytope and at the end of this discussion, there is an example from literature, in which the pricing algorithm is updated based on the new set of cuts under certain conditions. If there exists an efficient way of handling inequalities in the form of (3.3) and (3.4), they would improve the performance of the algorithm as they are successful in the solution of $\mathcal{P}^j_{\text{KNP}}$.

Set Partitioning Polytope Constraints (1.14) and the integrality constraints in $\mathcal{P}_{\text{SPP}}$ define set partitioning polytope. Thus, set partitioning polytope is a relaxation of $\mathcal{P}_{\text{SPP}}$, so are set packing and set covering polytopes union of which results the set partitioning polytope. Cuts for set partitioning problem are derived by investigating the polyhedral structures of set packing and set covering polytopes. See Balas and PAdberg (1976) for a review of polyhedral structures of all these polytopes. A common class of valid inequalities used in the literature is *clique inequalities*. A clique is a complete subgraph. For a set packing or set partitioning instance one can draw an intersection graph in which there is one node for each variable and there are edges between variables if they have positive coefficients in the same constraint. Based on such graph, the following inequality is called clique inequality

$$\sum_{p \in S} z_p \leq 1 \tag{3.5}$$

where S is the set of nodes in a clique in the intersection graph. Since all pairings in set S have a coefficient 1 in the same row, we can only select one of these pairings in an integer solution.

Another common class of inequalities used for set partitioning problems is *odd hole inequalities* that are constructed from cordless cycles with odd number of vertices in the intersection graphs. Let C be the set of columns representing vertices in an odd cycle, then the following is valid for the set partitioning polytope:

$$\sum_{p \in C} z_p \leq \left\lfloor \frac{|C|}{2} \right\rfloor \tag{3.6}$$

There are other types of inequalities such as anti-hole, webs and wheels (see Eso (1999) for details) that are derived from the intersection graphs in a similar way and have structure similar to (3.5) and (3.6). Here, our aim is to provide a short review of cuts valid for set partitioning problem. Because of the same reason explained for knapsack cuts earlier in this section, we did not implement any of these cuts. We do not have a way of updating pricing algorithm efficiently to incorporate the information from these cuts.

There is a branch, price and cut algorithm example in the literature (Jepsen et al. (2008)) in which the authors provide a procedure to update the pricing problem in order to use the information from a new class of cuts generated based on the set partitioning polytope. Now, we review this paper in order to further clarify why updating the pricing problem for inequalities in the form of (3.3) to (3.6) is not trivial and why the structure of the cut is very important for a branch and price algorithm. Jepsen et al. (2008) introduced *subset-row inequalities* that are based on the idea of clique and odd hole inequalities. However, they constructed these inequalities based on the information from the rows of the set partitioning polytope instead of considering certain set of variables as in clique and odd hole inequalities. Based on our notation, Jepsen et al. (2008)'s subset-row inequalities are as follows

$$\sum_{j \in J} \sum_{p \in P_j} \left\lfloor \frac{1}{k} \sum_{i \in I_k} a_{ip} \right\rfloor z_p \leq \left\lfloor \frac{|I_k|}{k} \right\rfloor, \tag{3.7}$$

where I_k is a subset of customers (rows in set partitioning constraints) and $0 < k \leq |I_k|$. A column visiting at least k of the customers in set I_k has a nonzero coefficient in inequality (3.7), otherwise the coefficient is zero. They used this condition for a column to update the reduced cost calculation in pricing problem which is also an ESPPRC.

The reduced cost of a column should be updated with $-\gamma \left\lfloor \frac{\sum_{i \in I_k} a_{ip}}{k} \right\rfloor$ amount where γ is the dual variable that corresponds to constraint (3.7). To do that, they kept value $m_l = (|I_k \cap V_l| \bmod k)$ for each label l in the ESPPRC algorithm where V_l is the set of customers in label l. They modified domination rules based on m_l value for the labels. They proved that these inequalities were effective and they improved the performance of their branch-and-price algorithm. However, the size of set I_k is important and larger values make the pricing algorithm more complicated. Since these cuts have already been applied in a branch and price algorithm, we do not include these to our study. We continue our investigation by looking at the projections of the SPP-LRSP polytope.

3.3.2 Projection of the SPP-LRSP

To eliminate the problem with the cuts (i.e.,dependency to specific columns and thus passing dependent information for customers) that are valid for the relaxation polytopes, we use projection. Here, projection is a mapping of a polytope to another based on a defined relation between variables. We let $v_{ji} = \sum_{p \in P_j} a_{ip} z_p$ for all $j \in J$ and $i \in I$ and project out the variables z. The projection polytope $\mathcal{P}^p_{\text{SPP}}$ defined based on this relation is

$$\mathcal{P}^p_{\text{SPP}} = \text{conv } \{t \in \{0,1\}^{|J|}, v \in \{0,1\}^{|J|\times|I|} | \sum_{j \in J} v_{ji} = 1 \quad \forall i \in I, \quad (3.8)$$

$$\sum_{i \in I} D_i v_{ji} - L^F_j t_j \leq 0 \quad \forall j \in J, \quad (3.9)$$

$$\}.$$

We can interpret v_{ji} as the fraction of the demand of customer $i \in I$ assigned to facility $j \in J$. Again, we investigate some relaxations of the polytope $\mathcal{P}^p_{\text{SPP}}$ to derive some cuts. Then, the obtained cuts can be projected back to the original variable space (t, z) and used for $\mathcal{P}_{\text{SPP}}$. We implemented two classes of the cuts that will be presented next in this section: the lifted cover inequalities and the generalized assignment polytope cuts. We extended our branch and price algorithm with these and presented some computational results in Section 4.5.1.

Multiple Knapsack Polytope If we replace variable t_j with 1 in inequalities (3.9) and relax the equality restriction in (3.8), these inequalities with the binary restriction for variable v define a multiple knapsack polytope:

$$\mathcal{P}_{\text{MKN}} = \text{conv } \{v \in \{0,1\}^{|J|\times|I|} | \sum_{j \in J} v_{ji} \leq 1 \quad \forall i \in I, \quad (3.10)$$

$$\sum_{i \in I} D_i v_{ji} \leq L^F_j \quad \forall j \in J, \quad (3.11)$$

$$\}.$$

Valid inequalities for the single knapsack polytope are also valid for $\mathcal{P}_{\text{MKN}}$. The cover inequalities and (1-k) configuration inequalities described in Section 3.3.1 can be applied here.

Let $S \subseteq I$ be a minimal cover for facility j such that $\sum_{i \in S} D_i > L^F_j$. Then, the following inequality is valid for $\mathcal{P}_{\text{MKN}}$ and $\mathcal{P}^p_{\text{SPP}}$.

$$\sum_{i \in S} v_{ji} \leq |S| - 1. \quad (3.12)$$

The inequalities (3.12) can be extended further to get stronger inequalities using *lifting* which is a procedure to extend an inequality to another possibly including nonzero coefficients for some

of the variables that have zero coefficient in the original inequality. The extension of the cover inequalities and the lifting procedures to get stronger inequalities are described in Nemhauser and Wolsey (1989) and Gu et al. (1998). To be more specific, after the lifting procedure, the inequality will be as follows:

$$\sum_{i \in S} v_{ji} + \sum_{i \in I \setminus S} \alpha_{ji} v_{ji} \leq \beta, \tag{3.13}$$

for some α and β values that are calculated in a lifting procedure. This inequality can be transformed to the following for $\mathcal{P}_{\text{SPP}}$:

$$\sum_{i \in S} \sum_{p \in P_j} a_{ip} z_p + \sum_{i \in I \setminus S} \sum_{p \in P_j} \alpha_{ji} a_{ip} z_p \leq \beta. \tag{3.14}$$

An inequality in the form of (3.14) can easily be added to the RMP without destroying the pricing problem structure, i.e., our current pricing problem solution algorithms can be used. The inequality (3.14) has the information of whether a customer visited by a pairing or not for all possible pairings (even the ones that are not currently in RMP). The inequality may have information about more than one customer, however this information for a customer is independent of other customers. Let ω_S^j be the dual variable that corresponds to the inequality (3.14) for customer set S and facility j. To incorporate the information from this inequality into our algorithm, in the pricing problem for facility j, we update the cost of incoming arcs for customers in set S with ω_S^j, and all other customers in $I \setminus S$ with $\omega_S^j \alpha_{ji}$. These updates for single customers result an updated total reduced cost for a column with a nonzero coefficient in inequality (3.14).

We implemented inequalities (3.14) that are derived from lifted cover inequalities in our branch and price algorithm. MINTO has a built-in function to detect violated lifted cover inequalities for general MIP instances. This function implements a heuristic algorithm details of which is described in Gu et al. (1998). We export this function into our algorithm to detect violated lifted cover inequalities for a given solution information based on $(t, v)^j$ variable vector that corresponds to some facility $j \in J$.

In our CUTGEN(RMP) procedure, for a given fractional solution $(\bar{t}, \bar{z}) \in \mathbb{R}^{+|J| + \sum_{j \in J} |\bar{P}_j|}$ where $\bar{P}_j$ is the current set of pairings for facility $j \in J$, we do the following.

- For each facility $j \in J$, if $\bar{t}_j = 1$, then we find $\bar{v}^j \in \mathbb{R}^{|I|}$ using relation $\bar{v}_{ji} = \sum_{p \in \bar{P}_j} a_{ip} \bar{z}_p$ for all $i \in I$.
- For a given $\bar{v}^j$, we search for a cover inequality in the form of (3.13) using MINTO's function.
- If the function returns a cut, then we rewrite it in the form of (3.14) and add to our RMP.

As mentioned earlier, computational results for the implementation of this set of cuts are presented in Section 4.5.1.

As mentioned in the knapsack polytope part in previous section, (1-k) configuration inequalities are valid for single knapsack polytope. Thus it is possible to derive (1-k) configuration inequalities for the $\mathcal{P}^p_{\text{SPP}}$ in terms of variable v. Similar to cover inequalities, (1-k) configuration inequalities in terms of variable v can be added to the RMP without causing significant changes in the pricing problem solution algorithms. Based on the computational experiments for lifted cover inequalities which are a special case of (1-k) configuration inequalities, we decided not to implement (1-k) configuration inequalities in our study.

Generalized Assignment Polytope Similar to the definition of $\mathcal{P}_{\text{MKN}}$, a generalized assignment polytope, $\mathcal{P}_{\text{GAP}}$ can be defined as follows:

$$\mathcal{P}_{\text{GAP}} = \text{conv}\{v \in \{0,1\}^{|J|\times|I|} | v \text{ satisfy (3.8) and } \sum_{i\in I} D_i v_{ji} \leq L^F_j \;\; \forall j \in J\}.$$

The generalized assignment problem (GAP) is a generalization of the multiple knapsack problem. The generalization is that the GAP may be formulated with different weights for each item (coefficients in 3.11, i.e.,D_i) for each knapsack. Thus, valid inequalities for the knapsack polytopes are also valid for $\mathcal{P}_{\text{GAP}}$. Besides the inequalities for the knapsack polytope, there are only a few papers investigating the facets and valid inequalities of GAP. Some of these are Gottlieb and Rao (1990), de Farias Jr. and Nemhauser (2001). Both papers relax constraint (3.8) and look at the relaxed polytope including $\leq$ part of inequalities (3.8). In this relaxed GAP, de Farias Jr. and Nemhauser (2001) added the condition that at most one of v_{ji} for all $j \in J$ can be positive for all $i \in I$. The following theorem from de Farias Jr. and Nemhauser (2001) presents a valid inequality for GAP. We rewrite the theorem based on our notation and variable definition.

Theorem 3.1 *[de Farias Jr. and Nemhauser (2001)] Let $S \subseteq I$ and $k \in J$. Suppose that $\sum_{i\in S} d_i > L^F_k$ and $\sum_{i\in S-\{r\}} d_i < L^F_k$ for some $r \in S$. Then,*

$$\sum_{i\in S} D_i v_{ki} + \sum_{i\in S} \left(max\{0, L^F_k - \sum_{a\in S-\{i\}} D_a\} \right) \sum_{j\in J-\{k\}} v_{ji} \;\leq\; L^F_k \tag{3.15}$$

defines a facet of relaxed GAP polytope.

The interpretation of (3.15) is simple: if the total demand of the customers in set S exceeds the capacity of facility k, then there must be some customer $r \in S$ that must be served by another facility and not by k. If the second part of the left hand side of (3.15) is zero, it is equal to the facility capacity constraint for k that is one of the trival inequalities. However, in a non integer solution, if the slack variable for the facility capacity constraint for k is zero (i.e., the capacity is fully used) and facility k has positive flow to all customers in set S, then the inequality (3.15) may likely be violated because of positive flow from other facilities to some customer $r \in S$ with

$L_k^F - \sum_{a \in S-\{r\}} d_a > 0$. We can rewrite inequality (3.15) in terms of pairing variables using the relation between v and z:

$$\sum_{i \in S} d_i \sum_{p \in P_k} a_{ip} z_p + \sum_{i \in S} \left(\max\{0, L_k^F - \sum_{a \in S-\{i\}} D_a\} \right) \sum_{j \in J-\{k\}} \sum_{p \in P_j} a_{ip} z_p \leq L_k^F \tag{3.16}$$

Since the inequality (3.16) carries independent information of presence of a customer in a pairing for each customer in set S, it can be added to the RMP without destroying the structure of the pricing problem. We implemented this class of inequalities which we refer as GAP inequalities. We extended our branch and price algorithm with (3.16) and evaluate the overall performance. Let γ_S^k be the dual variable corresponding to the inequality (3.16) for customer set S and facility k. To incorporate this inequality into our algorithm, in the pricing problem for facility k, we update the incoming arcs for customers in set S with $D_i \gamma_S^k$, and in the pricing problem for facility $j \neq k$, we update the incoming arcs for customers in set S with $(\max\{0, L_t^F - \sum_{a \in S-\{i\}} D_x\})\gamma_S^k$.

de Farias Jr. and Nemhauser (2001) present the following separation heuristic that is described based on our notation for inequality (3.15):

Heur-Sep-GAP [de Farias Jr. and Nemhauser (2001)]

0. Let $\bar{v}$ be a fractional solution to the LP relaxation of $\mathcal{P}_{\text{GAP}}$.
1. For each facility $k \in J$:
   i. If $\sum_{i \in I} D_i \bar{v}_{ki} = L_k^F$, then proceed. Otherwise go to step 1 and pick the next facility.
   ii. Let $S = \{i \in I | \bar{v}_{ki} > 0\}$ and $D(S) = \sum_{i \in S} D_i$.
   If $D(S) > L_k^F$, then for each $c \in S$, check the following conditions:
   – Is c also served by another facility?

$$1 > v_{kc} > 0, \text{ and } \sum_{j \in J \setminus \{k\}} v_{jc} > 0 \tag{3.17}$$

   – When we remove c from assignment set of facility k, does full assignment of the rest of the customers to k becomes feasible?

$$D(S \setminus \{c\}) < L_k^F \tag{3.18}$$

   iii. If there is at least one $c \in S$ satisfying both (3.17) and (3.18), then (3.15) is violated for facility k and customer set S. Return a GAP cut for facility k.

In our CUTGEN procedure, similar to the lifted cover inequalities, for a given fractional solution $(\bar{t}, \bar{z}) \in \mathbb{R}^{+|J| + \sum_{j \in J} |\bar{P}_j|}$ where $\bar{P}_j$ is the current set of pairings for facility $j \in J$, we find $(\bar{t}, \bar{v}) \in$

$\mathbb{R}^{|J|+|J|\times|I|}$ using relation $\bar{v}_{ji} = \sum_{p\in\bar{P}_j} a_{ip}\bar{z}_p$ for all $i \in I$ and $j \in J$. Then, we call Heur-Sep-GAP to separate cuts in the form of inequality (3.15).

We updated Heur-Sep-GAP slightly in our implementation. In step 1(i), we also require that the facility variable is 1, i.e., $t_k = 1$. Then, we rewrite a generated inequality in the form of (3.16). In the LRSP instances, most of the time Heur-Sep-GAP fails to find violated cuts. The reason for this failure is condition (3.18). Generally, condition (3.18) is not satisfied even though the facility capacity constraint is binding and (3.17) is true. In LRSP, we are dealing with the assignment of pairings (i.e., sets of customers to the facilities), therefore, fractional pairing variables lead to multiple customers (rather than a single customer) that are served by more than one facility. In a fractional solution, for some facility, even though the capacity is fully utilized, since more than one customer is served by a fractional amount from that facility, it is not enough to remove the assignment of a single customer to this facility to satisfy that the total demand of the remaining customers is less than the facility capacity, (i.e.,condition (3.18)). To increase the number of times a cut is generated, we updated determination of the set S in the step 1 (ii) of the Heur-Sep-GAP. We may shrink this set S whenever it is necessary.

In our algorithm, first we process step 1 (ii) and (iii) as in the original Heur-Sep-GAP procedure by setting $S = \{i \in I | v_{ki} > 0\}$. If we cannot find a violation, we reduce the size of set S by removing one customer at a time until a violation is found or $D(S) \leq L_k^F$. We remove customers in the order of non increasing demand. The steps of the updated algorithm can be listed as follows:

Heur-Sep-GAP-2

0. Let $(\bar{t}, \bar{v})$ be a fractional solution.
1. For each facility $k \in J$:
   i. If $\bar{t}_k = 1$ and $\sum_{i\in I} D_i\bar{v}_{ki} = L_k^F$, then proceed. Otherwise return to step 1.
   ii. Let $S = \{i \in I | \bar{v}_{ki} > 0\}$. If $D(S) > L_k^F$, then for each $c \in S$, check the conditions (3.17) and (3.18).
   iii. If there is at least one $c \in S$ satisfying (3.17) and (3.18), then return a cut for facility k and set S and return to step 1.
   iv. Sort the customers in S, $i_1 < i_2 < .. < i_{|S|}$ such that $D_{i_p} \geq D_{i_q}$ for all $p, q = 1, .., |S|$ and $i_p < i_q$.
   v. For $e = 1$ to $|S|$:
      a. Let $S = S - \{i_e\}$. If $D(S) \leq L_k^F$, then return to step 1.
      b. If there is at least one $c \in S$ satisfying (3.17) and (3.18), then return a cut for facility k and set S and return to step 1.

As mentioned earlier, computational results for the implementation of this set of cuts are presented

in Section 4.5.1.

Capacitated Facility Location Polytope In $\mathcal{P}^p_{\text{SPP}}$, if we relax the integrality of variable v and set bounds, we get a capacitated facility location polytope, $\mathcal{P}_{\text{CFL}}$. To be more specific:

$$\mathcal{P}_{\text{CFL}} = \text{conv } \{t \in \{0,1\}^{|J|}, v \in \mathbb{R}^{|J|\times|I|} | t, v \text{ satisfy } (3.8), (3.9), \text{ and } v_{ji} \in [0,1] \forall j \in J, i \in I\}.$$

Therefore, $\mathcal{P}_{\text{CFL}}$ is a relaxation of $\mathcal{P}^p_{\text{SPP}}$, and if the valid inequalities for $\mathcal{P}_{\text{CFL}}$ are projected to (t, z) space, they are valid for $\mathcal{P}_{\text{SPP}}$. Indeed, $\mathcal{P}^p_{\text{SPP}}$, the capacitated facility location polytope with binary restriction on variable v, is called the single source capacitated facility location polytope. In the problem, each customer has to be served by a single facility. We do not aware of any studies that investigate specifically the polyhedral structure of single source capacitated facility location problem, thus we review the studies for $\mathcal{P}_{\text{CFL}}$ which are a few. Aardal et al. (1995), Leung and Magnanti (1989) and Padberg et al. (1985) investigated valid inequalities and gave conditions for these inequalities to be facets for $\mathcal{P}_{\text{CFL}}$. We did not implement any of these inequalities, here we give a brief list of common ones.

One set of valid inequalities is the *knapsack cover inequality* in terms of t (facility) variables. Aardal et al. (1995) define "cover" that is a subset of facilities. Using our notation, M is a cover if $\sum_{j\in M} L^F_j > \sum_{j\in J} L^F_j - \sum_{i\in I} D_i$. Based on this definition, they state the following theorem:

Theorem 3.2 *[Aardal et al. (1995)] If M is a minimal cover with respect to J and I, $L^F_{min} = min_{j\in M}\{L^F_j\}$, and $\sum_{j\in J\setminus M} L^F_j + L^F_{min} > \sum_{i\in I} D_i$, then the knapsack cover inequality:*

$$\sum_{j\in M} t_j \geq 1 \tag{3.19}$$

defines a facet of $\mathcal{P}_{CFL} \cap \{t \in \{0,1\}^{|J|} : t_j = 1 \qquad \forall j \in J\setminus M\}$.

This theorem implies that at least one facility from set M must be open because the total capacities of facilities in the set $J\setminus M$ is not enough to cover the total demand of customers. Since 3.19 is valid for $\mathcal{P}_{\text{CFL}}$, then it is also valid for $\mathcal{P}^p_{\text{SPP}}$ and $\mathcal{P}_{\text{SPP}}$. Since the inequality defined in Theorem (3.2) only includes t variables, in other words it is a z-free inequality, we can use this inequality without affecting the structure of the pricing problem.

We did not implement this inequality in our study because of the effect of inequality (1.18) in the RMP. In our LRSP instances used for computational results, the facility capacity values are equal. Based on this, when all possible (3.19) inequalities are generated, summed and combined into one inequality, we get $\sum_{j\in J} t_j \geq m$ where $m \in \mathbb{R}^+$. With some algebra, we can easily show that the minimum number of required facilities, R (in (1.18)) is greater than m, if $1 < R < |J|$ (which is the case in our instances). Thus, the inequality (1.18) forcing that the total number of facilities must be greater than or equal to R is stronger than the inequality obtained by summing

all possible inequalities (3.19). In addition, we do not add the inequalities (3.19) to the SPP-LRSP, since generally inequality (1.18) results integer values for location variables.

Another class of valid inequality derived for $\mathcal{P}_{\text{CFL}}$ is *flow cover inequality*. Aardal et al. (1995) and Padberg et al. (1985) define M as flow cover if M is a subset of facilities and can strictly cover the total demand of customers, i.e., $\sum_{j\in M} L_j^F = \sum_{i\in I} D_i + \lambda$, for some $\lambda > 0$. Aardal et al. (1995) and Padberg et al. (1985) present the following theorem (based on the notation in this section) that describes the flow cover inequalities:

Theorem 3.3 *[Aardal et al. (1995)] Let $M \subset J$ be a flow cover with respect to J and I. If $max_{j\in M}\{L_j^F\} > \lambda$ and $\sum_{j\in J} L_j^F > \sum_{i\in I} D_i + L_r^F \ \forall r \in M$, then the flow cover inequality*

$$\sum_{j\in M}\sum_{i\in I} D_i v_{ji} + \sum_{j\in M}(L_j^F - \lambda)^+(1-t_j) \leq \sum_{i\in I} D_i \tag{3.20}$$

defines a facet of $\mathcal{P}_{CFL}$.

The inequality (3.20) defines a minimum set of facilities (i.e., M) to cover the total demand. If all customers are assigned to the facilities in M, the excess capacity λ leads at least one facility in set M to be fractional in an LP solution. The aim of the inequality (3.20) is to eliminate the effect of excess capacity. The second part in the left hand side of the inequality forces all facilities in set M take value close to 1 to satisfy the inequality. In inequality (3.20), if we replace v_{ji} variable with $\sum_{p\in P_j} a_{ip} z_p$, we have the following inequality in (t, z) space.

$$\sum_{j\in M}\sum_{p\in P_j}\sum_{i\in I} D_i a_{ip} z_p + \sum_{j\in M}(L_j^F - \lambda)^+(1-t_j) \leq \sum_{i\in I} D_i \tag{3.21}$$

Similar to the other inequalities defined in this section, we can incorporate the inequality (3.21) into a branch and price algorithm. However, we did not implement this inequality in our study, because in most of our LP relaxation solutions, the facilities serving the customers are fully open.

3.4 Disjunctive Cuts for LRSP

In this section, we discuss the generation of a set of cuts derived based on split disjunctions in the context of branch and price algorithm. We begin with a review of the methodology of disjunctive cuts. The generation of disjunctive cuts and the idea of disjunctive programming were introduced by Balas and Perregaard (1979). Disjunctive programming is finding the optimal value of an objective function over a finite union of polyhedra constructed based on conjunctive statements (connected with logical operator "AND") and disjunctive statements (connected with logical operator "OR"). Here, we focus on cuts based on disjunctive statements. To explain the methodology for an integer

model, suppose we have an integer program, P:

$$\begin{aligned} & \text{Minimize} \quad c^T x \\ \text{(P)} \quad & \text{subject to} \quad Ax \geq d, && (3.22) \\ & x \geq 0, \text{ and integer} && (3.23) \end{aligned}$$

where $A \in \mathbb{Q}^{m \times n}$ is constraint matrix, $c \in \mathbb{Q}^n$, $d \in \mathbb{Q}^m$ are rational vectors and $x \in \mathbb{Z}^n$ is a vector of decision variables. In addition, suppose we know that any feasible integer solution x for P, also satisfies at least one of the following s inequalities based on some disjunctive statements:

$$B^1 x \geq b^1, \tag{3.24}$$

$$B^2 x \geq b^2, \tag{3.25}$$

$$\ldots$$

$$B^s x \geq b^s, \tag{3.26}$$

where $B^k \in \mathbb{Q}^{m^k \times n}$ is constraint matrix and $b^k \in \mathbb{Q}^{m^k}$ is right hand side vector for some integer $m^k > 0, \forall k = 1, .., s$. Using these s disjunctive set of inequalities, a relaxation of P, P_{DR} can be defined:

$$\begin{aligned} & \text{Minimize} \quad c^T x \\ (\text{P}_{\text{DR}}) \quad & \text{subject to} \quad Ax \geq d, \\ & \vee_{k=1}^{s} B^k x \geq b^k, \\ & x \geq 0, \end{aligned}$$

where $\vee$ denotes "or" and is used to represent the disjunctive system. Using each disjunction, we can define s polyhedron.

$$\text{P}^k = \{x \in \mathbb{R}^{+n} | Ax \geq d, B^k x \geq b^k\} \ \forall k = 1, .., s$$

Based on the definitions of P^k and P_{DR}, P_{DR} can be represented by the union of these s polyhedron, i.e., $\text{P}_{\text{DR}} = \cup_{k=1}^{s} \text{P}^k$. Since P_{DR} is a relaxation of P, any valid inequality for P_{DR} is also valid for P and $\text{P} \subseteq \text{P}_{\text{DR}} \subseteq \text{P}_{\text{LP}}$, where P_{LP} is the LP relaxation of P. The aim of the disjunctive cut generation methodology is to drive an inequality, $\alpha x \geq \beta$ that is valid for P_{DR} and is violated by some points of P_{LP}.

Balas and Perregaard (1979) show that based on the assumption that P^k is non empty for all $k = 1, .., s$, $\alpha x \geq \beta$ is valid for P_{DR} if and only if there exist multipliers $u^k \in \mathbb{Q}^m$ and $v^k \in \mathbb{Q}^{m^k}$

such that

$$u^k A + v^k B^k \leq \alpha, \quad \forall k = 1, .., s \tag{3.27}$$
$$u^k b + v^k b^k = \beta, \quad \forall k = 1, .., s \tag{3.28}$$
$$u^k, v^k \geq 0. \quad \forall k = 1, .., s \tag{3.29}$$

Let $\bar{x}$ be a solution to the LP relaxation of P. To find a cut $\alpha x \geq \beta$ that is maximally violated by $\bar{x}$ and valid for $\mathrm{P_{DR}}$ (separation problem), we need to solve the following optimization model, which is called *cut generating LP* (CGLP).

$$\text{(CGLP) Minimize} \quad \alpha\bar{x} - \beta \tag{3.30}$$
$$u^k A + v^k B^k \leq \alpha, \quad \forall k = 1, .., s \tag{3.31}$$
$$u^k b + v^k b^k = \beta, \quad \forall k = 1, .., s \tag{3.32}$$
$$\sum_{i=1}^{n} u_i^k + \sum_{i=1}^{n} v_i^k = 1, \tag{3.33}$$
$$u^k, v^k \geq 0. \quad \forall k = 1, .., s \tag{3.34}$$

Constraint (3.33) is added to normalize the cut to be generated. The purpose of the normalization constraint is to truncate the cone defined by constraints (3.27) - (3.29) to a polyhedron. The normalization removes the unboundedness of the model and transforms extreme rays to extreme points. Here, we use one of the common normalization constraints, different versions of normalization constraints are described in the literature. Some examples are $\alpha-$normalization (i.e.,$\sum_{i=1,..,n} |\alpha_i| = 1$) and $\beta-$normalization (i.e.,$|\beta| = 1$) (Balas et al. (1996)).

As noted in Caprara and Letchford (2003) (based on the results in Balas et al. (1993) and Balas et al. (1996))), for a fixed disjunctive system, i.e., (3.24) - (3.26), separation of disjunctive cuts can be done in polynomial time.

In general, to derive disjunctive cuts, it is common to use disjunctive systems including two linear inequalities (i.e.,$s = 2$). One of the most common and successful set of disjunctive cuts is referred to as *lift-and-project cuts* (see Balas et al. (1993) and Balas et al. (1996)) that are derived from bounds of single binary variables. The disjunctions used to generate lift-and-projects cuts are in the form of $(x_i \leq 0) \vee (x_i \geq 1)$ for some binary variable x_i. Lift-and-project cuts are one of the widely used general purpose cutting planes for 0-1 mixed integer programming models. For models with a polynomial number of constraints, separation of these cuts can be done in polynomial time by solving the cut generating LP. In addition to this advantage, the problem of having to solve large cut generating LPs for many iterations in a branch-and-cut based algorithm was eliminated with results presented in Balas and Perregaard (2003). They showed that instead of explicitly formulating and solving CGLP, the simplex tableau of the current LP solution can be used to derive this type of cuts.

If the disjunctions used to derive cuts include more than one variable and are in the form of $(\pi x \leq \pi_0) \vee (\pi x \geq \pi_0 + 1)$ for some integer vector $(\pi, \pi_0) \in \mathbb{Z}^{n+1}$, they are called *split disjunctions*. And the cuts generated are referred to as *split cuts* (Cook et al. (1990) and Balas and Saxena (2008)). Caprara and Letchford (2003) proved that the separation problem for split cuts is strongly NP-Complete. Many of the general purpose cutting planes for mixed integer programming models can be included in the class of split cuts. Lift-and-project cuts are simpler forms of split cuts. In addition, mixed integer cuts of Gomory (Gomory (1963)) and mixed integer rounding inequalities (Nemhauser and Wolsey (1990)) are shown to be in the class of split cuts (Cornuejols and Li (2001) and Cornuejols (2006)).

In addition to these studies that derive cutting planes based on some disjunctions, Letchford (2001) proved that many facet defining inequalities for some well-known combinatorial optimization problems investigated before are actually disjunctive cuts derived from certain simple disjunctions. They used these results to prove that the separation problems for these facet inducing inequalities are solvable in polynomial time. They provided results for some facets of the clique partitioning problem, the maximum cut problem, the acyclic sub digraph problem, the linear ordering problem, the asymmetric traveling salesman problem and the set covering problem.

As mentioned before, many general purpose cutting planes (such as Gomory cuts, mixed integer rounding cuts) for mixed integer programming models can be derived based on disjunctions and can be included in the class of disjunctive cuts. Although these general purpose cutting planes are very popular in a branch and cut based algorithm, we haven't seen a study in the literature that is utilizing disjunctive cuts in a branch and price algorithm before, except in a recent presentation describing the application of disjunctive cutting planes in a branch and price algorithm developed for capacitated VRP (Ropke (2008)). Next, we describe some implementations of disjunctive cutting planes designed for our branch and price algorithm for LRSP. Since we are applying a column generation algorithm, we avoided using formulation SPP-LRSP to generate disjunctive cuts. Instead we used some projections and relaxations of SPP-LRPS and G-LRSP. In addition, the structure of the generated cutting plane is crucial to be able to include it to the column generation algorithm. We designed four implementations generating disjunctive cuts based on some formulations of LRSP. The main steps of the implementations are similar and can be outlined as follows:

DISJCUTGEN:

0. Let $\bar{y}$ be a given fractional solution to the RMP (at any node).
1. Choose a model (representing a relaxation of the LRSP or its projection based on some formulation (SPP-LRSP or G-LRSP)) to derive cuts. Let P be the selected model (as defined in this section).
2. If P is from a projection, let $\bar{x}$ be the solution to P corresponding to $\bar{y}$.
3. Select two disjunctive inequalities violated by $\bar{x}$ but satisfied by all integer feasible solutions (we list possible disjunctions to be considered for each implementation).

   Let $B^1x \geq b^1 \vee B^2x \geq b^2$ be the selected disjunction.
4. Construct CGLP as defined by constraints (3.31) - (3.34) and objective (3.30).
5. Solve CGLP. If the objective function value is negative, then $\alpha x \geq \beta$ is a valid cut violated by $\bar{x}$. Else STOP with no cut generated.
6. If P is from a projection, project the generated cut into the original space, let $\alpha' y \geq \beta'$ be the obtained cut.

In each of four implementations, step 1 is different. The procedures start with different models and generate cuts based on the selected model. The details of each implementation are explained next.

3.4.1 Implementation 1

In this implementation, we use the projection of $\mathcal{P}_{\text{SPP}}$, $\mathcal{P}^p_{\text{SPP}}$ described in Section 3.3.2. Let $(\bar{t}, \bar{z}) \in \mathbb{R}^{+|J|+\sum_{j\in J}|\bar{P}_j|}$ be a fractional solution to the RMP (at any node of the tree, after column generation algorithm is complete) where $\bar{P}_j$ is the current set of pairings for facility $j \in J$. Then we can find $(\bar{t}, \bar{v}) \in \mathbb{R}^{+|J|+|J|\times|I|}$ with relation $\bar{v}_{ji} = \sum_{p\in\bar{P}_j} a_{ip}\bar{z}_p$ for all $i \in I$, $j \in J$.

We check the following disjunctions to find two disjunctive inequalities violated by the fractional solution $(\bar{t}, \bar{v})$.

1. Flow from a facility to a customer is either 1 or 0:

$$v_{ji} \leq 0 \ \vee \ v_{ji} \geq 1 \quad \forall j \in J, i \in I. \tag{3.35}$$

2. Total number of customers assigned to a facility is an integer:

$$\sum_{i\in I} v_{ji} \leq g_j \ \vee \ \sum_{i\in I} v_{ji} \geq g_j + 1 \quad \forall j \in J, \tag{3.36}$$

for some integer g_j and for all $j \in J$.

After deciding on the form (3.35) or (3.36) of the violated disjunctive inequalities, we write the CGLP ((3.30) - (3.34)) with $s = 2$ based on model $\mathcal{P}^p_{\text{SPP}}$. As we noted in the description of the branch and price algorithm, we use AUG-SPP-LRSP in the algorithm, thus $\mathcal{P}^p_{\text{SPP}}$ is augmented with projections of inequalities (1.18) and (1.25). The LP relaxation of the projected model used for the CGLP is as follows:

$$[M_1] \qquad \sum_{j \in J} v_{ji} = 1 \quad \forall i \in I, \tag{3.37}$$

$$\sum_{i \in I} D_i v_{ji} - L^F_j t_j \leq 0 \quad \forall j \in J, \tag{3.38}$$

$$\sum_{j \in J} t_j \geq R, \tag{3.39}$$

$$v_{ji} - t_j \leq 0 \quad \forall j \in J, \forall i \in I, \tag{3.40}$$

$$0 \leq t_j \leq 1 \quad \forall j \in J, \tag{3.41}$$

$$0 \leq v_{ji} \leq 1 \quad \forall j \in J, \forall i \in I. \tag{3.42}$$

We investigate a disjunctive cut that is maximally violated by $(\bar{t}, \bar{v})$ in the following form:

$$\sum_{j \in J} \alpha^t_j t_j + \sum_{j \in J} \sum_{i \in I} \alpha^v_{ji} v_{ji} \geq \beta. \tag{3.43}$$

The objective function of the CGLP is to minimize $\sum_{j \in J} \alpha^t_j t_j + \sum_{j \in J} \sum_{i \in I} \alpha^v_{ji} v_{ji} - \beta$. The solution to the CGLP provides coefficients $(\alpha, \beta) \in \mathbb{R}^{|J|+|J|\times|I|+1}$, and we can rewrite a cut in the form of (3.43) in the (t, z) variable space:

$$\sum_{j \in J} \alpha^t_j t_j + \sum_{j \in J} \sum_{p \in P_j} \sum_{i \in I} \alpha^v_{ji} a_{ip} z_p \geq \beta. \tag{3.44}$$

A cut in the form of (3.44) can easily be added to the column generation algorithm by adding cost $\alpha^v_{ji}\omega$ to the cost of the links to customer i for all $i \in I$ in the pricing problem for facility $j \in J$ where ω is the dual variable that corresponds to (3.44).

3.4.2 Implementation 2

In the second implementation of DISJCUTGEN, in step 1, we choose a graph based formulation which is a projection of a relaxation of G-LRSP. In SPP-LRSP, each z variable corresponds to a possible vehicle schedule, and since all vehicles are identical, the vehicles are not differentiated with an index. In the light of this information, to be able to relate a generated cut to model SPP-LRSP, we project the variables in G-LRSP to a space with variables without the vehicle index. Let

$\sum_{h \in H_j} x_{kih} = u_{kij}$ for all $(k,i) \in A$, $j \in J$, and $w_j = \sum_{h \in H_j} v_h$ for all $j \in J$. Variable u_{kij} will take value 1 if any vehicle located at facility j travels link (k,i). w_j is the number of vehicles used at facility j. We start the implementation DISJCUTGEN by choosing the following graph based formulation in step 1.

$$[M_2] \qquad \sum_{j \in J} \sum_{k \in V} u_{kij} = 1 \quad \forall i \in I, \tag{3.45}$$

$$-\sum_{i \in I} D_i \sum_{k \in V} u_{kij} + C_j^F t_j \geq 0 \quad \forall j \in J, \tag{3.46}$$

$$\sum_{k \in V} u_{ikj} - \sum_{k \in V} u_{kij} = 0 \quad \forall i \in I, j \in J, \tag{3.47}$$

$$TLw_j - \sum_{k \in V} \sum_{i \in V} t_{ki} u_{kij} \geq 0 \quad \forall j \in J, \tag{3.48}$$

$$0 \leq t_j \leq 1 \quad \forall j \in J, \tag{3.49}$$

$$0 \leq u_{ikj} \leq 1 \quad \forall i, k \in V, j \in J, \tag{3.50}$$

$$w_j \geq 0 \quad \forall j \in J. \tag{3.51}$$

Constraints in formulation M_2 are similar to formulation G-LRSP. Constraints (3.45) require that each customer must be served. Constraints (3.46) are facility capacity constraints. Constraints (3.47) equate the number of vehicles in and out of a customer. G-LRSP has a set of constraints that are similar to (3.47) but for each vehicle, M_2 has the aggregated version (aggregated for all vehicles located at the same facility). Again, constraints (3.48) are the aggregated version of the time limit constraints in G-LRSP. While constructing formulation M_2, in addition to projecting (x, v) in G-LRSP to (u, w) (i.e.,aggregating vehicles located at the same facility), we relaxed some constraints of G-LRSP such as the constraints that are including flow variables.

Flow variables (y in G-LRSP) keep the amount of vehicle load on the links. The reason for relaxing the constraints including flow variables is that we want to avoid generating cuts which are including flow variables (or any projection of flow variables) with nonzero coefficients. Based on the DWD procedure, we know how to rewrite the flow variables in terms of master problem variables (z) using relation (1.38). However, we do not know how to incorporate an information passed from the flow variables into the pricing solution algorithm. Therefore, we want to avoid generating a cut that includes flow variables. By using model M_2, we aim to generate disjunctive cuts that do not include flow variables. Model M_2 enables a search for disjunctive cuts in the following form:

$$\sum_{j \in J} \alpha_j^t t_j + \sum_{j \in J} \alpha_j^w w_j + \sum_{j \in J} \sum_{k \in V} \sum_{i \in V} \alpha_{kij}^u u_{kij} \geq \beta. \tag{3.52}$$

As in implementation 1, let $(\bar{t}, \bar{z}) \in \mathbb{R}^{+|J| + \sum_{j \in J} |\bar{P}_j|}$ be a fractional solution to optimal LP relaxation of RMP. We can find $(\bar{t}, \bar{w}, \bar{u}) \in \mathbb{R}^{+|J| + |J| + |J| \times |A|}$ using relations $\bar{w}_j = \sum_{p \in \bar{P}_j} \bar{z}_p$,

$\bar{u}_{kij} = \sum_{p \in \bar{P}_j} l_{kip} \bar{z}_p$ for all $(k,i) \in A$, $j \in J$ where l_{kip} is 1 if pairing p includes link (k,i) and 0 otherwise.

If $(\bar{t}, \bar{w}, \bar{u})$ is fractional, then we consider the following disjunctions in order to determine two disjunctive inequalities violated by this solution.

1. A facility is either open or closed.

$$t_j \leq 0 \ \vee \ t_j \geq 1 \quad \forall j \in J. \tag{3.53}$$

2. Total number of vehicles used at a facility is an integer.

$$w_j \leq g_j \ \vee \ w_j \geq g_j + 1 \quad \forall j \in J, \tag{3.54}$$

for some integer g_j and for all $j \in J$.

3. Number of links into a set of customers is integer. Let S be a customer set such that $2 \leq |S| \leq 3$:

$$\begin{aligned} \sum_{j \in J} \Big(\sum_{k \in V \setminus S} \sum_{i \in S} u_{kij} + \sum_{k \in V \setminus S} \sum_{i \in S} u_{ikj} \Big) &\leq g \\ \vee \\ \sum_{j \in J} \Big(\sum_{k \in V \setminus S} \sum_{i \in S} u_{kij} + \sum_{k \in V \setminus S} \sum_{i \in S} u_{ikj} \Big) &\geq g+1, \end{aligned} \tag{3.55}$$

for some integer g.

By solving the CGLP constructed using model M_2 and one set of violated disjunctive inequalities, we investigate for a cut that is maximally violated (by fraction solution $(\bar{t}, \bar{w}, \bar{u})$) and is in the form of (3.52). A solution to the CGLP provides coefficients $(\alpha, \beta) \in \mathbb{R}^{|J|+|J|\times|A|+1}$, and we can rewrite the cut in terms of variables t, w and z:

$$\sum_{j \in J} \alpha_j^t t_j + + \sum_{j \in J} \alpha_j^w w_j + \sum_{j \in J} \sum_{p \in P_j} \sum_{(i,k) \in A} \alpha_{ikj}^u l_{ikp} z_p \geq \beta. \tag{3.56}$$

Note that w_j is the number of vehicles located at facility $j \in J$ and defined with relation $\bar{w}_j = \sum_{p \in P_j} z_p$ in AUG-SPP-LRSP. A cut in the form of (3.56) can be added to the column generation. Let ω be the dual variable corresponds to (3.56). In the pricing algorithm for facility j for all $j \in J$, we update the cost of link (i,k) with $\alpha_{ikj}^u \omega$ for all $(i,k) \in A$ and we update the total cost of all labels at the end with fixed cost $\alpha_j^w \omega$.

3.4.3 Implementation 3

In the third implementation, instead of a relaxation or a projection, model G-LRSP is selected in step 1 of the disjunctive cut generation algorithm. The CGLP will be formulated based on the LP relaxation of the G-LRSP. Recall that the model is as follows:

$$
\begin{aligned}
[M_3] \qquad \sum_{h\in H}\sum_{k\in N} x_{ikh} &= 1 \quad \forall i \in I,\\
\sum_{k\in N} x_{ikh} - \sum_{k\in N} x_{kih} &= 0 \quad \forall i \in N, h \in H,\\
\sum_{h\in H_j}\sum_{k\in I} y_{jkh} - L_j^F t_j &\le 0 \quad \forall j \in J,\\
y_{ikh} - L^V x_{ikh} &\le 0 \quad \forall (i,k) \in A, h \in H,\\
\sum_{k\in N} y_{ikh} - \sum_{k\in N} y_{kih} + D_i \sum_{k\in N} x_{ikh} &= 0 \quad \forall i \in I, h \in H,\\
\sum_{(i,k)\in A} T_{ik} x_{ikh} - L^T v_h &\le 0 \quad \forall h \in H,\\
0 \le t_j &\le 1 \quad \forall j \in J,\\
0 \le x_{ikh} &\le 1 \quad \forall i,k \in V, h \in H,\\
0 \le v_h &\le 1 \quad \forall h \in H,\\
y_{ikh} &\ge 0 \quad \forall i,k \in V, h \in H.
\end{aligned}
$$

Based on model M_3 and one of the disjunctions (3.53) - (3.55), we formulate the CGLP. Disjunction (3.55) can be rewritten in terms of x variables in M_3:

$$
\begin{aligned}
\sum_{j\in J}\Big(\sum_{k\in V\setminus S}\sum_{i\in S}\sum_{h\in H_j} x_{kih} + \sum_{k\in V\setminus S}\sum_{i\in S}\sum_{h\in H_j} x_{kih}\Big) &\le g\\
\vee \qquad\qquad\qquad\

We let $(\bar{t}, \bar{z}) \in \mathbb{R}^{+|J|+\sum_{j\in J}|\bar{P}_j|}$ be a fractional solution to the LP relaxation of RMP where $\bar{P}_j$ be the set of pairings with nonzero values in the solution, i.e.,$\bar{z}_p > 0$ for all $p \in \bar{P}_j$ for all $j \in J$. We can generate vector $(\bar{t}, \bar{v}, \bar{x}, \bar{y}) \in \mathbb{R}^{+|J|+|J|\times|H|+|H|\times|A|+|H|\times|A|}$ representing the projection of the RMP solution for model M_3. Since we have identical vehicles, we can assign nonzero pairings from 1 to $|\bar{P}_j|$ for all $j \in J$ to vehicles (in M_3) arbitrarily, starting from the first vehicle. Let $(h \leftrightarrow p)$ represent such correspondence where p is the h^{st} smallest index in $\bar{P}_j$ for all $h = 1, .., |\bar{P}_j|$ and $j \in J$. Note that a vector $(\bar{t}, \bar{z})$ may lead to multiple corresponding $(\bar{t}, \bar{v}, \bar{x}, \bar{y})$ vectors under the identical vehicles assumption. By defining correspondence $(h \leftrightarrow p)$ between h and p, we restrict the correspondence between vectors $(\bar{t}, \bar{z})$ and $(\bar{t}, \bar{v}, \bar{x}, \bar{y})$ to one-to-one. To construct $(\bar{t}, \bar{v}, \bar{x}, \bar{y})$, we use the following relations based on defined correspondence between index h and p:

$$\bar{v}_h = \begin{cases} \bar{z}_p & \text{If } (h \leftrightarrow p), \forall p \in \bar{P}_j, h \in H_j, j \in J, \\ 0 & \text{If } h = |\bar{P}_j| + 1, .., |H_j|, \forall j \in J, \end{cases}$$

$$\bar{x}_{ikh} = \begin{cases} l_{ikp}\bar{z}_p & \text{If } (h \leftrightarrow p), \forall p \in \bar{P}_j, h \in H_j, j \in J, \\ 0 & \text{If } h = |\bar{P}_j| + 1, .., |H_j|, \forall j \in J, \end{cases}$$

$$\bar{y}_{ikh} = \begin{cases} f_{ikp}\bar{z}_p & \text{If } (h \leftrightarrow p), \forall p \in \bar{P}_j, h \in H_j, j \in J, \\ 0 & \text{If } h = |\bar{P}_j| + 1, .., |H_j|, \forall j \in J, \end{cases}$$

where l_{ikp} is 1 if pairing p includes link (i, k) and f_{ikp} is equal to the vehicle load on link (i, k) of pairing p, for all $(i, k) \in A$, $p \in P_j$ and $j \in J$. After a cut that is in the form of (3.58) and satisfying (3.59) is generated for a given fractional solution, $(\bar{t}, \bar{v}, \bar{x}, \bar{y})$, we can write more cuts by circulating coefficients α^x_{ikh} and α^v_h over all $h \in H_j$ for all $j \in J$ using the assumption of identical vehicles. In other words, we assume that we obtain multiple $(\bar{t}, \bar{v}, \bar{x}, \bar{y})$ vectors from one $(\bar{t}, \bar{z})$ vector (one-to-many correspondence between solution vectors). Then, we can combine the coefficients, α^x and α^v indexed by h (for same facility), and rewrite the cut (3.58):

$$\sum_{j\in J} \alpha^t_j t_j + \sum_{j\in J} \alpha^{v_c}_j \sum_{h\in H_j} v_h + \sum_{j\in J} \sum_{(k,i)\in A} \sum_{h\in H_j} \alpha^{x_c}_{kij} x_{kih} \geq \beta^c \tag{3.60}$$

$$\text{such that} \quad \alpha^{v_c}_j = \sum_{h\in H_j} \alpha^v_h, \tag{3.61}$$

$$\alpha^{x_c}_{ikj} = \sum_{h\in H_j} \alpha^x_{ikh}, \tag{3.62}$$

$$\beta^c = |I|\beta,$$

where $|H_j| = |I|$ for all $j \in J$. Then, we can project the cut to the RMP space:

$$\sum_{j\in J} \alpha^t_j t_j + \sum_{j\in J} \alpha^{v_c}_j w_j + \sum_{j\in J} \sum_{(i,k)\in A} \alpha^{x_c}_{ikj} \sum_{p\in P_j} l_{ikp} z_p \geq \beta^c \tag{3.63}$$

As a summary, We solve the CGLP to find a cut in the form of (3.58) that violates the current fractional solution. Then we update the coefficients of (3.58) to get cut in the form of (3.60). Since the CGLP is solved for (3.58) and not for (3.60), the fractional solution $(\bar{t}, \bar{z})$ may not violate (3.63). Therefore, we designed implementation 4 to be able to overcome this problem.

3.4.4 Implementation 4

This implementation is designed to improve implementation 3. Similarly, step 1 of DISJCUTGEN chooses model M_3 and the CGLP is defined based on this model. However, we directly search for the most violated cut in the form of (3.60) instead of the form (3.58). To construct a cut of the form (3.60), we force constraints (3.59) in the CGLP, i.e., α coefficients for flow variables (for y) are set to zero. In addition, to be able to obtain the combined (aggregated for all vehicles located at the same facility) coefficients for x and v, we set α coefficients for variables x_{ikh} and v_h to be equal for all $h \in H_j$ for $j \in J$.

$$\alpha_j^{v_c} = \alpha_h^v, \quad \forall h \in H_j, \forall j \in J, \tag{3.64}$$

$$\alpha_{ikj}^{x_c} = \alpha_{ikh}^x \quad \forall h \in H_j, \forall j \in J. \tag{3.65}$$

As in implementation 3, we use one of the disjunctions (3.53), (3.54) and (3.57) to obtain a pair of disjunctive inequalities.

3.5 Cutting Planes Based on Arc Based Formulations

Another approach to derive problem specific cutting planes for LRSP is to study the LRSP polytope or its relaxations based on arc based formulations (like G-LRSP or similar relaxed models) that are defined based on variables that correspond to links between the nodes in the problem. When a cut is derived in terms of link variables, then this cut can be projected to SPP-LRSP by defining certain relationships between variables.

The literature does not include any studies devoted to polyhedral structure of LRSP. However, there are studies that investigate cuts and valid inequalities for VRP and LRP which are special cases of LRSP. In this section, we investigate some sets of cutting planes that are derived for VRP or LRP and originally defined in terms of variables corresponding to edges in the problem. These cutting planes are also valid for LRSP. Here, we discuss how we can incorporate these inequalities to SPP-LRSP and whether these inequalities have a positive impact to the model. In fact, we learned from the literature or showed that most of these inequalities are already implied by SPP-LRSP.

It is common to use two index vehicle flow formulations in the studies for VRP and LRP that investigate cutting planes. These formulations use binary variables, defined for each link, representing whether the link is used in the solution or not. Cutting planes based on link variables can easily

be modified for SPP-LRSP and may be used to extend the branch and price algorithm. Let x_{ik} be a link variable used in VRP or LRP formulations where x_{ik} equals to 1 if link (i, k) is used in the solution. When we have a cut in terms of x, we can use the relation,

$$x_{ik} = \sum_{j \in J} \sum_{p \in P_j} l_{ikp} z_p \quad \forall (i, k) \in A, \tag{3.66}$$

to project the cut into the RMP space where $l_{ikp} = 1$ if link (i, k) is in pairing p. In some graph based formulations, like G-LRSP, variables x_{ik} are replaced with three index variables that also include a vehicle index, such as x_{ikh}, where x_{ikh} is 1 if vehicle $h \in H$ travels link (i, k) in a solution to a three index formulation. We know the relation between a two index and a three index variable, which is $x_{ik} = \sum_{h \in H} x_{ikh}$.

Some cutting planes used in branch and cut algorithms for a VRP or LRP are implied by the set partitioning formulations of these problems. In the literature, there are limited number of studies investigating the theoretical relationship between a set partitioning formulation and a graph based formulations of the VRP, but there are none for the LRP. Letchford and González (2006) theoretically prove that a solution to the LP relaxation of a set partitioning formulation for the VRP (with elementary resource constrained paths) implies some of the valid inequalities (such as *knapsack large multistar, sub tour elimination, generalized large multistar, hypotour-like inequalities*) that are derived for a graph based formulation in the two index space. This result is also applicable to the LRSP as the LP relaxation of SPP-LRSP satisfies these type of cuts. Ropke and Cordeau (2007) do a similar study for the VRPTW with pickup and delivery; they investigate the relationship between a branch and cut algorithm and a branch, cut and price algorithm for the problem. They show that some of the valid inequalities for the problem (e.g. *fork inequalities, reachability inequalities, precedence inequalities*) are implied by the set partitioning formulation of the problem with a pricing problem that generates elementary resource constrained paths.

We started our study by investigating two of the most common classes of inequalities for the VRP: fractional capacity inequalities and rounded capacity inequalities.

Fractional Capacity Inequalities Letchford and González (2006) note that fractional capacity inequalities are dominated by generalized large multistar (GLM) inequalities and GML inequalities are included in knapsack large multistar inequalities, which are proved to be implied by the set partitioning formulation of the VRP. Therefore, Letchford and González (2006)'s results prove that fractional capacity inequalities are redundant for the set partitioning formulation. Here, we re-derive this result. Let S be a set of customers, then the fractional capacity inequality for S based on two index variables is as follows:

$$\sum_{i \in S} \sum_{k \in V \setminus S} x_{ik} \geq \frac{\sum_{i \in S} D_i}{L^V}. \tag{3.67}$$

In other words, total capacity of the vehicles leaving S must be greater than the total demand of customers in S. Using relation (3.66), we can rewrite inequality (3.67) in terms of z variables.

$$\sum_{j\in J}\sum_{p\in P_j}\sum_{i\in S}\sum_{k\in V\backslash S} l_{ikp}z_p = \sum_{j\in J}\sum_{p\in P_j}\alpha_p z_p \;\geq\; \frac{\sum_{i\in S} D_i}{L^V}, \tag{3.68}$$

where α_p is the coefficient that corresponds to z_p for all $p \in \cup_{j\in J} P_j$. α_p is the number of links that are from set S to set $V\backslash S$ and included in pairing p. Let I_p be the set of customers visited by pairing p, i.e., $I_p = \{i \in I | a_{ip} = 1\}$. For ease of notation, we define n_K as the minimum number of vehicle routes that are needed to cover customer demand in set K, in other words, $n_K = \lceil \frac{\sum_{i\in K} D_i}{L^V} \rceil$. Since a pairing represents a feasible vehicle schedule, by construction, the total number of links (in some p) leaving customers in set K that are visited by p ($K \subseteq I_p$) is at least n_K. The following lemma is stated based on this fact.

Lemma 3.1 *Let K be the intersection of set of customers visited by p with S in inequality (3.67), i.e.,$K = I_p \cap S$. If $K \neq \emptyset$, then the contribution (coefficient) of variable z_p to the left hand side of the inequality (3.68) is at least n_K. In other words, $\alpha_p \geq n_K$.*

Let $(t^*, z^*) \in \mathbb{R}^{+|J|+|\cup_{j\in J} P_j^*|}$ be a solution to the LP relaxation of SPP-LRSP where P_j^* is the current set of pairings for facility $j \in J$.

Proposition 3.1 *If a vector (t^*, z^*) satisfies (1.14) and (1.15) then it also satisfies (3.68).*

Proof We multiply constraint (1.14) for each customer i with its demand D_i for all $i \in S$. Then, we take the summation of the obtained constraints for all customers in set S:

$$\sum_{j\in J}\sum_{p\in P_j^*}\sum_{i\in S} D_i a_{ip} z_p^* \;=\; \sum_{i\in S} D_i, \tag{3.69}$$

$$\sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P_{Kj}^*} D(K) z_p^* \;=\; D(S), \tag{3.70}$$

where $D(K) = \sum_{i\in K} D_i$ and P_{Kj}^* is the set of pairings visiting all customers in set K and possibly some other customers in $I\backslash S$ for all nonempty subsets of S, i.e. $I_p \cap S = K$ for all $p \in P_{Kj}^*$. P_{Kj}^* is defined for all $K \subseteq S$ and for all $j \in J$. Note that $P_{K_1 j}^* \cap P_{K_2 j}^* = \emptyset$ for all $K_1, K_2 \subseteq S$, $K_1 \neq K_2$ and K_1, K_2 are nonempty. Since the pairings are feasible with respect to vehicle capacity from construction, $n_K L^V \geq D(K)$ for all $K \subseteq S$. Therefore,

$$\sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P_{Kj}^*} n_K L^V z_p^* \geq \sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P_{Kj}^*} D(K) z_p^* \;=\; D(S), \tag{3.71}$$

$$\sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P_{Kj}^*} n_K z_p^* \;\geq\; \frac{D(S)}{L^V} \tag{3.72}$$

From lemma 3.1:

$$\sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P^*_{Kj}} \alpha_p z^*_p \;\geq\; \sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P^*_{K}} n_K z^*_p \;\;\geq\;\; \frac{D(S)}{L^V}. \tag{3.73}$$

Therefore, inequality (3.68) is satisfied for any solution of the LP relaxation of the SPP-LRSP. □

Rounded Capacity Inequalities Another common class of inequalities is rounded capacity inequalities. These inequalities dominate the fractional capacity inequalities and they are in the following form:

$$\sum_{i\in S}\sum_{k\in V\setminus S} x_{ik} \;\geq\; n_S, \tag{3.74}$$

where $n_S = \lceil \frac{\sum_{i\in S} D_i}{L^V} \rceil$ for some customer set S. Neither Letchford and González (2006) nor Ropke and Cordeau (2007) has a result that says that the rounded capacity inequalities are implied by set partitioning formulation. Here, we investigate whether these inequalities can make an impact to the SPP-LRSP. Based on relation (3.66), if we rewrite (3.74) in terms of z variables, we obtain an inequality in the form of (3.68) with a right hand side that is equal to n_S.

Let $(t^*, z^*) \in \mathbb{R}^{+|J|+|\cup_{j\in J} P^*_j|}$ be a solution to the LP relaxation of SPP-LRSP where P^*_j is the current set of pairings for facility $j \in J$.

Lemma 3.2 *If $n_S = 1$ or $n_S = |S|$, then (t^*, z^*) satisfies inequality (3.74).*

Proof If $n_S = 1$, inequality (3.74) is equal to a subtour elimination inequality, which is implied by the set partitioning formulation (proved in Letchford and González (2006)).

If $n_S = |S|$, then no two customers in S can be visited in the same route. Thus, if Lemma 3.1 is applied, the coefficient of pairing p that visits customers in set $K \subseteq S$ in (3.74) is at least $n_K = |K|$. If we take the summation of constraints (1.14) for all customers in set S, the validity of inequality (3.74) follows:

$$\sum_{j\in J}\sum_{p\in P^*_j}\sum_{i\in S} a_{ip} z^*_p \;=\; |S|, \tag{3.75}$$

$$\sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P^*_{Kj}} \alpha_p z^*_p \geq \sum_{j\in J}\sum_{K\subseteq S}\sum_{p\in P^*_{Kj}} |K| z^*_p \;=\; |S|. \tag{3.76}$$

□

If the minimum number of vehicle routes that are needed to serve customers in S is greater than 1 and less than $|S|$, then the rounded capacity inequalities may be violated by some fractional LP solutions of SPP-LRSP. An example of such a solution for a small instance follows:

Example An instance with 4 customers and 2 facilities:

- Let $I = \{1, 2, 3, 4\}$, $J = \{\text{facility}_1, \text{facility}_2\}$.
- Demand is 20 for all customers. Vehicle capacity is 50, and facility capacity is 70.
- Let (t^*, z^*) be the LP relaxation solution value at root node for SPP-LRSP.
- Nonzero variable values and the pairings represented by pairing variables are:

$$\begin{aligned} t^*_{\text{facility}_1} = t^*_{\text{facility}_2} &= 1 \\ z^*_1 &= 1 \quad : \quad \text{facility}_1 - 4 - \text{facility}_1 \\ z^*_2 &= 0.5 \quad : \quad \text{facility}_2 - 3 - 1 - \text{facility}_2 \\ z^*_3 &= 0.5 \quad : \quad \text{facility}_2 - 1 - 2 - \text{facility}_2 \\ z^*_4 &= 0.5 \quad : \quad \text{facility}_2 - 3 - 2 - \text{facility}_2 \end{aligned}$$

- Rounded capacity inequality for $S = \{1, 2, 3\}$ is violated,

$$\sum_{i \in S} \sum_{k \in V \setminus S} x^*_{ik} = \sum_{j \in J} \sum_{p \in P_j} \sum_{i \in S} \sum_{k \in V \setminus S} l_{ikp} z^*_p = 1.5 < 2$$

At this point of the research, we have decided to leave the implementation of these inequalities in our branch and price algorithm as a future work.

Next, we investigate some cutting planes used for LRP, and discuss their validity for LRSP and their effect in formulation SPP-LRSP.

Chain Barring Inequalities Laporte et al. (1986) introduced chain barring constraints to eliminate paths from one depot to another in a solution of the LRP. Let $r = (j_1, i_1, i_2, i_3, .., i_k, j_2)$ be a path from node j_1 to j_2 where j_1 and j_2 are two depots and i_1, i_2, .., i_k are customer nodes. To eliminate path r, the following chain barring inequalities are defined:

$$x_{j_1 i_1} + 3x_{i_1 i_2} + x_{i_2 j_2} \leq 4 \qquad \text{if } k = 2 \tag{3.77}$$

$$x_{j_1 i_1} + 2\sum_{t=1}^{k-1} x_{i_t i_{t+1}} + x_{i_k j_2} \leq 2k - 1 \quad \text{if } k \geq 3 \tag{3.78}$$

We will prove that the LP relaxation of SPP-LRSP satisfies the following chain barring inequalities

that are obtained by rewriting (3.77) and (3.78) in terms of z variables.

$$\sum_{p\in P_{j_1}} l_{j_1 i_1 p} z_p + \sum_{j\in J}\sum_{p\in P_j} l_{i_1 i_2 p} z_p + \sum_{p\in P_{j_2}} l_{i_2 j_2 p} z_p \;\leq\; 4 \qquad \text{if } k=2 \tag{3.79}$$

$$\sum_{p\in P_{j_1}} l_{j_1 i_1 p} z_p + 2\sum_{j\in J}\sum_{p\in P_j}\sum_{t=1}^{k-1} l_{i_t i_{t+1} p} z_p + \sum_{p\in P_{j_2}} l_{i_k j_2 p} z_p \;\leq\; 2k-1 \quad \text{if } k\geq 3 \tag{3.80}$$

For the ease of notation, let α_p be the coefficient corresponds to variable z_p for all $p \in \cup_{j\in J} P_j$. Let (t^*, z^*) be a solution to the LP relaxation of SPP-LRSP and in this solution $r = (j_1, i_1, i_2, i_3, .., i_k, j_2)$ be a path from depot j_1 to j_2 including k customers.

Lemma 3.3 *If pairing $p \in \cup_{j\in J} P_j$ visits exactly n customers in path r, then it can visit at most n arcs in path r. Depending on the number of customers in the path, the contribution (coefficient) of z_p^* in the left hand side of inequalities (3.79) and (3.80) is as follows:*

i. If $k = 2$, the contribution of p in (3.79) is $3(n-1)+1$.

ii. If $k \geq 3$, the contribution of p in (3.80) is $2(n-1)+1 = 2n-1$.

Proof If p visits 1 customer in the path, then it can use at most one arc in r. If p only visits one of the customers in $\{i_2, .., i_{k-1}\}$, it cannot include any arc included in r. If p visits either i_1 or i_k, then it can include either the arc (j_1, i_1) or (i_k, j_2). The path cannot include both arcs since from the construction of p, p can only visit one of the facilities. Therefore, the contribution of variable z_p^* to both (3.79) and (3.80) can be at most 1 because of either (j_1, i_1) or (i_k, j_2).

If p visits n ($n > 1$) customers included in r, then it can use the largest number of arcs in r only if it visits these customers consecutively by using $n-1$ arcs. And if p visits the first (i_1) or last customer (i_k) in the path, then it can also include either the arc (j_1, i_1) or (i_k, j_2). Therefore, p can use at most n arcs in r. The contribution of p to (3.79) and (3.80) follows from the coefficients in the inequalities and the fact that at most $n-1$ of n arcs are between two customers. □

Proposition 3.2 *If a vector (t^*, z^*) satisfies (1.14) and (1.15) then it also satisfies the chain barring inequalities (3.79) and (3.80).*

Proof Let $r = (j_1, i_1, i_2, i_3, .., i_k, j_2)$ be a path from depot j_1 to j_2 including customers $i_1, i_2, .., i_k$ and assume that z^* violates either (3.79) or (3.80). Let P_{nj}^* be the set of pairings visiting exactly n of the customers in r for facility j, for all $j \in J$ and $n = 1, .., k$.

We take the summation of constraints (1.14) for customers in set $S = \{i_1, i_2, .., i_k\}$:

$$\sum_{j \in J} \sum_{p \in P_j^*} \sum_{i \in S} a_{ip} z_p^* \; = \; k, \tag{3.81}$$

$$\sum_{j \in J} \sum_{n=1}^{k} \sum_{p \in P_{nj}^*} n z_p^* \; = \; k. \tag{3.82}$$

For $k = 2$, using lemma 3.3 part (i):

$$\begin{aligned} \sum_{j \in J} \sum_{p \in P_j^*} \alpha_p z_p \; &\leq \; \sum_{j \in J} \sum_{n=1}^{2} (3n-2) \sum_{p \in P_{nj}^*} z_p^* = \sum_{j \in J} \sum_{p \in P_{1j}^*} z_p^* + 4 \sum_{j \in J} \sum_{p \in P_{2j}^*} z_p^* \\ &\leq \; 2(\sum_{j \in J} \sum_{p \in P_{1j}^*} z_p^* + 2 \sum_{j \in J} \sum_{p \in P_{2j}^*} z_p^*) = 4 \end{aligned} \tag{3.83}$$

The equality in (3.83) follows from the equality (3.82) for $k = 2$. Therefore, (3.79) cannot be violated for any fractional solution.
For $k \geq 3$, using lemma 3.3 part (ii):

$$\begin{aligned} \sum_{j \in J} \sum_{p \in P_j^*} \alpha_p z_p \; &\leq \; \sum_{n=1}^{k} (2n-1) \sum_{j \in J} \sum_{p \in P_{nj}^*} z_p^* \\ &\leq \; \sum_{n=1}^{k} (n \frac{(2k-1)}{k}) \sum_{j \in J} \sum_{p \in P_{nj}^*} z_p^* \; = \; 2k-1 \end{aligned} \tag{3.84}$$

To second inequality in (3.84) follows from the fact that $(2n-1) \leq n\frac{(2k-1)}{k}$ for all $n = 1..k$. To obtain the equality, multiply (3.82) with $(2k-1)/k$. The right hand side of (3.82) after the multiplication with $(2k-1)/k$ is equal to $2k-1$. Therefore, (3.80) can not be violated for any fractional solution of SPP-LRSP. □

Facility Capacity Inequalities Belenguer et al. (2006) define the following inequalities to satisfy facility capacity constraints:

$$\sum_{i \in S \cup \{j\}} \sum_{k \in V \setminus (S \cup \{j\})} x_{ik} \; \geq \; 1 \quad \forall j \in J, \forall S \subseteq I | \sum_{i \in I} D_i) > L_j^F \tag{3.85}$$

ZE: I do not think this is implied by AUG-SPP-LRSP. But, also I do not think, they would be helpful. Since there are multiple customers visited by more than one facility in a fractional solution. This forces total flow (in the left hand side of inequality) to be $\geq$ 1. So either, I should comment on that, or remove this inequality from the section.

ZE: Is there any other possible contribution to this chapter? Do you have any suggestions?

Chapter 4

Computational Experiments

4.1 Introduction

In this chapter, we present computational experiments to empirically validate the results obtained in previous three chapters and demonstrate the performance of the algorithms used to solve LRSP. In this chapter, first we describe the details of the instances used in our experiments. Second, we present computational results related to Chapter 1. In this context, we compare the LP relaxation bounds for the LRSP formulations and validate the effect of valid inequalities presented in the first chapter. Third, we get together a set of experiments to demonstrate the evolution of the branch and price algorithms for LRSP described in the second chapter. This section evaluates the components of the branch and price algorithms as well as the performance of the whole algorithms. Finally, the last section displays the effect of extending the algorithm to a branch price and cut algorithm using the cutting planes described in the third chapter. We investigate each implemented cutting plane.

As noted in Chapter 3, the branch, price and cut algorithms were coded in MINTO with CPLEX 9.1 as an LP solver. Unless otherwise stated for the experiments, we implemented the experiments in MINTO. We ran our experiments on a 64-node Linux-based Beowulf cluster using condor.

4.2 Description of the Instances

The only LRSP instances available in the literature are those of Lin et al. (2002). We could get only four small instances from Lin et al. (2002). We describe those instances and present our results for those in Section 4.4.6. Since they are very limited and small in size, for testing, we generated several sets of new instances.

Location and Demand Data We generated four sets of base instances from the MDVRP instances of Cordeau et al. (1997): instances with 20, 25, 30 and 40 customers. Each instance contains a subset of the customer locations and demand data from one of the Cordeau et al. (1997) instances. In particular, we selected instances $p01$ (50 customers), $p03$ (75 customers), $p07$ (100 customers) and $p11$ (249 customers) from which to generate our base sets of instances (these reference instances differ from each other in the locations of the customers and the candidate facilities).

Table 4.1 presents the set of customers used to construct LRSP instances in each group of instances (20, 25, 30 and 40 customer instances). Table 4.1 specifies the marker letter used to label our instances, the corresponding Cordeau et al. (1997) instance (denoted as *Ref*) and the indices of customers included from the reference instance. The details of labeling is explained later in this Section. In addition to the customer indices, Table 4.2 lists the coordinates of the five facility locations in each instance group differentiated with each reference instance.

Distance Matrix Rounded Euclidean distances between each pair of nodes were used in the algorithm, and we ensured that the distance matrices satisfied the triangle inequality.

Table 4.1: Location & Demand Data for LRSP instances generated from Cordeau et al. (1997)

Marker Letter	Ref	Indices of the Coordinates from Reference			
		20-cust.	25-cust.	30-cust.	40-cust.
f	p01	1 - 20	1 - 25	1-30	1-40
l	p01	31 - 50	26 - 50	-	11-50
f	p03	1 - 20	1 - 25	1-30	1-40
m	p03	28 - 47	26 - 50	-	16-35, 41-60
l	p03	56 - 75	51 - 75	46-75	36-75
f	p07	1 - 20	1 - 25	1-30	1-40
s	p07	21 - 40	26 - 50	-	31-70
t	p07	31 - 50	51 - 75	-	1-20, 51-70
f	p11	-	-	1-30	-
l	p11	-	-	101-130	-

Table 4.2: Facility Locations for LRSP instances

Instance Group	Ref	Facility Coordinates (X,Y)
20- and 25-customer	p01	(20,20), (30,40), (15,40), (45,40) (55,50)
	p03	(40,40), (50,22),(25,45), (20,20), (55,55)
	p07	(15,35), (55,35),(35,20), (35,50), (25,45)
30-customer	p01	(25,20), (40,40),(50,30), (55,55), (15,45)
	p03	(20,20), (40,40),(50,22), (55,55), (25,45)
	p07	(15,35), (55,35),(35,20), (35,50), (25,45)
	p11-f	(70,0), (40,80),(40,-80), (-60,20), (-60,-20)
	p11-l	(70,0), (40,80),(20,-30), (-60,20), (-60,-20)
40-customer	p01	(20,20), (30,40), (15,40), (50, 30), (55,55)
	p03	(40,40), (50,22), (25,45), (20,20), (55,55)
	p07	(15,35), (55,35),(35,20), (35,50), (25,45)

Capacity and Cost Parameters Based on the demand information, we assumed that the capacity of all facilities was 250, which requires at least two facilities to be open for each instance.

For each instance, we considered two time limit values and two vehicle capacity values. Time limits of 7 and 8 hours (which can be converted to distance limits of 140 and 160 miles under the assumption that vehicles travel 20 miles per hour) were used. In all instances, 140 is more than twice the distance between each customer and its farthest candidate facility. To be able to satisfy this condition, for instances generated from $p11$, we used distance limits of 250 and 270 instead of 140 and 160. We used "t1" or "t2" to mark the instances based on distance limit; "t1" was used for instances with distance limit 140 (or 240) and "t2" was used for instances with distance limit 160 (or 260).

We chose two vehicle capacity values. To determine these values, we used the fact that at least two facilities are required to be open (due to the capacity) and used the following equation similar

to Laporte et al. (1986):

$$B = (1-\alpha)\max_{i\in I}\{D_i\} + \alpha\frac{\sum_{i\in I} D_i}{2}, \tag{4.1}$$

with α equal to 0.1 and 0.2. Small α values were considered since a schedule employing multiple trips for a single vehicle is only realistic if the vehicle capacity is small compared to the demands of the customers. We used the 25 customer instances to determine vehicle capacities and used the same values in the 20, 30 and 40 customer instances. For each α value, we took the average of the calculated B values for the instances generated from the same Cordeau et al. (1997) instance, divided it by 10, rounded it up, and multiplied by 10. We obtained vehicle capacity values 50 (for $\alpha = 0.1$) and 70 (for $\alpha = 0.2$) for instances generated from $p01$ and $p07$, 60 (for $\alpha = 0.1$) and 80 (for $\alpha = 0.2$) for the instances generated from $p03$, and 150 and 200 for the instances generated from $p11$. The smaller vehicle capacity values can cover at least one customer plus the customer that has the largest demand in the instance. We used "v1" or "v2" to mark the instances based on vehicle capacity; "v1" was used to represent smaller capacity value and "v2" was used for larger capacity value.

Based on the cost values reported by Burlett (2002) and Kenworth Truck Company (2003), fixed costs for the facilities were generated using a uniform distribution on the interval [1500,2300], the fixed cost and the operating cost of a mid-size vehicle were estimated to be \$225 per day and \$1 per 1 mile traveled, respectively. The same cost parameters were used in all instances.

Table 4.3 summarizes the capacity and cost parameters. In the Table, as in Chapter 1, L^V and L^T represent the vehicle capacity and distance limit, respectively. C_j^F denotes the fixed cost of the facility j for all $j \in J$ while C^V and C^O correspond the vehicle fixed and operating costs, respectively.

Table 4.3: Summary of Parameters

Generated Using	L_1^V	L_2^V	L_1^T	L_2^T	Fac. cap.
p01,p07	50	70	140	160	250
p03	60	80	140	160	250
p11	150	200	240	260	250

C_1^F	C_2^F	C_3^F	C_4^F	C_5^F	C^V	C^O
\$1615 /day	\$1845 /day	\$1781 /day	\$1975 /day	\$1753 /day	\$225 /day	\$ 1 /mile

Labels of the LRSP Instances We label our instances considering reference Cordeau et al. (1997) instances, set of customers included from the corresponding reference (the marker letters listed in Table 4.1 used to indicate this information), the number of customers, vehicle capacity (one of the markers v1 or v2) and distance limit values (one of t1 or t2). For example, we used label "p01-f25-v1t1" for the LRSP instance (in 25-customer instance group) generated using customers 1 to

25 (based on letter "f") in $p01$. The "v1" and "t1" markers inform that the smaller vehicle capacity and distance limit values (50 and 140, respectively, based on Table 4.3) are used in the instance. Another example instance label is "p03-m40-v1t2". This label denotes that the instances is one of the 40-customer instances and is generated from reference $p03$ using customers 16 to 35, 41 to 60. Vehicle capacity and distance limits are 60 and 160, respectively.

4.3 Computational Experiments For Chapter 1

The goal of the computational experiments in this section is two-fold: to compare the LP relaxation bounds of G-LRSP and SPP-LRSP and to empirically demonstrate the benefit of the valid inequalities described in Section 1.5.

4.3.1 Comparing the LP Relaxations of SPP-LRSP and G-LRSP

In Section 1.6, it is shown that the Dantzig Wolfe decomposition of graph-based model G-LRSP is equal to the set partitioning based model, SPP-LRSP. In Section 1.7 we give the theoretical comparison between the LP relaxation bounds of G-LRSP and SPP-LRSP. In this section, we validate this result and empirically demonstrate the strength of the LP relaxation of SPP-LRSP compared to G-LRSP.

We used instances with 25 customers and 30 customers in this experiment. The LP relaxation of G-LRSP was solved using CPLEX 9.1 and AMPL interface. The number of vehicles at each facility is set to number of customers. This increases the number of variables, problem symmetry and the LP solution times significantly. Because of this reason, the LP relaxation values for some instances could not be obtained within the time limit. The LP relaxation of SPP-LRSP (model (1.13) through (1.17)) was obtained by using column generation algorithm described in Chapter 2. The algorithm was coded in MINTO with CPLEX 9.1 as an LP solver. Since in this experiment, we used an earlier version of our column generation algorithm, we could not get LP relaxation values for some of the instances within the time limit. All instances in both experiment groups were run with a time limit of 8 CPU hours.

The solution information for 25- and 30-customer instances are provided in Tables 4.4 and 4.5. Tables report LP relaxation values and solution times for G-LRSP and SPP-LRSP for each instance. Based on Table 4.4 and Table 4.5, for all solved instances, the LP relaxations obtained from model SPP-LRSP are %4.89 to %12.82 higher than the LP relaxations obtained from model G-LRSP. In addition, the computation times for the SPP-LRSP are much better than the times for G-LRSP in all solved instances. Furthermore, only 2 instances cannot be solved within time limit using model SPP-LRSP, while the number of unsolved instances is 8 if model G-LRSP is used.

Table 4.4: LP Relaxation Bounds of G-LRSP and SPP-LRSP: 25 Customer Instances

	SPP-LRSP		G-LRSP		(a-b)/b*100
Data File	LP (a)	CPU (s)	LP (b)	CPU (s)	% Gap
p01-f25-v1t1	3936.58	114.66	3705.06	5058.9	6.25
p01-f25-v1t2	3855.91	346.18	3641.57	4719.67	5.89
p01-f25-v2t1	3793.43	672.07	3610.19	4896.98	5.08
p01-f25-v2t2	3724.91	4240.58	3551.19	5046.43	4.89
p03-f25-v1t1	4620.03	26.18	4267.66	8969.62	8.26
p03-f25-v1t2	4538.07	256.77	4195.35	8729.64	8.17
p03-f25-v2t1	4443.34	121.03	4179.83	10256.6	6.3
p03-f25-v2t2	4351.62	791.77	4113.72	10934.8	5.78
p03-l25-v1t1	3928.73	34.83	3599.43	10906.3	9.15
p03-l25-v1t2	3829.09	112.31	3527.39	10216.5	8.55
p03-l25-v2t1	3812	78.6	3514.9	8722.87	8.45
p03-l25-v2t2	3713.14	355.46	3445.92	8835.58	7.75
p07-f25-v1t1	3413.54	63.77	3151.23	15809.4	8.32
p07-f25-v1t2	3341.53	394.39	3085.95	12698.5	8.28
p07-f25-v2t1	3301.11	198.84	3069.45	13462.4	7.55
p07-f25-v2t2	3198.43	677.1	3007.23	11832.1	6.36
p11-f25-v1t1	6615.99	0.42	5934.5	13097.1	11.48
p11-f25-v1t2	6536.44	0.72	5865.22	14250.6	11.44
p11-f25-v2t1	6378.77	0.73	5712.4	14334.6	11.67
p11-f25-v2t2	6261.63	1.19	5650.04	14252.1	10.82
p11-l25-v1t1	7298.5	0.26	6469.19	9429.75	12.82
p11-l25-v1t2	7187.6	0.33	6402.79	9334.2	12.26
p11-l25-v2t1	6939.53	0.36	6188.64	11521.1	12.13
p11-l25-v2t2	6904.43	0.44	6131.83	11561.7	12.6

4.3.2 Comparing Alternative Set Partitioning Formulations

In this section, we compare the strength of the formulations constructed with the valid inequalities described in Section 1.5. We enumerated all of the feasible pairings and solved the instances using a standard branch-and-bound algorithm without column generation. Due to the large number of feasible pairings, we had to conduct our experiments with smaller instances containing only 20 customers.

Let SPP denote the base SPP-LRSP formulation that contains only constraints (1.14) and (1.15). We constructed SPP_1 by adding constraints (1.24), SPP_2 by adding constraints (1.25), SPP_3 by adding constraint (1.18), and SPP_4 by adding both constraints (1.18) and (1.25). In Table 4.6, for each instance, we report the total number of feasible pairings, the integer optimal objective value, and the percentage gap of the LP relaxation associated with each of the formulations. In the table, a $-$ indicates that the branch-and-bound algorithm could not solve the relaxation within 6 hours of CPU time. For one instance, the set of feasible pairings could not be generated, so we

Table 4.5: LP Relaxation Bounds of G-LRSP and SPP-LRSP: 30 Customer Instances

	SPP-LRSP		G-LRSP		(a-b)/b*100
Data File	LP (a)	CPU (s)	LP (b)	CPU (s)	% Gap
p01-f30-v1t1	4640.47	643.4	4287.79	17337.3	8.23
p01-f30-v1t2	4561.62	8721.2	4219.16	17751.3	8.12
p01-f30-v2t1	4462.66	4464.09	4169.85	21347.8	7.02
p01-f30-v2t2	-	29209.74	4106.6	18790	-
p03-f30-v1t1	5413.72	370.4	-	28829.6	-
p03-f30-v1t2	5330.33	6625.36	-	28827.6	-
p03-f30-v2t1	5209.41	2953.18	-	28835	-
p03-f30-v2t2	-	29356.69	-	28834.2	-
p03-l30-v1t1	4707.72	131.18	4317.71	20964.7	9.03
p03-l30-v1t2	4615.22	2806.27	4240.26	19245.8	8.84
p03-l30-v2t1	4566.1	521.17	4221.57	19880.2	8.16
p03-l30-v2t2	4459.03	8129.62	4148.77	19549.6	7.48
p07-f30-v1t1	4153.16	705.12	3858.62	20347.3	7.63
p07-f30-v1t2	4077.68	2625.43	3782.79	20362.7	7.8
p07-f30-v2t1	3986.37	1870.87	3746.43	22084.3	6.4
p07-f30-v2t2	3899.99	15385.17	3674.65	22243.4	6.13
p11-f30-v1t1	7232.02	0.59	6483.4	17474.6	11.55
p11-f30-v1t2	7124.96	0.9	6410.9	18558.7	11.14
p11-f30-v2t1	6965.07	1.22	6238.88	22240	11.64
p11-f30-v2t2	6875.87	1.81	6172.69	23976.6	11.39
p11-l30-v1t1	8265.62	0.63	-	28843	-
p11-l30-v1t2	8161.91	1	-	28823.6	-
p11-l30-v2t1	8006.08	1.09	-	28817.9	-
p11-l30-v2t2	7798.55	2.53	-	28816	-

- Time limit is exceeded before LP solution is found.

did not use it in the experiments.

We compared SPP with SPP_1 and SPP_2 to evaluate the strength of constraints (1.24) and (1.25). Notice that adding inequalities (1.24) in SPP_1 made the problem difficult to solve and yielded at most only a small improvement in the LP relaxation bound for each solved instance. Adding inequalities (1.25) in SPP_2 improved the LP relaxation bound by a moderate amount.

Comparing SPP with SPP_3 and SPP_4 evaluated the impact of adding constraint (1.18) alone and in combination with constraints (1.25). In all instances, adding inequality (1.18) in SPP_3 lead to a dramatic reduction in the gap as compared to adding (1.25) alone. Together, the two inequalities (1.18) and (1.25) in SPP_4 further reduced the gap for all but 3 instances. The average gap for SPP_3 is 3.54 while the average gap for SPP_4 is 2.28. These results support the theoretical results in Section 1.5 and suggest that adding both inequalities (1.18) and (1.25) to the SPP-LRSP yields a stronger formulation, which we denote as AUG-SPP-LRSP and used in Chapter 2 for the solution algorithms.

Table 4.6: Comparison of LP relaxation bounds with constraints (1.24), (1.25) and (1.18)

Data	# of Pairing	IP Obj	SPP % Gap	SPP_1 % Gap	SPP_2 % Gap	SPP_3 % Gap	SPP_4 % Gap
p01-f20-v1t1	67558	4420	24.54	24.49	19.79	2.48	1.78
p01-f20-v1t2	154903	4419	26.06	-	22.08	3.69	3.52
p01-f20-v2t1	147368	4378	26.17	-	23.24	3.80	3.44
p01-f20-v2t2	349885	4369	27.52	-	25.05	4.86	4.83
p01-l20-v1t1	26266	4780	33.51	33.38	27.98	5.37	2.16
p01-l20-v1t2	60211	4599	32.77	32.72	28.08	3.49	0.58
p01-l20-v2t1	51538	4605	33.64	33.51	29.91	4.30	1.57
p01-l20-v2t2	121027	4481	33.24	33.23	30.81	2.97	1.54
p03-f20-v1t1	52619	4468	15.64	15.63	14.08	0.72	0.43
p03-f20-v1t2	121933	4461	17.25	-	16.12	2.32	2.13
p03-f20-v2t1	107172	4421	18.04	-	17.42	2.88	2.75
p03-f20-v2t2	254863	4421	19.78	-	19.12	4.58	4.44
p03-m20-v1t1	37751	4543	18.6	18.42	15.83	3.20	3.02
p03-m20-v1t2	89465	4442	18.13	-	16.02	2.35	2.35
p03-m20-v2t1	78580	4383	19.15	-	17.59	3.19	3.19
p03-m20-v2t2	193509	4376	20.13	-	19.2	4.12	4.12
p03-l20-v1t1	32801	4617	31.27	30.84	26	4.74	1.72
p03-l20-v1t2	79215	4467	30.19	30.13	27.08	2.77	1.30
p03-l20-v2t1	51052	4404	29.84	29.84	28.35	1.45	1.04
p03-l20-v2t2	133853	4399	31.76	-	30.45	3.55	3.11
p07-f20-v1t1	68267	4426	38.22	37.96	34.78	2.81	1.13
p07-f20-v1t2	166551	4421	39.33	-	37.18	3.60	2.57
p07-f20-v2t1	133873	4380	39.45	-	38.55	3.25	2.58
p07-s20-v1t1	27623	4840	31.69	31.47	25.75	4.73	1.01
p07-s20-v1t2	61753	4801	32.79	32.72	27.97	5.61	2.91
p07-s20-v2t1	54964	4751	32.8	32.48	29.09	5.34	2.32
p07-s20-v2t2	127679	4543	31.15	-	28.1	2.43	0.68
p07-t20-v1t1	11267	4758	29.49	29.47	26.03	3.62	1.05
p07-t20-v1t2	28315	4670	30.44	30.06	28.58	3.77	2.08
p07-t20-v2t1	19283	4688	32.39	31.83	30.98	5.57	4.14
p07-t20-v2t2	53689	4460	30.47	30.28	29.2	2.11	1.33
Avg.			28.24	29.91	25.50	3.54	2.28

4.4 Computational Experiments For Chapter 2

The goal of the computational experiments in this section is to demonstrate the effect of the algorithms for the components of a branch and price algorithm (described in Chapter 2) and to reason the extensions done to the branch and price algorithms to improve the overall performance. We evaluate the different pricing algorithms used in column generation process, effect of updating column generation with heuristic pricing algorithms, the strength of the primal heuristics to update the upper

bound, the effect of the of the initial set of columns to the overall branch and price algorithm, **ZE: and the effect of branching rule selection procedure** . Finally, the performance of the branch and price algorithms is demonstrated for 25- and 40-customer instances.

4.4.1 Performance of Exact Pricing Algorithms: 1p-ESPPRC and 2p-ESPPRC

In this section, we validate the claimed performance changes with different exact pricing problem solution algorithms in Section 2.4 by empirically demonstrating the performance change.

We solved the LP relaxation of AUG-SPP-LRSP using column generation algorithm (referred as COLGEN) with one of the pricing algorithms: one phase pricing problem solution algorithm (referred as 1p-ESPPRC), one phase pricing problem solution algorithm with new domination rule (presented in Section (2.4.1)) (referred as 1p-ESPPRC-NwD), and two phase pricing problem solution algorithm (referred as 2p-ESPPRC). In the experiments, at each iteration of COLGEN, at most one column for each facility (one with most negative reduced cost) was added to the RMP. In addition, we tested the effect of adding multiple columns instead of at most one at each iteration of COLGEN. We added at most 40 columns with most negative reduced cost for each facility at each iteration of COLGEN with 2p-ESPPRC.

In Table 4.7, for each instance and for each design of COLGEN, we report the total CPU time (in seconds) to solve the LP relaxation of RMP (at root node) and average time spent at each COLGEN iteration (total time / number of column generation iterations). For 2p-ESPPRC and 2p-ESPPRC with 40 columns, we also report number of COLGEN iterations. In Table 4.7, we calculate the fraction of total time (for 1p-ESPPRC-NwD, 2p-ESPPRC and 2p-ESPPRC with 40 columns) with respect to total time for 1p-ESPPRC.

Results on Table 4.7 show that new domination rule significantly improves the performance of 1p-ESPPRC. On average, total COLGEN time for 1p-ESPPRC-NwD is %64 of the total time in 1p-ESPPRC. Average time per COLGEN iteration improves about %20 to %50 with the addition of new domination rule to 1p-ESPPRC.

As seen from Table 4.7, 2p-ESPPRC reduces the total time %92 on average. And average time per COLGEN iteration reduces significantly for every instance. Further improvement is obtained by adding 40 columns at each iteration in 2p-ESPPRC. Adding multiple columns at each iteration instead of one column reduces number of COLGEN iterations in every instance. On average, total time for 2p-ESPPRC with multiple columns is %49 better than the time with single column 2p-ESPPRC.

Based on these experiments, 2p-ESPPRC algorithm performs significantly better than 1p-ESPPRC and 1p-ESPPRC-NwD to solve exact pricing problem. In addition, benefit of adding multiple columns instead of single best column at each iteration empirically demonstrated with this experiment.

Table 4.7: Performance of 1p-ESPPRC, 1p-ESPPRC-NwD, 2p-ESSPRC

	1p-ESPPRC		1p-ESPPRC-NwD		
Data File	Tot.time (s) (a)	Time per iter	Tot.time (s) (b)	(b/a)	Time per iter
p01-f25-v2t2	13467.95	134.68	6515.66	0.48	69.32
p01-l25-v2t2	2127.41	31.75	1601.02	0.75	24.63
p03-f25-v2t2	3273.45	32.73	2054.31	0.63	23.08
p03-m25-v2t2	12680.8	162.57	6380.45	0.5	76.87
p03-l25-v2t2	1417.49	19.96	962.73	0.68	13.56
p07-f25-v2t2	2231.63	31.88	1664.52	0.75	24.48
p07-s25-v2t2	305.07	4.49	220.66	0.72	3.56
p07-t25-v2t2	4036.72	61.16	2492.33	0.62	39.56
Average	4942.57	59.9	2736.46	0.64	34.38

	2p-ESPPRC				2p-ESPPRC with 40 col.per iter.			
Data File	Tot.time (s) (c)	(c/a)	# of iter	Time per iter	Tot.time (s) (d)	(d/c)	# of iter	Time per iter
p01-f25-v2t2	515.59	0.04	94	5.49	143.92	0.28	52	2.77
p01-l25-v2t2	195.69	0.09	70	2.8	95.13	0.49	29	3.28
p03-f25-v2t2	142.52	0.04	81	1.76	70.43	0.49	35	2.01
p03-m25-v2t2	321.98	0.03	78	4.13	173.35	0.54	38	4.56
p03-l25-v2t2	114.12	0.08	75	1.52	76.26	0.67	33	2.31
p07-f25-v2t2	293.53	0.13	64	4.59	140.89	0.48	27	5.22
p07-s25-v2t2	51.44	0.17	59	0.87	27.7	0.54	31	0.89
p07-t25-v2t2	223.61	0.06	68	3.29	124.55	0.56	29	4.29
Average	232.31	0.08	73.63	3.05	106.53	0.51	34.25	3.17

4.4.2 Effect of Adding Heuristic Pricing Algorithms: ESPPRC-LL and ESPPRC-CS

In this section, we empirically demonstrate the performance improvement that can be obtained by addition of heuristic pricing algorithms to overall column generation algorithm. In Section 2.6, we describe two heuristic versions of pricing algorithm: ESPPRC with label limit (ESPPRC-LL) and ESPPRC for a subset of customers (ESPPRC-CS).

We applied branch and price algorithm using different versions of column generation algorithm to solve the instances. We designed four experiments. In the first experiment, at each node of the branch and price algorithm, we used only exact pricing, 2p-ESPPRC in COLGEN. In the second experiment, at each node, we ran heuristic pricing algorithm 2p-ESPPRC-LL with label limits 50, 100 and 200 before the exact pricing algorithm. At each node we solved 2p-ESPPRC-LL(50) until no column is found. Then we incremented the label limit to 100 and applied 2p-ESPPRC-LL(100).

When 2p-ESPPRC-LL(100) terminated with empty column list, we started applying 2p-ESPPRC-LL(200). Finally, we applied 2p-ESPPRC until the column generation terminates. In the third experiment, we applied 2p-ESPPRC-CS with subset size 12. 2p-ESPPRC-CS(12) is ran at most 12 iterations before 2p-ESPPRC was called. In the last experiment, we applied both heuristic pricing algorithms before exact pricing algorithm. We applied 2p-ESPPRC-CS(12), then 2p-ESPPRC-LL with label limits 50, 100 and 200, and finally 2p-ESPPRC.

In Table 4.8, for each instance and for each design of branch and price algorithm, we report average column generation time (total COLGEN time / # of evaluated nodes), # of evaluated nodes and total algorithm time (in CPU seconds). In Table 4.8, we calculate the fraction of total time in each version of the branch and price algorithm with respect to total time of the algorithm with only 2p-ESPPRC (without any heuristic pricing algorithms).

Results on Table 4.8 show that applying 2p-ESPPRC-LL before exact pricing algorithm improves the performance of the overall algorithm, i.e., total algorithm time improves %20 on average. In all instances, total solution time and average COLGEN time is same or lower in branch and price algorithm with 2p-ESPPRC-LL.

The algorithm with 2p-ESPPRC-CS improves the solution time only %2 on average. In half of the test instances, solution time with ESPPRC-CS is better than the algorithm with only exact pricing. The algorithm using heuristic pricing algorithms 2p-ESPPRC-CS and 2p-ESPPRC-LL in combination is about %17 better on average. This result supports that %20 effect of 2p-ESPPRC-LL in the second experiment. Therefore, we can conclude that in the fourth experiment the performance improvement was obtained by application of 2p-ESPPRC-LL, and 2p-ESPPRC-CS seems to have a negative effect to the performance. Even though the contribution of 2p-ESPPRC-CS seems negligible, the effect of heuristic pricing 2p-ESPPRC-CS is more significant with larger instances. We have implemented another set of experiments to show this effect. We describe it next.

To be able to see the effect of ESPPRC-CS in larger instances, we designed two more experiments using instances with 40 customers. In the first design, first, we solved heuristic branch and price algorithm (H-BP) described in Section 2.9.3. In H-BP, we applied 2p-ESPPRC-CS with subset size 15 for at most 20 iterations. Then we applied 2p-ESPPRC-LL with 5, 10 and 15 label limits. After running H-BP with a time limit of 2 CPU hours, we started exact branch and price (E-BP) using the upper bound and columns obtained by H-BP. This two step algorithm is referred as E-BP-2S in Section (2.9.4). At each node of E-BP, we applied 2p-ESPPRC-CS with subset size 15 for at most 15 iterations. After, algorithm 2p-ESPPRC-LL with 50, 100 and 200 label limits was applied and the evaluation of the node terminated with exact pricing 2p-ESPPRC. The second step, E-BP was run with a time limit of 6 CPU hours. To evaluate the contribution of ESPPRC-CS to the overall performance of the algorithm, in the second experiment, we applied same 2 step branch and price algorithm except in H-BP and E-BP we did not apply 2p-ESPPRC-CS. We refer the first branch and price algorithms in the first design including algorithm 2p-ESPPRC-CS as H-BP-W and E-BP-W, and the algorithms in the second design without 2p-ESPPRC-CS as H-BP-WO and E-BP-WO.

Table 4.8: Performance improvement with 2p-ESPPRC-LL and 2p-ESSPRC-CS

	BP with 2p-ESPPRC			BP with 2p-ESPPRC-LL + 2p-ESPPRC			
Data File	Avg. CG time (s)	# of BB Nd.	Tot.time (s) (a)	Avg. CG time (s)	# of BB Nd.	Tot.time (s) (b)	(b/a)
p01-f25-v2t2	10.86	151	1670.57	7.79	119	954.41	0.57
p01-l25-v2t2	14.25	37	533.13	10.48	39	414.51	0.78
p03-f25-v2t2	6.41	49	322.28	5.92	52	316.88	0.98
p03-m25-v2t2	92.85	9	838.71	57.89	9	523.75	0.62
p03-l25-v2t2	13.76	41	568.25	12.11	37	452.17	0.8
p07-f25-v2t2	8.84	135	1214.31	4.39	197	907.33	0.75
p07-s25-v2t2	2.05	102	217.42	2.1	100	217.97	1
p07-t25-v2t2	9.08	87	816.85	7.92	91	754.9	0.92
Average	19.76	76.38	772.69	13.58	80.5	567.74	0.8

	BP with 2p-ESPPRC-CS + 2p-ESPPRC				BP with 2p-ESPPRC-LL + 2p-ESPPRC-CS + 2p-ESPPRC			
Data File	Avg. CG time (s)	# of BB Nd.	Tot.time (s) (c)	(c/a)	Avg. CG time (s)	# of BB Nd.	Tot.time (s) (d)	(d/a)
p01-f25-v2t2	8.68	101	897.41	0.54	11.78	71	849.63	0.51
p01-l25-v2t2	14.63	37	546.99	1.03	12.12	39	478.4	0.9
p03-f25-v2t2	7.1	50	365.63	1.13	6.4	44	289.4	0.9
p03-m25-v2t2	63.71	9	576.48	0.69	49.05	9	444.39	0.53
p03-l25-v2t2	14.06	39	552.7	0.97	12.18	37	454.43	0.8
p07-f25-v2t2	4.71	219	1076.36	0.89	3.76	235	938.72	0.77
p07-s25-v2t2	2.3	94	223.8	1.03	2.13	97	214.26	0.99
p07-t25-v2t2	11.58	107	1277.6	1.56	8.01	117	991.46	1.21
Average	15.85	82	689.62	0.98	13.18	81.13	582.59	0.83

Table 4.9 shows the results of the two designs and presents comparisons.

In Table 4.9, for each design (2 step branch and price algorithm with and without 2p-ESPPRC-CS), we report upper bounds obtained at the end of H-BP-W and H-BP-WO, and the optimality gaps at the end of E-BP-W and E-BP-WO. Therefore, we can evaluate the effect of using 2p-ESPPRC-CS to both steps of the algorithm, H-BP and E-BP. In addition, last three columns in Table 4.9 present the ratio of the running times in the design with 2p-ESPPRC-CS to the times in the design without 2p-ESPPRC-CS (denoted as *W/WO*). We report fraction of times in each of H-BP, E-BP and total algorithm time (H-BP + E-BP).

Table 4.9, indicates that addition of 2p-ESSPRC-CS to the algorithm improves the performance of both H-BP and E-BP. In H-BP, in 14 out of 32 instances, the upper bounds obtained using H-BP-W (the algorithm with 2p-ESSPRC-CS) are strictly better while H-BP-WO provides better upper bounds in 3 out of 32 instances. In E-BP, the algorithm with 2p-ESSPRC-CS results strictly better

optimality gaps in 13 instances while this number is only 2 for the algorithm without 2p-ESSPRC-CS. In time comparison, even though the time spent in H-BP for the algorithm with 2p-ESSPRC-CS is 1.6 times greater than the corresponding time in the algorithm without 2p-ESSPRC-CS, in only 10 out of 32 instances time in H-BP for the algorithm with 2p-ESSPRC-CS is greater than the the algorithm without 2p-ESSPRC-CS. 2p-ESSPRC-CS results about %30 improvement in both the time in E-BP and total time. Therefore, we can conclude that the effect of heuristic pricing 2p-ESSPRC-CS for larger instances is more significant and using 2p-ESSPRC-CS boosts the overall performance of the algorithm, as well as the performance of H-BP and E-BP.

4.4.3 Performance of Primal Feasible Heuristics

As described in Chapter 2, in our study, we could employ three approaches to find primal feasible solutions for LRSP: construction base simple heuristic I-Heur (described in Section 2.9.1), a standard branch and bound using MINTO and heuristic branch and price algorithm H-BP (described in Section 2.9.3). In this Section, we empirically demonstrate the quality of the upper bound obtained using each heuristic approach. Branch and bound approach may be used at the root node of either heuristic branch and price (H-BP) or exact branch and price algorithm (E-BP). Here, in this experiment, we used branch and bound (refer as BB) at the root node of H-BP based on the RMP with the set of columns present at the end of root node. We set a time limit of 1 CPU hour for BB. In the experiment, H-BP is run with a time limit of 2 CPU hours and only heuristic pricing algorithm 2p-ESPPRC-LL with label limits of 5 and 10 is used in the column generation algorithm.

Table 4.10 reports upper bound values obtained from and time spent at each heuristic bounding procedure, I-Heur, BB and H-BP. Time for BB is the time to solve the IP model and does not include the time required to solve the root node of H-BP and generate the columns in the model to be used as input for BB algorithm. I-Heur is very fast. Time required for BB and H-BP depends on the size of the instance; they terminate very quickly in 25-customer instances, 40-customer instances require more time and may terminate with the time limit. Even though, BB and H-BP are not very quick for large instances, they improve the performance of the exact branch and price algorithm significantly by providing very good upper bounds for large instances.

Table 4.11 reports the % gaps between the known LP and optimal (or best) solutions of each instance and upper bounds obtained from each heuristic bounding approach. As seen from Table 4.11 I-Heur and BB provide upper bounds which are about %5 to %11 away from the LP relaxation bounds. BB bounds are slightly better than the I-Heur bounds. This difference is larger for 40 customer instances; BB provides bounds that are %4.8 away from the LP relaxation bound on average while the average gap is 6.7 in I-Heur. H-BP provides the strongest bound compared to I-Heur and BB. The average gap between H-BP upper bounds and the LP relaxation bounds is about 3.4 in all instances, and the average values are similar in both 25- and 40-customer instances. Consistent results are obtained for the gaps from the known optimal or best IP solutions. I-Heur

Table 4.9: Performance improvement with 2p-ESSPRC-CS in BP algorithm for 40 customers

	E-BP-2S Without ESPPRC-CS		E-BP-2S With ESPPRC-CS		Time Comparison		
Data File	IP H-BP	% Gap E-BP	IP H-BP	% Gap E-BP	H-BP W/WO	E-BP W/WO	Tot Time W/WO
p01-f40-v1t1	7131	0.01	7131	0	1.24	0.42	0.5
p01-f40-v1t2	6986	0.14	7170	0.32	1	1	1
p01-f40-v2t1	7017	0	6866	0	1.31	0.46	0.47
p01-f40-v2t2	6821	1.99	6821	1.26	0.29	0.97	0.81
p01-l40-v1t1	7252	4.14	7183	3.22	1	0.99	0.99
p01-l40-v1t2	6934	0	6934	0	0.64	0.49	0.5
p01-l40-v2t1	6823	0	6823	0	0.33	0.68	0.67
p01-l40-v2t2	6634	0	6633	0	1.26	0.58	0.6
p03-f40-v1t1	8780	0	8780	0	0.94	1.08	1.07
p03-f40-v1t2	8756	0	8753	0	1.66	0.66	0.73
p03-f40-v2t1	8663	0	8663	0	1.53	0.37	0.57
p03-f40-v2t2	8456	0	8438	0	2.91	0.55	0.59
p03-m40-v1t1	7793	0.23	7740	0	1	1	1
p03-m40-v1t2	7544	6.32	7236	2.95	0.7	0.97	0.9
p03-m40-v2t1	7140	2.29	7132	2.79	1	0.97	0.98
p03-m40-v2t2	7430	8.52	6947	4.18	0.31	0.97	0.81
p03-l40-v1t1	7167	0	7167	0	0.06	0.44	0.37
p03-l40-v1t2	6950	0.07	6950	0.05	1	1	1
p03-l40-v2t1	6857	0	6849	0	0.03	0.54	0.35
p03-l40-v2t2	6846	3.19	6846	0	1.09	0.81	0.83
p07-f40-v1t1	7167	0	7167	0	0.37	0.5	0.48
p07-f40-v1t2	7001	0	7001	0	0.29	0.55	0.49
p07-f40-v2t1	6925	0.01	6881	0	0.33	0.41	0.41
p07-f40-v2t2	6851	2.55	6845	2.34	1.41	1	1.01
p07-s40-v1t1	7343	0	7343	0	0.57	0.55	0.55
p07-s40-v1t2	7117	0	7124	0	0.16	0.61	0.38
p07-s40-v2t1	7000	0	7005	0	0.1	0.56	0.49
p07-s40-v2t2	6944	0	6944	0	0.43	0.64	0.63
p07-t40-v1t1	7012	0.01	7013	0	0.6	0.39	0.4
p07-t40-v1t2	6972	1.22	6972	0.35	2.43	0.99	1.1
p07-t40-v2t1	6869	3.01	6857	0.04	24.3	0.81	0.94
p07-t40-v2t2	6667	1.78	6658	1.64	0.22	1.01	0.81
Average					1.58	0.72	0.7

provides bounds that are about %4.3 away from the optimal solution on average. The effect of BB is more significant for 40-customer instances compared to 25-customer instances. H-BP is very strong in almost all instances; in 15 out of 16 instances, H-BP bound is either optimal or at most %1.5 away from the optimal.

These results suggest that all of the bounding procedures perform well and provide good upper bounds. Depending on the performance of the overall exact branch and price algorithm, one or more of these heuristics can be employed. If the exact branch and price algorithm terminates quickly, then using only I-Heur or BB may be enough.

Table 4.10: Performance of I-Heur, BB, H-BP

	Upper Bound Values			Time (s)		
Data file	I-Heur	BB in H-BP Root	H-BP	I-Heur	BB in H-BP Root	H-BP
p01-f25-v2t2	4722	4710	4425	0.04	21.15	264.96
p01-l25-v2t2	4762	4762	4650	0.04	0.48	25.32
p03-f25-v2t2	4831	4831	4742	0.04	11.38	135.94
p03-m25-v2t2	4767	4761	4548	0.03	6.87	80.35
p03-l25-v2t2	4805	4794	4734	0.03	1.56	45.28
p07-f25-v2t2	4721	4576	4483	0.03	0.09	71.32
p07-s25-v2t2	4934	4856	4781	0.04	3.03	57.88
p07-t25-v2t2	4769	4769	4654	0.03	1.71	40.63
Average				0.04	5.78	90.21
p01-f40-v1t1	7170	7170	7131	0.05	14.35	1452.48
p01-l40-v1t1	7449	7252	7252	0.07	46.31	7201.3
p03-f40-v1t1	9253	9009	8780	0.04	38.73	608.19
p03-m40-v1t1	7827	7793	7793	0.06	2589.2	7201.11
p03-l40-v1t1	7386	7386	7167	0.05	190.03	2222.19
p07-f40-v1t1	7424	7324	7167	0.04	63.21	680.46
p07-s40-v1t1	7860	7343	7343	0.05	0.28	75.01
p07-t40-v1t1	7283	7283	7012	0.05	51.52	825.27
Average				0.05	374.2	2533.25

Earlier in this section, we noted that BB approach may be used at the root node of either H-BP or E-BP. In our experiments in Tables 4.10 and 4.11, we used BB at the root node of H-BP. We also tested the performance of the E-BP algorithm in which BB is used at the root node to provide an upper bound. Table 4.12 presents the results comparing an E-BP with BB upper bounding procedure at root node with an E-BP without BB procedure. Table 4.12 reports the gap between the upper bound obtained using BB and the known optimal IP solution, number of evaluated branch and price nodes in both E-BP with BB and E-BP without BB, and the fraction of total algorithm time of E-BP with BB to the time of E-BP without BB. For both 25- and 40-customer instances, total algorithm time increases if BB is employed at root node. On average, algorithm time increases about %13. The number of evaluated nodes is decreased in only 5 out of 16 instances with the BB procedure. Even though the upper bound obtained using BB procedure is about %2.5 away from the optimal solution, using BB at the root node of E-BP does not improve, but worsen the overall algorithm performance. Therefore, based on these results, we have decided to use BB only at the root node of H-BP.

Table 4.11: I-Heur, BB, H-BP Upper Bounds vs. LP Relaxations and Optimal IPs

Data file	%Gap from LP Relax.			%Gap from Optimal (or Best IP)		
	I-Heur	BB in H-BP Root	H-BP	I-Heur	BB in H-BP Root	H-BP
p01-f25-v2t2	9.93	9.65	3.02	6.71	6.44	0
p01-l25-v2t2	4.96	4.96	2.49	3.48	3.48	1.04
p03-f25-v2t2	7.29	7.29	5.32	1.88	1.88	0
p03-m25-v2t2	8.44	8.3	3.45	6.43	6.3	1.54
p03-l25-v2t2	6.6	6.35	5.02	3.22	2.99	1.7
p07-f25-v2t2	7.58	4.28	2.16	5.33	2.1	0.02
p07-s25-v2t2	7.03	5.34	3.71	3.2	1.57	0
p07-t25-v2t2	5.67	5.67	3.12	3.85	3.85	1.35
Average	7.19	6.48	3.54	4.26	3.57	0.71
p01-f40-v1t1	2.78	2.78	2.22	0.55	0.55	0
p01-l40-v1t1	7.96	5.11	5.11	3.7	0.96	0.96
p03-f40-v1t1	6.39	3.58	0.95	5.39	2.61	0
p03-m40-v1t1	11.01	10.53	10.53	7.91	7.45	7.45
p03-l40-v1t1	6.73	6.73	3.56	3.06	3.06	0
p07-f40-v1t1	5.26	3.84	1.62	3.59	2.19	0
p07-s40-v1t1	8.32	1.19	1.19	7.04	0	0
p07-t40-v1t1	4.93	4.93	1.03	3.86	3.86	0
Average	6.67	4.84	3.28	4.39	2.58	1.05

Table 4.12: Performance of BB at Root Node of E-BP

Data File	E-BP, BB at Root		E-BP	Time
	BBIP-Opt Gap	Nds	Nds	BB/WOBB
p01-f25-v2t2	1.04	83	81	1.12
p01-l25-v2t2	3.48	37	37	1.23
p03-f25-v2t2	1.88	50	50	1.34
p03-m25-v2t2	5.4	9	9	1.25
p03-l25-v2t2	3.22	43	43	1.07
p07-f25-v2t2	0.98	205	251	0.98
p07-s25-v2t2	0	105	92	1.03
p07-t25-v2t2	2.03	157	149	1.02
Average	2.25			1.13
p01-f40-v1t1	0.52	431	415	1.2
p01-l40-v1t1	0.1	893	911	1
p03-f40-v1t1	2.72	65	79	1
p03-m40-v1t1	8.01	1199	1340	1
p03-l40-v1t1	0.6	193	196	1.05
p07-f40-v1t1	2.54	95	95	1.22
p07-s40-v1t1	3.19	73	73	1.49
p07-t40-v1t1	3.58	229	229	1.1
Average	2.66			1.13

4.4.4 Performance of Initializing Branch & Price Algorithm Using H-BP (E-BP vs E-BP-2S)

As explained in Section 2.9, the upper bounding procedures, I-Heur and H-BP, have also a role of providing a starting set of feasible columns to initialize the branch and price algorithm. When we solve exact branch and price algorithm, E-BP, we initialize it with the columns and upper bound from I-Heur. Since H-BP is itself also a branch and price algorithm, when we use H-BP to initialize exact branch and price algorithm, E-BP, we call the overall algorithm as 2-step branch and price algorithm E-BP-2S. The first step refers to H-BP, and the second step is the exact algorithm, E-BP. As seen from the results in Section 4.4.3, H-BP may take too much time and this may worsen the total algorithm time. Even though H-BP may be time consuming, it provides very strong upper bounds and spending this amount of time for upper bound and nice columns will be worthwhile for large instances especially for ones in which exact branch and price algorithm terminates with the time limit. Here, we provide more computational results demonstrating the effect of employing H-BP to initialize E-BP to the overall branch and price algorithm. In other words, we compare 1 step and 2 step algorithms, E-BP and E-BP-2S.

We designed two experiments. In the first experiment, we used 1 step branch and price algorithm, i.e., we used I-Heur to initialize the E-BP. In E-BP, at each node of the tree, we first employed heuristic pricing algorithm 2p-ESPPRC-LL with label limits 50, 100 and 200, and finally exact pricing algorithm 2p-ESPPRC. We set total time limit to 8 CPU hours. In the second experiment, we used 2 step branch and price algorithm, i.e., we solved H-BP first. In H-BP, we employed heuristic pricing algorithm 2p-ESPPRC-LL with label limits of 5 and 10. We set time limit to 2 CPU hours. Then, E-BP started with the columns and the upper bound from H-BP. Time limit was set to 6 CPU hours.

Table 4.13 and 4.14 present results of the designed experiments for 25-customer and 40-customer instances, respectively. Table 4.13 and 4.14 reports the number of evaluated nodes in both E-BP and E-BP-2S and the fraction of total algorithm time in E-BP to the E-BP-2S. In 25-customer instances, both algorithms terminated with optimal solutions. However, in some of 40-customer instances the algorithms terminate with optimality gap. Thus, Table 4.14 also reports % gap at the end of the algorithm. Table 4.13 indicates that on average, 2 step algorithm is about %18 faster than 1 step E-BP. Initializing the exact branch and price algorithm with the H-BP boosts the overall performance of the algorithm. There are also some instances for which E-BP performs better than E-BP-2S. For most of the 25-customer instances, computation times are very short. The average computation time for instances in Table 4.13 is about 340 CPU seconds in E-BP and about 295 seconds in E-BP-2S. Therefore, using 1 step exact branch and price algorithm, E-BP for 25-customer instances is not a bad choice. We may prefer using 1 step algorithm for this size of instances.

Table 4.14 reports that on average E-BP time is %8 better than E-BP-2S time. However, Table 4.14 shows that only 6 out of 16 instances E-BP time is strictly better than E-BP-2S while E-BP-2S

is strictly better in 7 instances. In addition, when we compare the optimality gap columns for two designs, we can see that E-BP-2S is more successful in closing the gaps. The strong upper bound obtained in H-BP has a significant role in the termination with better gaps. Therefore, for large instances, since the algorithm terminates with the time limit, for the sake of good optimality gaps, we prefer using E-BP-2S.

Table 4.13: E-BP vs E-BP-2S: Effect of H-BP, 25-Customer Instances

	Nds		CPU Time (s)		Time
Data File	E-BP	E-BP-2S	E-BP	E-BP-2S	E-BP/E-BP-2S
p01-f25-v1t1	25	25	77.81	64.26	1.21
p01-f25-v2t2	119	115	954.41	665.3	1.43
p01-l25-v1t1	24	23	35.45	29.14	1.22
p01-l25-v2t2	39	35	414.51	345.96	1.2
p03-f25-v1t1	236	231	182.13	227.96	0.8
p03-f25-v2t2	52	43	316.88	233.05	1.36
p03-m25-v1t1	509	483	314.12	431.17	0.73
p03-m25-v2t2	9	9	523.75	284.47	1.84
p03-l25-v1t1	17	15	36.21	29.6	1.22
p03-l25-v2t2	37	45	452.17	453.74	1
p07-f25-v1t1	55	37	81.28	63.86	1.27
p07-f25-v2t2	197	135	907.33	619.14	1.47
p07-s25-v1t1	599	197	87.77	52.2	1.68
p07-s25-v2t2	100	93	217.97	179.56	1.21
p07-t25-v1t1	35	47	51.8	44.94	1.15
p07-t25-v2t2	91	211	754.9	997.57	0.76
average			338.03	295.12	1.22

4.4.5 Results of Branch & Price Algorithm for Generated Instances

In this section, we present solution and performance details of the exact branch and price algorithms for 25- and 40-customer instances. Based on the results of our previous experiments, we came up with ideal designs of branch and price algorithms to solve the generated LRSP instances. The objective of the algorithms is to provide optimal or best solutions, and optimality gaps in case of not proving optimality for the LRSP instances. Here, we present computational results for all 64 instances in 25- and 40-customer instance sets.

25-Customer Instances Our previous computational experiments showed that our 1 step exact branch and price algorithm, E-BP is as successful as our 2 step exact branch and price algorithm, E-BP-2S for medium-size instances. Therefore, for 25-customer instances, we may either use E-BP or E-BP-2S. Here, we used E-BP which only employs I-Heur to initialize.

Table 4.14: E-BP vs E-BP-2S: Effect of H-BP

Data File	Gap %		Nds		Time
	E-BP	E-BP-2S	E-BP	E-BP-2S	E-BP/E-BP-2S
p01-f40-v1t1	0	0.01	415	909	0.19
p01-f40-v2t2	4.28	1.99	15	16	1
p01-l40-v1t1	6.9	4.14	911	324	0.99
p01-l40-v2t2	0	0	25	25	1.01
p03-f40-v1t1	0	0	79	65	0.52
p03-f40-v2t2	0	0	11	11	1.19
p03-m40-v1t1	0.1	0.23	1340	1021	1
p03-m40-v2t2	8.07	8.52	32	24	1
p03-l40-v1t1	0	0	196	197	0.86
p03-l40-v2t2	5.53	3.19	20	22	1.22
p07-f40-v1t1	0	0	95	101	0.86
p07-f40-v2t2	4.55	2.55	34	19	1.28
p07-s40-v1t1	0	0	73	65	1.14
p07-s40-v2t2	0	0	213	173	1.12
p07-t40-v1t1	0	0.01	229	220	0.25
p07-t40-v2t2	5.71	1.78	6	5	1.04
Average					0.92

Based on the performance results from our previous experiments, we designed the following exact branch and price algorithm, E-BP for 25-customer instances. The main components and input parameters of E-BP are as follows:

- Run I-Heur to find initial columns and an initial upper bound.
- At each node node of the tree, column generation algorithm uses the following pricing algorithms:
  - Run 2p-ESPPRC-LL with label limit 50, 100 and 200. The main implementation steps are described in Section 2.9.4. **ZE: may move the implementation details from ch2 to here.**
  - Run 2p-ESPPRC.
- Set time limit of 6 CPU hours.

Table 4.15 shows the results for instances with 25 customers. For each instance, Table 4.15 reports the upper bound obtained from I-Heur (I UB), the LP relaxation value at the root node (LP), the optimal objective value (IP), the integrality gap at the end of the running time (Gap %), the number of evaluated nodes (Nds), and the total CPU time in minutes (CPU). The last three columns present some details about the optimal solution: the indices of open facilities (Loc), the number of vehicles used at each open facility (# Veh), and the total number of routes in each solution (# Rt).

Overall, the algorithm handled the 25-customer instances well; 29 of the 32 instances were solved to optimality in less than 25 minutes with the majority (24) solved in 10 minutes or less. For only one instance did the algorithm terminate due to the time limit before proving optimality. The initial feasible solutions provided by I-Heur were good for the 25-customer instances with an average deviation of 4.4% from optimality which is consistent with the results from Section 4.4.3. The LP relaxation bounds were quite strong, as expected with the addition of valid inequalities at the root node. The average deviation of the LP relaxation bound from the optimality was 2.6%, which is in line with the results of the smaller instances (instances with 20 customers in Section 4.3.2).

In addition to the solution summary given in the last three columns of Table 4.15, we listed the constructed vehicle routes and schedules (i.e., selected pairings) in the optimal solutions for all instances in Appendix A.1. In all instances, the number of facilities selected to be used is 2. In addition, in Table 4.16, we present some summary statistics about the solutions of all 32 instances, such as the number of vehicles used, the average number of routes per vehicle, the average number of customers per route and per vehicle, and the maximum number of customers per vehicle. In all instances, total number of vehicles used changes between 3 to 5 while number of vehicles per facility changes between 1 to 3. Again, based on the table, we can see that each vehicle's schedule covers at least 2 routes (2.1 routes per vehicle on average) and some vehicles cover more than 3 routes (average # of routes is between 1.5 to 3.3), which may include 11 customers at most. These statistics give us an intuition about the difficulty of the instances.

As can be understood from the labels of the instances, the computational tests can also measure the sensitivity of the algorithm to vehicle capacity and time limit. As the labels of the instances indicate that there are 16 pairs of instances which differ only in vehicle capacity (instances with label x-v1t1 and x-v2t1) and 16 pairs which differ only in distance limits (the instances with label x-v1t1 and x-v1t2). And there are 8 instances in which both vehicle capacity and time limit increases from a reference instance (in an instances with label x-v2t2, both the vehicle capacity and the distance limit are higher then the instance x-v1t1). As expected, since the pricing problem gets more difficult, the solution time tends to increase with the vehicle capacity and the vehicle time limit. In addition, the optimal cost does not increase when the vehicle capacity and/or the vehicle time limit increase(s). In 10 out of 16 pairs the computation time increases when the vehicle capacity increases, in 14 out of 16 pairs the computation time increases when the distance limit increase. In 8 cases out of 8, the computation time increases when both time and vehicle limits increase. However, there are also instances which favor the increase in time limit and/or capacity.

An increase in the vehicle capacity and time limit can reduce the cost through a change in the routing cost together with a decrease in the number of vehicles and/or a change in the set of selected facilities. On average, we calculated about %3.7 and %2.2 reduction in cost with vehicle capacity increase and time limit increase, respectively. The cost reduces about %5.8 on average if both vehicle and time limit increases.

Table 4.15: Results for instances with 25 customers and 5 facilities

Data File	I UB	LP	IP[1]	Gap %	Nds	CPU (m)	Loc	# Veh	# Rt
p01-f25-v1t1	4795	4504.4	4699	0	25	1.3	0,4	2,2	10
p01-f25-v1t2	4795	4425.8	4511	0	297	208.1	0,4	1,2	10
p01-f25-v2t1	4722	4366.1	4425	0	99	9.9	0,4	1,2	7
p01-f25-v2t2	4722	4295.4	4425	0	119	15.9	0,4	1,2	7
p01-l25-v1t1	5193	4849.8	4899	0	24	0.6	0,1	2,2	8
p01-l25-v1t2	4968	4725.4	4815	0	139	2.9	0,4	2,2	8
p01-l25-v2t1	4809	4669.0	4759	0	157	5.7	0,4	2,2	6
p01-l25-v2t2	4762	4537.1	4602	0	39	6.9	0,1	1,2	7
p03-f25-v1t1	5281	4810.0	5062	0	236	3.0	0,2	3,2	10
p03-f25-v1t2	5139	4701.1	4837	0	77	2.5	0,2	2,2	10
p03-f25-v2t1	5048	4606.1	4758	0	393	4.6	0,4	2,2	8
p03-f25-v2t2	4831	4502.5	4742	0	52	5.3	0,4	2,2	8
p03-m25-v1t1	4995	4697.4	4785	0	509	5.2	0,4	2,2	10
p03-m25-v1t2	4869	4578.2	4785	0.1	3790	360.0	0,4	2,2	10
p03-m25-v2t1	4992	4485.8	4689	0	59	2.9	0,4	2,2	7
p03-m25-v2t2	4767	4396.1	4479	0	9	8.7	0,4	1,2	7
p03-l25-v1t1	5186	4795.3	4842	0	17	0.6	0,2	2,2	9
p03-l25-v1t2	4871	4670.2	4803	0	37	1.9	0,4	2,2	7
p03-l25-v2t1	4805	4620.9	4741	0	41	2.5	0,4	2,2	6
p03-l25-v2t2	4805	4507.6	4655	0	37	7.5	2,4	1,2	7
p07-f25-v1t1	4870	4642.6	4761	0	55	1.4	0,2	1,3	7
p07-f25-v1t2	4823	4544.5	4685	0	67	131.2	2,4	2,1	9
p07-f25-v2t1	4721	4485.6	4703	0	229	8.8	0,2	2,2	6
p07-f25-v2t2	4721	4388.3	4482	0	197	15.1	0,2	1,2	6
p07-s25-v1t1	5412	5000.1	5098	0	599	1.5	0,4	2,3	9
p07-s25-v1t2	5238	4852.2	4889	0	123	2.9	0,4	2,2	9
p07-s25-v2t1	5159	4739.4	4787	0	21	0.6	0,4	2,2	7
p07-s25-v2t2	4934	4609.8	4781	0	100	3.6	0,4	2,2	8
p07-t25-v1t1	5134	4836.5	4898	0	35	0.9	0,1	2,2	8
p07-t25-v1t2	5162	4703.1	4886	0	1345	25.0	0,2	2,2	8
p07-t25-v2t1	5025	4646.2	4767	0	777	18.8	0,2	2,2	6
p07-t25-v2t2	4769	4513.1	4592	0	91	12.6	0,1	1,2	6

[1]Optimal or best IP found by the algorithm within the time limit.

Table 4.16: More Solution Details for 25-customer instances

Case	Changes btw [min - max]	Avg.
# of vehicles per facility	1 - 3	
Total # of vehicles	3 - 5	
Avg. # of routes per vehicle	1.5 - 3.3	2.1
Avg. # of customers per route	2.5 - 4.2	3.3
Avg. # of customers per vehicle	5 - 8.3	6.9
Max # of customers per vehicle	7 - 11	8.5

40-Customer Instances Since our previous results suggested that 2 step branch and price algorithm performs better for larger instances, here we used 2 step branch and price algorithm (E-BP-2S) for 40-customer instances. As explained before, E-BP-2S calls H-BP to obtain an initial set of columns and an upper bound, then starts running E-BP with the obtained initialization from H-BP. Based on the performance results from our previous experiments, we designed E-BP-2S with the following steps and design parameters.

- Run H-BP with a time limit of 2 CPU hours. Steps of the H-BP are:
  - Run I-Heur to find initial columns and an initial upper bound.
  - At each node node of the tree, column generation algorithm uses the following pricing algorithms:
    - Run at most 20 iterations 2p-ESPPRC-CS(15) with subset size 15.
    - Run 2p-ESPPRC-LL with label limit 5 and 10.
  - At root node, run standard branch and bound (BB) with current set of columns to update the primal solution.
- Run E-BP with a time limit of 6 CPU hours. Steps of the E-BP are:
  - Initialize E-BP with the columns and upper bound from H-BP.
  - At each node node of the tree, column generation algorithm uses the following pricing algorithms:
    - Run 2p-ESPPRC-LL with label limit 50, 100 and 200. The main implementation steps are described in Section 2.9.4.
    - Run 2p-ESPPRC.

Table 4.17 shows the results for instances with 40 customers. For each instance, Table 4.17 reports the upper bound obtained from I-Heur (I UB), the upper bound obtained at the end of H-BP (H-BP UB), the LP relaxation value at the root node (LP), the optimal objective value (IP), the integrality gap at the end of the running time (Gap %), the number of evaluated nodes (Nds), the CPU times spent in H-BP, E-BP and the total CPU time in minutes. As expected, the solution times in the 40-customer instances were larger than the 25-customer instances. Even though the algorithm terminated with a nonzero gap in some cases, the algorithm was able to prove optimality most of the time. For other instances, good solutions with small integrality gaps were obtained within the time limit. The gap was at most 0.35% for four of the instances and between 1.26% and 4.18% for the other seven instances.

The average deviation of the upper bound obtained from I-Heur from the optimal or best integer solution was 4.5%. However, the overall success of the algorithm can be attributed to the effectiveness of H-BP in finding good upper bounds. In all but one instance, the H-BP upper bound was

better than the I-Heur upper bound, and for 23 instances out of 32, the H-BP upper bound value equaled that of the optimal (or best integer) solution. The LP relaxation bounds were quite strong at the root node; the average deviation of the LP relaxation bound from the optimality was 2.2% in 40-customer instances, which are in line with the results of the smaller instances (instances with 20 customers in Section 4.3.2).

Table 4.17: Results for instances with 40 customers and 5 facilities

	I	H-BP			Gap		CPU(m)		
Data file	UB	UB	LP	IP[1]	%	Nds	H-BP	E-BP	Total
p01-f40-v1t1	7170	7131	6976.0	7131	0	807	53.2	150.0	203.2
p01-f40-v1t2	7170	7170	6826.2	6999	0.32	1065	120.0	360.0	480.0
p01-f40-v2t1	7064	6866	6700.2	6866	0	203	4.9	113.0	117.9
p01-f40-v2t2	6984	6821	6581.0	6821	1.26	35	35.4	360.7	396.1
p01-l40-v1t1	7449	7183	6899.7	7183	3.22	573	120.1	360.4	480.5
p01-l40-v1t2	7236	6934	6757.4	6934	0	43	5.6	48.0	53.6
p01-l40-v2t1	7166	6823	6616.7	6823	0	129	2.4	103.2	105.6
p01-l40-v2t2	7166	6633	6513.6	6612	0	25	8.4	187.6	196.0
p03-f40-v1t1	9253	8780	8697.4	8780	0	97	9.6	97.8	107.4
p03-f40-v1t2	9028	8753	8578.6	8753	0	41	28.0	154.6	182.6
p03-f40-v2t1	8943	8663	8439.0	8440	0	3	7.8	8.8	16.6
p03-f40-v2t2	8943	8438	8390.4	8438	0	9	11.5	138.2	149.7
p03-m40-v1t1	7827	7740	7050.9	7253	0	2524	120.0	360.1	480.1
p03-m40-v1t2	7544	7236	6907.7	7236	2.95	24	84.0	360.4	444.4
p03-m40-v2t1	7479	7132	6793.9	7132	2.79	27	120.0	360.1	480.1
p03-m40-v2t2	7430	6947	6655.5	6947	4.18	4	37.2	360.3	397.5
p03-l40-v1t1	7386	7167	6920.4	7167	0	183	1.5	55.6	57.1
p03-l40-v1t2	7212	6950	6771.6	6950	0.05	2121	120.0	360.0	480.0
p03-l40-v2t1	7121	6849	6662.9	6849	0	159	3.9	113.8	117.7
p03-l40-v2t2	7018	6846	6544.7	6639	0	53	39.0	290.9	329.9
p07-f40-v1t1	7424	7167	7053.1	7167	0	95	3.3	22.1	25.3
p07-f40-v1t2	7340	7001	6906.1	7001	0	433	11.0	75.5	86.5
p07-f40-v2t1	7174	6881	6745.5	6881	0	317	8.5	149.0	157.5
p07-f40-v2t2	7000	6845	6629.4	6845	2.34	43	20.3	360.1	380.4
p07-s40-v1t1	7860	7343	7256.3	7343	0	79	0.8	4.4	5.2
p07-s40-v1t2	7656	7124	7091.1	7117	0	29	1.8	6.5	8.3
p07-s40-v2t1	7466	7005	6924.5	7000	0	277	3.5	128.6	132.2
p07-s40-v2t2	7241	6944	6762.8	6944	0	251	5.7	211.2	216.9
p07-t40-v1t1	7283	7013	6940.5	7012	0	206	10.2	147.8	158.1
p07-t40-v1t2	7058	6972	6806.6	6972	0.35	66	76.2	360.5	436.7
p07-t40-v2t1	6945	6857	6649.8	6665	0.04	19	68.2	391.1	459.2
p07-t40-v2t2	6945	6658	6535.4	6658	1.64	6	26.1	363.3	389.5

[1]Optimal or best IP found by the algorithm within the time limit.

Table 4.18 presents some details about the optimal solution: the indices of open facilities (Loc), the number of vehicles used at each open facility (# Veh), and the total number of routes in each solution (# Rt). We listed the constructed vehicle routes and schedules (i.e., selected pairings) in the optimal (or best) solutions for all instances in Appendix A.2. In all instances 3 to 4 facilities are selected to be open, and total number of vehicles used changes between 4 to 6. Table 4.19 presents some summary statistics about the solutions, such as the number of vehicles used, the average number of routes per vehicle, the average number of customers per route and per vehicle, and the maximum number of customers per vehicle. In an optimal or best solution reported, a facility employs up to 3 vehicles. And each vehicle serves at least 2 routes and may serve up to 4 routes. On average, a vehicle covers 8.3 customers and the maximum number of customers per vehicle changes between 7 to 14.

Similar to the analysis done for 25-customer instances considering the change in the performance of the algorithm with vehicle capacity and time limit values, we can analyze the results for 40-customer instances. In 7 out of 16 pairs the computation time increases when the vehicle capacity increases, in 12 out of 16 pairs the computation time increases when the distance limit increase. In 6 cases out of 8, the computation time increases when both time and vehicle limits increase. Since in one third of the 40-customer instances, the algorithm terminated with the time limit, the solution times are not very sensitive to the changes in the capacity parameters. On average, we calculated about %3.8 and %1.6 reduction in cost with vehicle capacity increase and time limit increase, respectively. The cost reduces about %5.3 on average if both vehicle and time limit increases. These results are in line with the corresponding averages for 25-customer instances.

The solution information provided in Table 4.15 and Table 4.18 highlights the value of solving the integrated problem instead of a series of problems. Although the same number of facilities is selected within an instance group (meaning a fixed set of customer locations and a set of candidate facility locations), the particular facilities selected may differ depending on the vehicle capacity and time limit values. Which facilities are selected impacts which customers are served by each facility and hence the sequence of the routes and the structure of the pairings. From the solution information, we can see that, within an instance group, increases in the vehicle capacity and the time limit reduce the total cost through changes in facility, decreases in the number of vehicles required, changes in the underlying route sequences.

4.4.6 Performance of Branch and Price Algorithm for Instances from Literature

The only LRSP instances available in the literature are those of Lin et al. (2002); Lin et al. (2002) present six instances: a base instance that includes 27 customers and four facility locations in Hong Kong plus five instances generated from the original by considering three subsets of 10 customers and two subsets of 12 customers while keeping the same four facility locations. We used only the four instances available to the reader to test our branch-and-price algorithm, in particular, our E-BP

Table 4.18: Solution details for instances with 40 customers and 5 facilities

Data file	Loc	# Veh	# Rt	Data file	Loc	# Veh	# Rt
p01-f40-v1t1	0,3,4	2,2,1	14	p03-l40-v1t1	0,2,4	2,2,2	13
p01-f40-v1t2	0,2,4	1,2,2	14	p03-l40-v1t2	0,2,4	2,1,2	13
p01-f40-v2t1	0,2,4	1,2,2	11	p03-l40-v2t1	0,2,4	1,2,2	10
p01-f40-v2t2	0,3,4	2,1,1	10	p03-l40-v2t2	0,2,4	1,1,2	10
p01-l40-v1t1	0,2,4	1,2,3	15	p07-f40-v1t1	0,2,3	1,2,2	13
p01-l40-v1t2	0,2,4	1,2,2	15	p07-f40-v1t2	0,2,4	1,2,2	13
p01-l40-v2t1	0,2,4	1,2,2	10	p07-f40-v2t1	0,2,4	1,2,2	10
p01-l40-v2t2	0,2,4	1,1,2	10	p07-f40-v2t2	1,2,4	1,1,2	10
p03-f40-v1t1	0,1,2,4	2,1,1,1	16	p07-s40-v1t1	0,1,4	2,2,2	14
p03-f40-v1t2	0,1,2,4	1,1,2,1	16	p07-s40-v1t2	0,1,4	1,2,2	13
p03-f40-v2t1	0,1,2,4	1,1,1,1	11	p07-s40-v2t1	0,1,4	1,2,2	10
p03-f40-v2t2	0,1,2,4	1,1,1,1	11	p07-s40-v2t2	0,2,4	1,2,2	10
p03-m40-v1t1	0,2,4	2,2,2	15	p07-t40-v1t1	0,1,4	1,2,2	13
p03-m40-v1t2	0,2,4	2,2,2	15	p07-t40-v1t2	0,2,4	1,2,2	13
p03-m40-v2t1	0,2,4	2,2,2	11	p07-t40-v2t1	0,1,4	1,1,2	10
p03-m40-v2t2	0,1,4	2,1,2	12	p07-t40-v2t2	0,2,4	1,1,2	9

Table 4.19: More Solution Details for 40-customer Instances

Case	Changes btw [min - max]	Avg.
# of vehicles per facility	1 - 3	
Total # of vehicles	4 - 6	
Avg. # of routes per vehicle	1.8 - 3.2	2.5
Avg. # of customers per route	2.5 - 4.4	3.4
Avg. # of customers per vehicle	6.7 - 10	8.3
Max # of customers per vehicle	7 - 14	10.2

algorithm.

Lin et al. (2002) solve these instances with a branch-and-bound algorithm and with their heuristic algorithm. Within a time limit of 10,000 seconds, their branch-and-bound algorithm is able to prove optimality only for the 10 customer instances and is not able to find a feasible solution for the original 27 customer problem. The heuristic quickly finds solutions for all instances. Table 4.20 compares their reported results to results obtained with E-BP. As shown in Table 4.20, our algorithm quickly provides optimal solutions for all four instances. (The * indicates the best solution found in 10,000 CPU seconds on a Pentium III machine.)

Table 4.20: Comparison of Lin et al. (2002) results and branch-and-price results

Instance		Lin B & B		Lin Heuristic		Akça E-BP	
# Cust	# Fac	Objective	CPU (s)	Objective	CPU(s)	Objective	CPU(s)
10	4	309,817	1155	309,817	0.44	309,817	0.66
10	4	309,808	982	309,808	0.49	309,808	0.10
12	4	312,036*	$> 10{,}000$	312,036	0.82	312,036	0.05
27	4	-	-	625,752.5	6	625,750.2	57.83

4.5 Computational Experiments For Chapter 3

The goal of the computational experiments in this section was to test the performance of the branch and price algorithm extended with some of the cutting planes described in Chapter 3. We empirically show the effect of each implemented type of cut. We implemented lifted cover inequalities, generalized assignment polytope cuts and disjunctive cuts.

4.5.1 Lifted Cover Inequalities and GAP Cuts

In this section, we evaluated the effect of lifted cover inequalities (inequalities in the form of 3.14) and the GAP cuts (inequalities in the form of 3.16) which are valid for the projections (knapsack polytope and generalized assignment polytope) of SPP-LRSP. The details of these inequalities were described in Section 3.3.2.

For the experiments in this Section, we used eight 25-customer instances with largest vehicle capacity and time limit (x-v2t2 instances). We used 1-step branch and price algorithm with 2p-ESPPRC-CS(12), 2p-ESPPRC-LL(50), 2p-ESPPRC-LL(100), 2p-ESPPRC-LL(200) and 2p-ESPPRC pricing algorithms. We ran cut generation at each node if it its depth is less than 9. At each node the steps of the algorithm extended with cut generation is as follows:

- Call COLGEN.
- If the LP is fractional and depth ≤ 8, call CUTGEN, otherwise STOP.

- If a cut is generated and added to the model, call COLGEN, otherwise STOP.

Here, we refer cut generation function as CUTGEN. Depending on the type of the cut we wanted, CUTGEN procedure searches for either lifted cover inequalities or GAP cuts, or both. We refer these procedures as CUTGEN(KNAP), CUTGEN(GAP) and CUTGEN(KNAP,GAP). As explained in Section 3.3.2, at each node, CUTGEN procedure projects the LP solution from (t, z) space to (t, v) space and uses separation algorithms given in the Section to generate required type of cuts. If the new cuts are generated, they are projected back to the original space as in (3.14) and (3.16). At each node, we called CUTGEN only once and added generated cuts all at once before we do column generation again to find the optimal LP relaxation solution with cuts at the evaluated node. In addition, these cuts are global cuts.

Table 4.21 compares the performance of the algorithms with CUTGEN(KNAP) and CUTGEN(GAP) with the base algorithm without any cuts. Table 4.21 reports the number of evaluated nodes (Column Nds) and the number of cuts generated in each algorithm. In addition, Table gives the ratio of the time of the algorithm with CUTGEN to the time of the algorithm without cuts (column Time W/WO).

Table 4.21 shows that CUTGEN procedure slightly effects the performance of the algorithm. In all of the instances except one case, if the cut generation is successful the number of evaluated nodes either stayed same or decreased. On average, the computational time of the algorithms with CUTGEN is almost same to the base algorithm. On each instance, the time in the algorithms with CUTGEN is either slightly lower or slightly higher than the time in the base algorithm.

Table 4.21: Cover Inequalities and GAP Cuts for 25-customer Instances

	Base Alg.	Alg. with GAP Cuts			Alg. with COVER Cuts		
Data file	Nds	Nds	Time W/WO	# cuts	Nds	Time W/WO	# cuts
p01-f25-v2t2	29	29	1	0	41	0.97	3
p01-l25-v2t2	33	33	0.96	1	33	0.99	1
p03-f25-v2t2	44	40	1.03	5	36	1.14	14
p03-m25-v2t2	9	9	1.07	0	9	1.02	0
p03-l25-v2t2	37	37	1.04	0	37	1.06	4
p07-f25-v2t2	143	143	1.06	0	143	1.02	0
p07-s25-v2t2	85	83	0.97	1	81	1	5
p07-t25-v2t2	91	81	0.94	2	69	0.9	7
avg			1.01			1.01	

Table 4.22 presents the results of the algorithm with CUTGEN(KNAP,GAP) and compares it with the base algorithm. The labels in the Table are same as Table 4.21. The results are parallel to the algorithms with CUTGEN(KNAP) and CUTGEN(GAP). Even though the LP relaxation bound improves slightly with the generated cuts at the relevant nodes, the CUTGEN procedure does not have a significant contribution to the performance of the overall algorithm in this experiment.

Table 4.22: Cover Inequalities and GAP Cuts for 25-customer Instances

	Base Alg.	Alg. with COVER + GAP Cuts			
Data file	Nds	Nds	Time W/WO	# gap cuts	# cover cuts
p01-f25-v2t2	29	41	0.9	0	3
p01-l25-v2t2	33	33	0.99	1	1
p03-f25-v2t2	44	36	1.14	0	14
p03-m25-v2t2	9	9	1.03	0	0
p03-l25-v2t2	37	37	1.05	0	4
p07-f25-v2t2	143	143	1.01	0	0
p07-s25-v2t2	85	81	0.99	0	5
p07-t25-v2t2	91	69	0.9	0	7
avg			1		

4.5.2 Disjunctive Cuts

In this Section, we present some computational results for four implementations of disjunctive cutting plane generation algorithm described in Section 3.4. We refer the cut generation procedures for each of four implementations as DISJCUTGEN(1), DISJCUTGEN(2), DISJCUTGEN(3), and DISJCUTGEN(4).

In this set of experiments, we used the same 1 step branch and price algorithm design as in Section 4.5.1, except we used DISJCUTGEN procedures instead of CUTGEN procedures. Again, at each node, we called the cut generation procedure at most once up to depth 8 in the tree. Another difference was that we added the generated cuts as local cuts for the corresponding node. In DISJCUTGEN(2), DISJCUTGEN(3) and DISJCUTGEN(4) (second, third and fourth implementations), at each node we searched one set of disjunctive inequality and at most one disjunctive cut. In DISJCUTGEN(1) (first disjunctive cutting plane implementation described in Section 3.4.1), at each node when we called the cut generation function, we searched at most two sets of disjunctions in the form of (3.35) and (3.36) and searched for at most four cuts.

We compared the performance of each implementation of DISJCUTGEN using eight 25-customer instances. Table 4.23 evaluates the effect of using DISJCUTGEN(1) and DISJCUTGEN(2), and Table 4.24 evaluates the effect of using DISJCUTGEN(3) and DISJCUTGEN(4). In Tables, base algorithm refers to the algorithm without any cutting planes. Both Tables 4.23 and 4.24 present the number of evaluated nodes (Nds) in each algorithm, fraction of time of each algorithm with disjunctive cuts to the base algorithm (Time W/WO), and the number of total cuts (# cuts) generated in each algorithm. In all four implementations, in most of the instances, the number of evaluated nodes stays same or increases compared to the base algorithm. On average, the algorithm time increases 8% with DISJCUTGEN(1) and 25% with DISJCUTGEN(2). DISJCUTGEN(3) increases the total algorithm time about 3 times while DISJCUTGEN(4) increases the algorithm time significantly, about 13 times.

To evaluate the performance reduction with DISJCUTGEN procedures further, in Table 4.25, for each implementation and for each instance, we present percentage of total algorithm time spent to generate cuts (DISJCUTGEN time /Total), and average cut generation time per cut (DISJCUTGEN time per cut (s)). Time spent in DISJCUTGEN procedure is mostly the time to solve the CGLPs. Since the CGLP in implementation 1 is very small compared to other implementations, the time spent in DISJCUTGEN is negligible. CGLP in implementation 2 is larger than implementation 1, but much smaller than the CGLPs in implementations 3 and 4 with respect to number of constraints and variables. The percentage of total time spent in DISJCUTGEN procedure increases significantly in implementation 3 and 4. The number of variables and main set of constraints in CGLPs for implementations 3 and 4 are same. However, in implementation 4, we also added some restrictions for the coefficients of variables in the generated cut, this makes the CGLP more difficult to solve and results more significant increase in time for DISJCUTGEN. These results can also be seen from the reported DISJCUTGEN times per cut.

Table 4.23: Disjunctive Cutting Planes: Implementation 1 and 2

	Base Alg.	Alg. with DISJCUTGEN(1)			Alg. with DISJCUTGEN(2)		
Data file	Nds	Nds	Time W/WO	# cuts	Nds	Time W/WO	# cuts
p01-f25-v2t2	29	69	1.01	7	40	1.25	10
p01-l25-v2t2	33	35	1.15	9	33	1.17	10
p03-f25-v2t2	44	-	-	18	50	1.59	27
p03-m25-v2t2	9	9	1.27	6	11	1.29	6
p03-l25-v2t2	37	39	1.09	10	39	1.16	15
p07-f25-v2t2	143	143	0.98	0	119	1.18	11
p07-s25-v2t2	85	96	1.08	30	98	1.35	26
p07-t25-v2t2	91	85	0.96	16	93	0.99	12
avg			1.08			1.25	

- terminates since number of non zero coefficients for constraints $\geq$ 76000.

Table 4.24: Disjunctive Cutting Planes: Implementation 3 and 4

	Base Alg.	Alg. with DISJCUTGEN(3)			Alg. with DISJCUTGEN(4)		
Data file	Nds	Nds	Time W/WO	# cuts	Nds	Time W/WO	# cuts
p01-f25-v2t2	29	71	2.31	34	41	7.06	13
p01-l25-v2t2	33	37	2.18	28	31	8.38	11
p03-f25-v2t2	44	591	6.26	41	44	29.26	32
p03-m25-v2t2	9	9	1.78	7	7	3.83	5
p03-l25-v2t2	37	43	1.89	23	45	10.69	19
p07-f25-v2t2	143	53	1.66	28	157	8	12
p07-s25-v2t2	85	213	4.48	66	95	31.38	37
p07-t25-v2t2	91	52	1.85	37	83	6.08	19
avg			2.8			13.08	

Table 4.25: Time in DISJCUTGEN procedure for implementations 1, 2, 3 and 4

Data file	DISJCUTGEN time /Total				DISJCUTGEN time per cut (s)			
	Imp.1	Imp.2	Imp.3	Imp.4	Imp.1	Imp.2	Imp.3	Imp.4
p01-f25-v2t2	0	0.02	0.2	0.6	0.1	1.46	7.89	191.41
p01-l25-v2t2	0	0.02	0.31	0.8	0.05	0.88	10.62	264.22
p03-f25-v2t2	0	0.05	0.47	0.91	0	0.85	20.71	243.07
p03-m25-v2t2	0	0.01	0.09	0.37	0.02	0.52	10.24	126.99
p03-l25-v2t2	0	0.02	0.33	0.81	0.05	0.68	12.49	211.58
p07-f25-v2t2	0	0.03	0.25	0.75	-	1.98	9.9	329.24
p07-s25-v2t2	0	0.09	0.66	0.94	0.04	1.03	9.94	176.54
p07-t25-v2t2	0	0.04	0.31	0.7	0.12	1.92	9.96	142.53
avg	0	0.03	0.33	0.74	0.05	1.16	11.47	210.7

The presented results showed that the DISJCUTGEN procedure reduces the algorithm performance. However, even though the time increases in other parts of the main algorithm, are these cuts helping somehow by improving the LP relaxation bound at the related nodes? To be able see the result of this question, we looked at the optimal LP solutions before and after adding the cuts in each implementation. **ZE: maybe prepare a table?**

In *implementation 3*, almost none of the cuts violates the current fractional LP solution of RMP ($\bar{z}$), thus the generated cuts do not separate the solution from the feasible search region for the algorithm. The reason for that to happen is the combination of the coefficients obtained by solving the CGLP. As explained in Section 3.4.3, fractional LP solution of RMP $\bar{z}$, projected on to the variable space based on the links of the graph and the vehicles. A disjunctive cut which is violating the current projected fractional solution is found first (by solving the CGLP), but before projecting the obtained cut to the master problem space, the coefficients are combined accordingly to get rid of the vehicle index. Therefore, the new cut is not necessarily violates the current solution in either space. Therefore, implementation 3 performs very poorly with respect to LP bound improvement as well as the time. Therefore, we can easily eliminate implementation 3 of the disjunctive procedure from the set of cuts considered to extend the algorithms.

Implementation 4 was designed to overcome the implementation 3's problem of generating cuts which do not violate the LP solution. Instead of combining the coefficients after the cut is generated, implementation 4 adds new conditions to the CGLP to eliminate the combination procedure. Thus, the generated cuts with implementation 4 provide coefficients without vehicle index, i.e., the vehicle index is dropped at the beginning because of equality conditions. Therefore, the generated cut must violate the current fractional solution. Implementation 4 is successful for small size instances, however, our computational tests with 25-customer instances showed that addition of new conditions to the CGLP increases the solution time significantly as seen from Table 4.25. In addition to this drawback, by investigating the LP relaxation solution after and before the cut, we found that there exists some nodes in which even though the generated cut improves the LP relaxation value at

first, after the additional column generation steps designed to consider the new cut, the optimal LP solution moves to a point which has the objective value that is same or only slightly better than the previous objective value. As seen from the form of the generated cut (3.63), there are coefficients corresponds to directed links between nodes in the problem. Since the distance matrix is symmetric, the column generation algorithm generates new columns which include different orientations of routes that are already included in the previous columns in the previous LP solution. The new LP solution satisfies the cut, but has the same objective value as before because of these new equivalent columns. Therefore, the LP solution value does not change while the set of non zero pairing variables changes. We provide an example to explain this problem.

Example Branch,price and cut algorithm with cuts from DISJCUTGEN(4) is run for instance p01-f25-v1t1. In one of the nodes, the LP relaxation value before the addition of the generated cut is equal to the LP relaxation value after the cut is added and column generation is done. We compared two LP solutions and only listed the non zero variables which are different in these two solutions.
Before cut is added:

$$\begin{aligned}
\bar{z}_1 &= 0.188 \quad : \quad \mathrm{f}_0 - 4 - 11 - \mathrm{f}_0 - 22 - 6 - 23 - \mathrm{f}_0 - 13 - 24 - \mathrm{f}_0 - 17 - \mathrm{f}_0 \\
\bar{z}_2 &= 0.16 \quad : \quad \mathrm{f}_0 - 3 - 16 - 18 - 12 - \mathrm{f}_0 - 22 - 6 - 5 - \mathrm{f}_0 - 13 - 24 - \mathrm{f}_0 \\
\bar{z}_3 &= 0.028 \quad : \quad \mathrm{f}_0 - 22 - 6 - 23 - \mathrm{f}_0 - 3 - 18 - 12 - \mathrm{f}_0 - 13 - 5 - \mathrm{f}_0
\end{aligned}$$

After cut is added:

$$\begin{aligned}
\bar{z}_2 &= 0.131 \quad : \quad \mathrm{f}_0 - 3 - 16 - 18 - 12 - \mathrm{f}_0 - 22 - 6 - 5 - \mathrm{f}_0 - 13 - 24 - \mathrm{f}_0 \\
\bar{z}_4 &= 0.188 \quad : \quad \mathrm{f}_0 - 11 - 4 - \mathrm{f}_0 - 23 - 6 - 22 - \mathrm{f}_0 - 24 - 13 - \mathrm{f}_0 - 17 - \mathrm{f}_0 \\
\bar{z}_5 &= 0.029 \quad : \quad \mathrm{f}_0 - 12 - 18 - 16 - 3 - \mathrm{f}_0 - 5 - 6 - 22 - \mathrm{f}_0 - 24 - 13 - \mathrm{f}_0 \\
\bar{z}_6 &= 0.028 \quad : \quad \mathrm{f}_0 - 23 - 6 - 22 - \mathrm{f}_0 - 12 - 18 - 3 - \mathrm{f}_0 - 13 - 5 - \mathrm{f}_0
\end{aligned}$$

After the cut was added, the column generation generated $\bar{z}_4$ and $\bar{z}_6$ and replaced $\bar{z}_1$ and $\bar{z}_3$ with them. As the sequences of customers in the corresponding pairings are investigated, equivalence of $\bar{z}_1$ and $\bar{z}_4$, and $\bar{z}_3$ and $\bar{z}_6$ are obvious. In addition, $\bar{z}_5$ which is equal to $\bar{z}_2$ was generated and took nonzero value in the solution.

Even though there are some nodes in which the disjunctive cut from DISJCUTGEN(4) strictly improves the LP solution value, the nodes which react the cuts in a similar way to the example reduce the performance of the overall algorithm especially because of the large CGLP solution times. Therefore, we can conclude that the branch and price algorithm without DISJCUTGEN(4) is better of than the the branch, price and cut algorithm with DISJCUTGEN(4).

Chapter 5

Case Study: LRP

Chapter 6

Case Study: Effect of Scheduling Constraint: LRSP vs LRP-DC

One of the open facilities is different in 3 of the instances.

6.1 Effect of Scheduling Constraint in the LRSP

In this section, we experimentely show the benefit of optimizing scheduling decision integrated with the location and routing decisions. In other words, the gain that can be obtained by solving the LRSP. To be able to measure this benefit, we solve the LRP with distance constraints (LRPDC). In LRPDC, each route corresponds to a vehicle. As the vehicle capacity gets larger, we expect that the number of vehicles used in optimal solution of the LRSP gets closer to the number of vehicles used in the LRPDC. Therefore, solving the LRSP gets closer to solving the LRPDC. After solving the LRPDC, bin packing problem is solved to assign routes to vehicles considering the time limit. Therefore, scheduling decision has been made independent of the location and routing decisions.

The branch and price algorithm for the LRSP can easily be changed to solve the LRPDC. The master problem formulations for both LRSP and LRPDC are same. These two problems differ in the definitions of their columns. A column in LRSP represents a set of routes while a column in LRPDC represent a single route feasible with respect to vehicle capacity and time limit. Therefore, the pricing problem for the LRPDC can be formulated as an ESPPRC where the vehicle capacity and time limit are the resources. In the 2 phase pricing problem designed for the LRSP, the first phase generates routes using ESPPRC with vehicle capacity and time limit resources. And the second phase combines these routes into pairings using ESPPRC with time limit as a resource. If we only solve the first phase, the problem generates feasible columns for the LRPDC. Therefore, by changing the pricing problem in the branch and price algorithm, we can solve the LRPDC.

Instances We used the lrsp instances generated from Cordeau et al. (1997) with 25 customers. We considered instances with the first time limit value (140). For each location set, we used 4 different vehicle capacity values. In addition to C_1^V, C_2^V values, we generated 2 larger vehicle capacities refered as C_3^V (generated by setting $\alpha = 0.5$) and C_4^V (generated by setting $\alpha = 0.8$). We ran 1 step exact branch and price algorithm with a 8 CPU hour limit for both LRSP and LRPDC.

Table 6.1: Vehicle Capacity values used in the instances

Ref	C_1^V	C_2^V	C_3^V	C_4^V
p01-f, p01-l	50	70	120	170
p03-f, p03-m, p03-l	60	80	140	190
p07-f, p07-s, p07-t	50	70	110	150

Table 6.2: Results for instances with 25 customer 5 facilities - LRPDC

Data	LP	IP[1]	Gap	Nodes	CG CPU(s)	LP CPU(s)	Total CPU(s)
p01-f25-v1t1	5776.23	5840	0	23	9.2	0.42	9.7
p01-f25-v2t1	5150.61	5320	0	57	90.53	1.74	92.56
p01-f25-v3t1	4495.46	4575	0	19	1273.52	1.53	1276.2
p01-f25-v4t1	4238.27	4567	0	67	17127.7	7.2	17147.41
p01-l25-v1t1	5536.4	5715	0	225	46.74	2.86	50.41
p01-l25-v2t1	4986.12	5190	0	149	187.69	3.45	191.74
p01-l25-v3t1	4409	4409	0	1	65.61	0.3	65.98
p01-l25-v4t1	4341.22	4397	0	27	93.2	2.51	95.99
p03-f25-v1t1	-	-	-	-	-	-	-
p03-f25-v2t1	5260.20	5727	3.33%	11629	5326.70	22901.09	28803.22
p03-f25-v3t1	4523.67	4631	0	49	892.3	5.29	898.62
p03-f25-v4t1	4291.83	4629	0	1441	5414.1	1945.14	9221.83
p03-m25-v1t1	5737.85	5917	0	169	18.1	3.27	21.64
p03-m25-v2t1	5186.79	5364	0	33	36.97	0.72	37.91
p03-m25-v3t1	4484.24	4604	0	29	2977.12	3.26	2982.8
p03-m25-v4t1	4250.79	4636	1.16%	60	28925.79	15.44	28954.55
p03-l25-v1t1	5390.38	5478	0	91	34.9	1.07	36.24
p03-l25-v2t1	4960.81	4984	0	73	107.97	6.13	114.35
p03-l25-v3t1	4438.88	4664	0	95	1258.08	16.69	1276.84
p03-l25-v4t1	4406.87	4438	0	5	35.06	0.45	35.58
p07-f25-v1t1	5397.9	5436	0	43	26.46	0.75	27.34
p07-f25-v2t1	4887.94	4933	0	77	162.09	2.51	165.17
p07-f25-v3t1	4419.96	4630	0	83	1774.32	5.03	1781.28
p07-f25-v4t1	4281.81	4397	0	87	1267.69	8.46	1281.26
p07-s25-v1t1	5802.57	5815	0	39	12.31	0.48	12.88
p07-s25-v2t1	5155.24	5231	0	70	40.3	1.33	41.77
p07-s25-v3t1	4623.57	4705	0	62	108.64	5.07	114.14
p07-s25-v4t1	4440	4440	0	1	6.8	0.24	7.06
p07-t25-v1t1	5601	5769	0	3055	239.89	37.42	357.96
p07-t25-v2t1	5038.68	5203	0	112	188.76	2.28	191.43
p07-t25-v3t1	4499.99	4675	0	665	11079.66	632.88	11867.98
p07-t25-v4t1	4295.71	4400	0	33	1135.84	7.51	1145.31

Table 6.3: Results for instances with 25 customer 5 facilities - LRPDC

Data	LP	Opt.	Nodes	CG CPU(s)	LP CPU(s)	Total CPU(s)
p01-f25-v1t1	4504.4	4699	23	91.61	11.23	103.9
p01-f25-v2t1	4366.12	4425	85	560.53	24.04	605.51
p01-f25-v3t1	4204.95	4350	23	1876.65	4.93	1883.14
p01-f25-v4t1	4167.08	4343	31	12970.11	7.15	12979.51
p01-l25-v1t1	4849.78	4899	25	42.06	3.31	45.82
p01-l25-v2t1	4668.98	4759	155	495.42	82.15	585.49
p01-l25-v3t1	4409	4409	1	48.89	0.33	49.29
p01-l25-v4t1	4341.22	4397	27	197.4	2.62	200.48
p03-f25-v1t1	4809.96	5062	244	202.05	223.47	432.03
p03-f25-v2t1	4606.09	4758	501	339.29	286	860.33
p03-f25-v3t1	4367.3	4631	83	1592.11	13.07	1607.04
p03-f25-v4t1	4257.87	4409	47	1970.93	16.41	1989.9
p03-m25-v1t1	4697.37	4785	755	199.48	1125.33	2154.94
p03-m25-v2t1	4485.79	4689	59	300.52	31.89	334.84
p03-m25-v3t1	4270.49	4404	33	2913.99	11.04	2926.61
p03-m25-v4t1	4197.1	4382	67	22760.99	45.69	22811.89
p03-l25-v1t1	4795.3	4842	17	46.08	3.12	49.72
p03-l25-v2t1	4620.94	4741	47	306.36	10.7	319.08
p03-l25-v3t1	4424.46	4451	7	58.99	0.54	59.63
p03-l25-v4t1	4404.59	4438	7	66.77	0.63	67.54
p07-f25-v1t1	4642.59	4761	45	113.31	7.97	122.33
p07-f25-v2t1	4485.56	4703	183	849.5	51.21	913.79
p07-f25-v3t1	4329.47	4405	31	1594.44	2.15	1597.3
p07-f25-v4t1	4281.81	4397	105	1325.89	10	1342.28
p07-s25-v1t1	5000.13	5098	419	82.52	20.39	163.67
p07-s25-v2t1	4739.42	4787	21	48.49	2.33	51.14
p07-s25-v3t1	4521.02	4695	51	181.78	2.68	184.83
p07-s25-v4t1	4440	4440	1	12.85	0.34	13.21
p07-t25-v1t1	4836.55	4898	33	86.97	11.45	99.73
p07-t25-v2t1	4646.18	4767	679	866.8	299.58	2601.99
p07-t25-v3t1	4420.98	4450	23	532.18	4.31	537.11
p07-t25-v4t1	4288.74	4400	37	1211.33	6.64	1218.96

Table 6.4: Solution details for LRPDC and LRSP

Data	LRPDC					LRSP			
	Fac. Index	**# of vec.**	**# of vec. after BP**	**Fac. Cost**	**Variable Cost**	**Fac. Index**	**# of vec.**	**Fac. Cost**	**Variable Cost**
p01-f25-v1t1	0,4	9	4	3368	447	0,4	4	3368	431
p01-f25-v2t1	0,4	7	4	3368	377	0,4	3	3368	382
p01-f25-v3t1	0,4	4	4	3368	307	0,4	3	3368	307
p01-f25-v4t1	0,4	4	4	3368	299	0,4	3	3368	300
p01-l25-v1t1	**0,4**	8	5	3368	547	**0,1**	4	3460	539
p01-l25-v2t1	0,4	6	5	3368	472	0,4	4	3368	491
p01-l25-v3t1	0,4	3	3	3368	366	0,4	3	3368	366
p01-l25-v4t1	0,4	3	3	3368	354	0,4	3	3368	354
p03-f25-v1t1	-	-	-	-	-	0,2	5	-	-
p03-f25-v2t1	-	-	-	-	-	0,4	4	-	-
p03-f25-v3t1	-	-	-	-	-	0,4	4	-	-
p03-f25-v4t1	-	-	-	-	-	0,4	3	-	-
p03-m25-v1t1	0,4	9	5	3368	524	0,4	4	3368	517
p03-m25-v2t1	0,4	7	4	3368	421	0,4	4	3368	421
p03-m25-v3t1	0,4	6	4	3368	336	0,4	3	3368	361
p03-m25-v4t1	0,4	4	4	3368	368	0,4	3	3368	339
p03-l25-v1t1	**0,4**	7	6	3368	535	**0,2**	4	3396	546
p03-l25-v2t1	0,4	5	5	3368	491	0,4	4	3368	473
p03-l25-v3t1	0,4	4	4	3368	396	0,4	3	3368	408
p03-l25-v4t1	0,4	3	3	3368	395	0,4	3	3368	395
p07-f25-v1t1	0,2	7	4	3396	465	0,2	4	3396	465
p07-f25-v2t1	0,2	5	4	3396	412	0,2	4	3396	407
p07-f25-v3t1	0,4	4	3	3368	362	0,4	3	3368	362
p07-f25-v4t1	0,4	3	3	3368	354	0,4	3	3368	354
p07-s25-v1t1	0,4	8	7	3368	647	0,4	5	3368	605
p07-s25-v2t1	0,4	6	5	3368	513	0,4	4	3368	519
p07-s25-v3t1	0,4	4	4	3368	437	0,4	4	3368	427
p07-s25-v4t1	0,4	3	3	3368	397	0,4	3	3368	397
p07-t25-v1t1	**0,2**	8	5	3396	573	**0,1**	4	3460	538
p07-t25-v2t1	0,2	6	5	3396	457	0,2	4	3396	471
p07-t25-v3t1	0,4	4	4	3368	407	0,4	3	3368	407
p07-t25-v4t1	0,4	3	3	3368	357	0,4	3	3368	357

Appendix A

Solution Details

A.1 Details of the Optimal Solutions in 25-customer Instances

Customers are indexed from 0 to 24, and facilities are indexed with t_0, to t_4.

p01-f25-v1t1.txt: Optimal Cost = 4699.

t_0- 22 - 6 - 23 - t_0- 11 - 5 - t_0- 17 - t_0- 24 - 13 - t_0

t_0- 12 - 18 - 16 - 3 - t_0

t_4- 4 - 14 - 9 - 8 - t_4- 10 - 15 - 20 - t_4

t_4- 0 - 7 - 21 - t_4- 19 - 2 - t_4- 1 - t_4

p01-f25-v1t2.txt: Optimal Cost = 4511.

t_0- 24 - 13 - t_0- 22 - 6 - 23 - t_0- 5 - 18 - 12 - t_0- 17 - t_0

t_4- 0 - 7 - 21 - t_4- 11 - 10 - t_4- 19 - 2 - t_4- 1 - t_4

t_4- 9 - 14 - 16 - 3 - 4 - t_4- 20 - 8 - 15 - t_4

p01-f25-v2t1.txt: Optimal Cost = 4425.

t_0- 5 - 22 - 6 - 23 - t_0- 24 - 12 - 18 - 3 - t_0- 17 - 13 - t_0

t_4- 20 - 8 - 9 - 14 - 16 - 11 - t_4- 2 - 19 - t_4

t_4- 1 - 0 - 7 - 21 - t_4- 15 - 4 - 10 - t_4

p01-f25-v2t2.txt: Optimal Cost = 4425.

t_0- 5 - 6 - 22 - 23 - t_0- 24 - 12 - 18 - 3 - t_0- 13 - 17 - t_0

t_4- 20 - 8 - 9 - 14 - 16 - 11 - t_4- 19 - 2 - t_4

t_4- 1 - 0 - 7 - 21 - t_4- 10 - 4 - 15 - t_4

p01-l25-v1t1.txt: Optimal Cost = 4899.

t_0- 17 - 0 - 22 - t_0- 18 - 11 - 21 - t_0

t_0- 15 - 14 - 16 - t_0- 20 - 6 - 1 - t_0
t_1- 9 - 10 - 2 - 5 - t_1- 8 - 4 - t_1
t_1- 19 - 7 - 13 - t_1- 12 - 23 - 24 - 3 - t_1

p01-l25-v1t2.txt: Optimal Cost = 4815.
t_0- 17 - 0 - 22 - t_0- 18 - 11 - 21 - t_0
t_0- 15 - 14 - 16 - t_0- 20 - 6 - 1 - t_0
t_4- 9 - 10 - 2 - 5 - t_4- 8 - 4 - t_4
t_4- 19 - 7 - 13 - t_4- 12 - 23 - 24 - 3 - t_4

p01-l25-v2t1.txt: Optimal Cost = 4759.
t_0- 17 - 0 - 5 - 2 - 22 t_0
t_0- 15 - 14 - 16 - 18 - t_0- 1 - 20 - 21- t_0
t_4- 13 - 7 - 19 - 11 - 6 -t_4
t_4- 3 - 8 - 10 - 9 - t_4- 24 - 4 - 23 - 12 - t_4

p01-l25-v2t2.txt: Optimal Cost = 4602.
t_0- 18 - 16 - 14 - 15 - t_0- 22 - 1 - 20 - 21 - t_0
t_1- 24 - 13 - 7 - 19 - 11 - t_1- 23 - 4 - 8 - 3 - t_1- 12 - t_1
t_1- 17 - 0 - 5 - 2 - 10 - 9 - t_1- 6 - t_1

p03-f25-v1t1.txt: Optimal Cost = 5062.
t_0- 22 - 23 - 17 - 2 - t_0- 16 - 15 - 5 - t_0
t_0- 4 - 20 - t_0
t_0- 0 - 21 - 1 - t_0- 3 - t_0
t_2- 24 - 8 - 11 - t_2- 18 - 13 - t_2- 10 - t_2
t_2- 12 - 19 - 14 - 7 - t_2- 9 - 6 - t_2

p03-f25-v1t2.txt: Optimal Cost = 4837.
t_0- 22 - 23 - 17 - 2 - t_0- 5 - 15 - 16 - t_0
t_0- 4 - 20 - t_0- 0 - 21 - 1 - t_0- 3 - t_0
t_2- 8 - 24 - 11 - t_2- 9 - 6 - 7 - t_2- 10 - t_2
t_2- 13 - t_2- 18 - 14 - 19 - 12 - t_2

p03-f25-v2t1.txt: Optimal Cost = 4758.
t_0- 10 - 9 - 11 - t_0- 6 - 13 - 18 - 7 - t_0
t_0- 4 - 19 - 14 - 12 - t_0- 3 - t_0
t_4- 5 - 1 - 20 - t_4- 22 - 23 - 2 - 15 - t_4

t_4- 17 - 24 - 8 - 16 - t_4- 0 - 21 - t_4

p03-f25-v2t2.txt: Optimal Cost = 4742.

t_0- 3 - t_0- 7 - 13 - 18 - 12 - t_0
t_0- 11 - 8 - 16 - t_0- 9 - 10 - 6 -t_0
t_4- 20 - 19 - 14 - 4 - t_4- 0 - t_4
t_4- 22 - 23 - 17 - 24 - 2 - t_4- 15 - 5 - 1 - 21 - t_4

p03-m25-v1t1.txt: Optimal Cost = 4785.

t_0- 5 - 12 - 9 - t_0- 14 - t_0
t_0- 0 - 13 - 18 - t_0- 8 - 20 - t_0- 1 - 19 - 4 - t_0
t_4- 23 - 24 - 6 - t_4- 22 - 2 - t_4
t_4- 21 - 10 - 11 - 3 - t_4- 16 - 15 - 17 - t_4- 7 -t_4

p03-m25-v1t2.txt: Optimal Cost = 4785.

t_0- 0 - 12 - 9 - t_0- 1 - 19 - 4 - t_0- 8 - 20 - t_0
t_0- 13 - 5 - 18 - t_0- 14 - t_0
t_4- 17 - 15 - 16 - t_4- 3 - 11 - 10 - 21 - t_4
t_4- 2 - 22 - t_4- 7 - t_4- 23 - 24 - 6 - t_4

p03-m25-v2t1.txt: Optimal Cost = 4689.

t_0- 13 - 5 - 12 - 9 - t_0- 8 - 1 - 19 - 4 - t_0
t_0- 14 - 0 - 20 - t_0
t_4- 18 - 6 - 24 - 23 - t_4- 17 - 15 - 16 - t_4
t_4- 21 - 10 - 11 - 3 - 22 - t_4- 2 - 7 - t_4

p03-m25-v2t2.txt: Optimal Cost = 4479.

t_0- 20 - 9 - 12 - 0 - ta - 4 - 19 - 1 - 8 - t_0- 18 - 6 - 14 - t_0
t_4- 13 - 5 - 24 - 23 - t_4- 17 - 15 - 16 -t_4
t_4- 21 - 10 - 11 - 3 - 22 - t_4- 7 - 2 - t_4

p03-l25-v1t1.txt: Optimal Cost = 4842.

t_0- 7 - 21 - 4 - 12 - 11 - t_0- 16 - t_0
t_0- 5 - 13 - 22 - t_0- 0 - 17 - 24 - t_0
t_2- 23 - 10 - 18 - 20 - 9 - 19 - t_2- 2 - t_2
t_2- 3 - 6 - 1 - t_2- 15 - 14 - t_2- 8 - t_2

p03-l25-v1t2.txt: Optimal Cost = 4803.

t_0- 1 - 3 - 6 - 17 - t_0- 2 - 15 - t_0
t_0- 21 - 7 - 14 - 8 - t_0- 16 - 24 - t_0
t_4- 10 - 18 - 20 - 9 - 19 - 23 - t_4- 13 - 11 - 22 - t_4
t_4- 12 - 0 - 4 - 5 - t_4

p03-l25-v2t1.txt: Optimal Cost = 4741.
t_0- 3 - 8 - 15 - t_0- 24 - 17 - 1 - 16 - t_0
t_0- 0 - 4 - 21 - 7 - 14 - 2 - t_0
t_4- 10 - 18 - 20 - 9 - 19 - 6 - 23 - 22 - t_4
t_4- 13 - 5 - 12 - t_4- 11 - t_4

p03-l25-v2t2.txt: Optimal Cost = 4655.
t_2- 8 - 15 - 14 - t_2- 2 - t_2- 3 - 6 - 1 - 16 - t_2
t_4- 22 - 17 - 24 - 7 - 21 - 4 - 0 - t_4- 12 - t_4
t_4- 10 - 18 - 20 - 9 - 19 - 23 - 11 - t_4- 5 - 13 - t_4

p07-f25-v1t1.txt: Optimal Cost = 4761.
t_0- 4 - 16 - 7 - 17 - t_0- 18 - 10 - 9 - 6 - t_0
t_2- 14 - 13 - 15 - 5 - t_2- 22 - 21 - t_2
t_2- 11 - 23 - 24 - 3 - t_2- 20 - 1 - 12 - t_2
t_2- 2 - 8 - 19 - 0 - t_2

p07-f25-v1t2.txt: Optimal Cost = 4685.
t_2- 15 - 16 - 4 - 5 - t_2- 21 - 22 - t_2- 20 - 3 - t_2
t_2- 11 - 2 - 23 - 24 - t_2- 13 - 14 - 1 - t_2- 12 - t_2
t_4- 6 - 18 - 10 - 9 - t_4- 0 - 8 - 19 - t_4- 17 - 7 - t_4

p07-f25-v2t1.txt: Optimal Cost = 4703.
t_0- 17 - 6 - 10 - 18 - 7 - t_0- 4 - 13 - 15 - 16 - t_0
t_0- 5 - 0 - 2 - 8 - 19 - 9 - t_0
t_2- 20 - 3 - 24 - 23 - 11 - t_2- 12 - t_2
t_2- 21 - 22 - 14 - 1 - t_2

p07-f25-v2t2.txt: Optimal Cost = 4482.
t_0- 7 - 18 - 10 - 9 - 6 - t_0- 16 - 15 - 13 - 5 - 17 - t_0- 4 - t_0
t_2- 0 - 19 - 8 - 2 - 23 - 11 - t_2
t_2- 20 - 3 - 24 - 22 - t_2- 1 - 21 - 14 - 12 - t_2

p07-s25-v1t1.txt: Optimal Cost = 5098.

t_0- 14 - 13 - 15 - 16 - t_0

t_0- 18 - 12 - 17 - 11 - t_0- 21 - 20 - 19 - t_0

t_4- 3 - 8 - 9 - 7 - t_4

t_4- 4 - 6 - t_4- 2 - 0 - 1 - t_4- 22 - t_4

t_4- 23 - 10 - t_4- 5 - 24 - t_4

p07-s25-v1t2.txt: Optimal Cost = 4889.

t_0- 14 - 13 - 15 - 16 - t_0- 22 - t_0

t_0- 18 - 12 - 17 - 11 - t_0- 23 - 20 - 19 - t_0

t_4- 3 - 8 - 9 - 7 - t_4- 1 - 2 - 0 - t_4

t_4- 6 - 4 - t_4- 21 - 10 - t_4- 5 - 24 - t_4

p07-s25-v2t1.txt: Optimal Cost = 4787.

t_0- 18 - 12 - 17 - 16 - 11 - t_0- 22 - 20 - 19 - t_0

t_0- 15 - 13 - 0 - 14 - t_0

t_4- 7 - 3 - 8 - 9 - 6 - t_4- 5 - t_4

t_4- 1 - 2 - 24 - 4 - t_4- 21 - 10 - 23 - t_4

p07-s25-v2t2.txt: Optimal Cost = 4781.

t_0- 11 - 16 - 17 - 12 - 18 - t_0- 21 - 23 - 10 - 20 - t_0

t_0- 19 - 22 - t_0

t_4- 6 - 4 - 7 - 24 - t_4- 1 - 2 - 0 - 14 - t_4- 5 - t_4

t_4- 15 - 13 - 3 - 8 - 9 - t_4

p07-t25-v1t1.txt: Optimal Cost = 4898.

t_0- 10 - 6 - 8 - t_0- 11 - 18 - 1 - 9 - t_0

t_0- 13 - 12 - 15 - 19 - t_0

t_1- 4 - 16 - 5 - 23 - 22 - t_1- 21 - 24 - t_1- 17 - t_1

t_1- 0 - 20 - 14 - t_1- 3 - 7 - 2 - t_1

p07-t25-v1t2.txt: Optimal Cost = 4886.

t_0- 0 - 14 - 20 - 19 - t_0- 9 - 8 - 10 - t_0

t_0- 13 - 12 - 15 - 18 - t_0

t_2- 7 - 1 - 11 - t_2- 6 - 24 - 5 - 23 - t_2- 22 - 21 - t_2

t_2- 3 - 4 - 16 - t_2- 17 - 2 - t_2

p07-t25-v2t1.txt: Optimal Cost = 4767.

t_0- 9 - 1 - 18 - 19 - 12 - 13 - 11 - t_0
t_0- 15 - 14 - 20 - 0 - t_0
t_2- 24 - 5 - 16 - 4 - 3 - t_2- 22 - 23 - 21 - t_2
t_2- 17 - 2 - 7 - t_2- 8 - 10 - 6 - t_2

p07-t25-v2t2.txt: Optimal Cost = 4592.
t_0- 1 - 11 - 13 - 12 - 19 - 2 - 9 - t_0- 10 - 8 - t_0
t_1- 0 - 15 - 14 - 20 - t_1- 18 - 17 - t_1
t_1- 3 - 4 - 16 - 5 - 24 - t_1- 7 - 6 - 23 - 21 - 22 - t_1

A.2 Details of the Optimal or Best Solutions in 40-customer Instances

Customers are indexed from 0 to 39, and facilities are indexed with t_0, to t_4.

p01-f40-v1t1.txt: Optimal Cost = 7131.
t_0- 14 - 32 - 9 - 36 - 16 - t_0- 3 - 12 - 39 - 18 - t_0
t_0- 38 - 29 - 8 - t_0- 4 - 11 - t_0
t_3- 21 - 7 - 25 - 30 - t_3- 15 - 37 - 10 - t_3- 19 - t_3
t_3- 33 - 20 - 28 - t_3- 34 - 35 - 2 - t_3- 27 - 1 - t_3
t_4- 23 - 6 - 22 - t_4- 17 - t_4- 24 - 13 - t_4- 5 - 26 - 0 - 31 - t_4

p01-f40-v1t2.txt: Cost = 6999.
t_0- 5 - 22 - 26 - 16 - t_0- 18 - 12 - t_0- 17 - 3 - t_0
t_2- 34 - 19 - t_2- 15 - 1 - t_2- 9 - 38 - t_2
t_2- 31 - 2 - 35 - 28 - 20 - t_2- 29 - 33 - t_2- 8 - 10 - 37 - t_2
t_4- 0 - 21 - 27 - 30 - 25 - t_4- 22 - 6 - 5 - t_4- 13 - 24 - t_4
t_4- 26 - 7 - 23 - t_4- 11 - 4 - t_4
Best Solution we got 6983.
t_0- 3 - 12 - 39 - 18 - t_0- 11 - 36 - 14 - t_0- 17 - 16 - t_0
t_2- 20 - 34 - 35 - 2 - t_2- 8 - 10 - 37 - t_2- 33 - 29 - t_2
t_2- 4 - t_2- 38 - 32 - 9 - t_2- 15 - 19 - 28 - t_2
t_4- 31 - 1 - 0 - t_4- 26 - 21 - 27 - 30 - t_4
t_4- 6 - 25 - 7 - t_4- 24 - 13 - t_4- 23 - 22 - 5 - t_4

p01-f40-v2t1.txt: Optimal Cost = 6866.
t_0- 11 - 4 - 36 - 16 - t_0- 12 - 39 - 18 - t_0- 17 - 3 - t_0
t_2- 2 - 35 - 34 - 19 - t_2- 29 - 33 - t_2

t_2- 9 - 38 - 32 - 14 - 37 - t_2- 8 - 20 - 28 - 1 - 15 - t_2
t_4- 5 - 26 - 31 - 10 - t_4- 24 - t_4- 22 - 6 - 23 - 13 - t_4
t_4- 0 - 21 - 27 - 30 - 25 - 7 - t_4

p01-f40-v2t2.txt: Cost = 6821.
t_0- 29 - 33 - 4 - t_0- 11 - 17 - t_0
t_0- 16 - 36 - 9 - 38 - 32 - 14 - t_0- 3 - 12 - 39 - 18 - t_0
t_3- 19 - 34 - 35 - 2 - t_3- 1 - 15 - 8 - 20 - 28 - t_3- 21 - 7 - 25 - 30 - 27 - t_3
t_4- 23 - 22 - 6 - 5 - t_4- 26 - 0 - 31 - 10 - 37 - t_4- 13 - 24 - t_4
Best Solution we got 6818.
t_0- 31 - 10 - 37 - 4 - 16 - t_0- 3 - 11 - 36 - 14 - t_0
t_0- 9 - 29 - 38 - 32 - t_0- 12 - 39 - 18 - t_0
t_3- 19 - 34 - 35 - 2 - t_3- 28 - 20 - 33 - 8 - 15 - t_3- 1 - 0 - 21 - 30 - 27 - t_3
t_4- 26 - 7 - 25 - 6 - t_4- 13 - 23 - 22 - 5 - t_4- 17 - 24 - t_4

p01-l40-v1t1.txt: Cost = 7183.
t_0- 33 - 31 - 29 - 8 - t_0- 1 - 26 - 6 - t_0- 2 - 30 - t_0
t_2- 18 - 10 - 23 - 39 - t_2- 5 - 0 - 27 - t_2- 28 - 19 - t_2
t_2- 9 - 24 - t_2- 22 - 34 - 4 - t_2- 38 - t_2
t_4- 35 - 21 - 16 - 37 - t_4- 13 - 32 - 12 - t_4- 3 - 14 - t_4
t_4- 11 - 25 - 17 - 20 - 15 - t_4- 7 - t_4
t_4- 36 - t_4

p01-l40-v1t2.txt: Optimal Cost = 6934.
t_0- 33 - 34 - 4 - 26 - 6 - t_0- 8 - 29 - 31 - t_0- 30 - 2 - t_0
t_2- 39 - 10 - 24 - 25 - 11 - t_2- 27 - 21 - 0 - t_2- 38 - t_2
t_2- 5 - 9 - 18 - t_2- 23 - 19 - t_2- 22 - 28 - t_2
t_4- 37 - 17 - 20 - 15 - t_4- 12 - 32 - 13 - t_4- 14 - 3 - t_4
t_4- 16 - 35 - 1 - t_4- 7 - t_4- 36 - t_4

p01-l40-v2t1.txt: Optimal Cost = 6823.
t_0- 6 - 1 - 26 - 33 - 31 - t_0- 8 - 29 - 30 - 2 - t_0
t_2- 18 - 9 - 24 - 25 - 10 - t_2- 5 - 39 - 23 - 19 - t_2
t_2- 4 - 34 - 22 - 28 - t_2- 27 - 0 - 38 - t_2
t_4- 13 - 32 - 12 - 37 - 16 - t_4- 14 - 3 - t_4- 7 - 36 - t_4
t_4- 15 - 20 - 17 - 11 - 21 - 35 - t_4

p01-l40-v2t2.txt: Optimal Cost = 6612.

t_0- 31 - 29 - 8 - 30 - t_0- 33 - 34 - 22 - 4 - 26 - t_0- 36 - 1 - 6 - t_0
t_2- 5 - 9 - 18 - 10 - 39 - t_2- 27 - 0 - 21 - 38 - t_2- 28 - 19 - 23 - t_2
t_4- 13 - 32 - 12 - 37 - 16 - t_4- 14 - 3 - t_4- 7 - 2 - t_4
t_4- 15 - 20 - 17 - 25 - 24 - 11 - 35 - t_4

p03-f40-v1t1.txt: Optimal Cost = 8780.
t_0- 8 - 24 - 17 - t_0- 16 - 39 - t_0- 25 - t_0
t_0- 30 - 38 - 11 - t_0- 5 - 2 - 31 - t_0
t_1- 35 - 36 - 19 - 14 - t_1- 3 - 33 - t_1- 20 - 29 - t_1- 4 - 28 - t_1
t_2- 18 - 12 - 26 - 7 - t_2- 37 - 9 - t_2- 13 - 34 - 6 - t_2- 10 - t_2
t_4- 15 - 23 - 22 - t_4- 21 - 27 - 0 - t_4- 32 - 1 - t_4

p03-f40-v1t2.txt: Optimal Cost = 8753. t_0- 5 - 16 - 25 - t_0- 39 - 11 - t_0- 31 - 8 - t_0- 33 - 3 - t_0
t_1- 28 - 26 - 12 - 14 - t_1- 4 - 36 - 19 - t_1- 35 - 20 - t_1- 29 - 1 - t_1
t_2- 9 - 37 - t_2
t_2- 6 - 7 - 18 - 34 - t_2- 10 - t_2- 13 - t_2- 30 - 24 - 38 - t_2
t_4- 22 - 23 - 17 - 2 - t_4- 0 - 27 - 21 - t_4- 15 - 32 - t_4

p03-f40-v2t1.txt: Optimal Cost = 8440.
t_0- 25 - 11 - 38 - 8 - t_0- 39 - 31 - 2 - t_0- 16 - 5 - 3 - t_0
t_1- 36 - 19 - 35 - 20 - t_1- 28 - 26 - 12 - 14 - 4 - t_1- 29 - 1 - 27 - t_1
t_2- 9 - 30 - 37 - t_2- 10 - 13 - t_2- 6 - 33 - 7 - 18 - 34 - t_2
t_4- 22 - 23 - 17 - 24 - 15 - t_4- 0 - 32 - 21 - t_4

p03-f40-v2t2.txt: Optimal Cost = 8438.
t_0- 16 - 2 - 39 - 11 - t_0- 31 - 8 - 38 - t_0- 3 - 33 - 25 - t_0
t_1- 20 - 35 - 19 - 36 - t_1- 4 - 14 - 12 - 26 - 28 - t_1- 29 - 1 - 27 - t_1
t_2- 13 - 10 - t_2- 37 - 30 - 9 - t_2- 34 - 18 - 7 - 6 - t_2
t_4- 22 - 23 - 17 - 24 - 15 - t_4- 0 - 32 - 5 - 21 - t_4

p03-m40-v1t1.txt: Optimal Cost = 7253.
t_0- 14 - 13 - 24 - t_0- 29 - 2 - 34 - 9 - t_0
t_0- 4 - 39 - 26 - t_0- 1 - 10 - t_0- 25 - 18 - t_0
t_2- 16 - 23 - 30 - t_2- 38 - 33 - 3 - t_2
t_2- 31 - 11 - 36 - 19 - t_2- 15 - 37 - t_2- 32 - t_2
t_4- 35 - 20 - 21 - t_4- 7 - 8 - 28 - 0 - t_4- 12 - 17 - t_4
t_4- 6 - 5 - 27 - t_4- 22 - t_4

p03-m40-v1t2.txt: Cost = 7236.

t_0- 34 - 2 - 29 - 23 - t_0- 30 - 16 - 1 - t_0

t_0- 10 - t_0- 14 - 27 - 13 - t_0- 31 - 11 - 24 - t_0

t_2- 36 - 4 - 39 - t_2- 18 - 25 - 19 - t_2- 32 - t_2

t_2- 37 - 9 - 15 - t_2- 38 - 33 - 3 - t_2

t_4- 22 - t_4- 12 - 17 - t_4- 6 - 5 - 26 - t_4

t_4- 7 - 8 - 28 - 0 - t_4- 35 - 20 - 21 - t_4

p03-m40-v2t1.txt: Cost = 7140.

t_0- 14 - 13 - 24 - t_0- 16 - 29 - 23 - 30 - t_0

t_0- 36 - 4 - 39 - 5 - t_0- 1 - 10 - t_0

t_2- 37 - 9 - 2 - 34 - 15 - t_2

t_2- 38 - 33 - 18 - 19 - t_2- 25 - 11 - 31 - 3 - t_2- 32 - t_2

t_4- 27 - 26 - 12 - 6 - t_4- 22 - 21 - 20 - 35 - 7 - t_4

t_4- 17 - 0 - 28 - 8 - t_4

Best Solution we got 7132.

t_0- 36 - 4 - 39 - 5 - t_0- 1 - 10 - t_0

t_0- 14 - 13 - 24 - t_0- 16 - 29 - 23 - 30 - t_0

t_2- 15 - 34 - 2 - 9 - 37 - t_2- 32 - t_2

t_2- 38 - 33 - 11 - 31 - t_2- 19 - 25 - 18 - 3 - t_2

t_4- 17 - 0 - 28 - 8 - t_4- 6 - 12 - 26 - 27 - t_4

t_4- 7 - 35 - 20 - 21 - 22 - t_4

p03-m40-v2t2.txt: Cost = 6947.

t_0- 37 - 15 - 34 - 9 - 30 - t_0- 10 - 19 - 25 - 18 - t_0

t_0- 32 - 38 - 3 - 33 - t_0- 1 - t_0

t_1- 26 - 39 - 4 - 36 - t_1- 11 - 31 - 24 - 13 - t_1- 14 - 12 - 5 - t_1- 27 - t_1

t_4- 23 - 16 - 29 - 2 - t_4- 6 - t_4

t_4- 22 - 21 - 20 - 35 - 7 - t_4- 17 - 0 - 28 - 8 - t_4

p03-l40-v1t1.txt: Optimal Cost = 7167.

t_0- 15 - 8 - 14 - 19 - t_0- 31 - 10 - t_0- 39 - t_0

t_0- 11 - 0 - 1 - 21 - t_0- 32 - 12 - 9 - t_0

t_2- 23 - 18 - 16 - t_2- 4 - 3 - 36 - t_2

t_2- 17 - 30 - t_2- 29 - 2 - 22 - t_2

t_4- 5 - 20 - 13 - 27 - t_4- 7 - 6 - 28 - t_4

t_4- 25 - 33 - 35 - 24 - 34 - 38 - t_4- 37 - 26 - t_4

p03-l40-v1t2.txt: Cost = 6950.

t_0- 9 - 21 - 1 - 32 - t_0- 31 - 10 - t_0- 39 - t_0

t_0- 15 - 8 - 14 - 19 - t_0- 36 - 3 - 4 - t_0

t_2- 22 - 2 - 29 - t_2- 23 - 18 - 16 - t_2- 30 - 17 - t_2

t_4- 25 - 11 - 12 - 37 - t_4- 27 - 13 - 20 - 5 - t_4- 7 - 6 - 28 - t_4

t_4- 38 - 0 - 34 - 24 - 35 - 33 - t_4- 26 - t_4

p03-l40-v2t1.txt: Optimal Cost = 6849.

t_0- 9 - 12 - 11 - 38 - 32 - t_0- 31 - 10 - 39 - t_0- 15 - 8 - 4 - t_0

t_2- 2 - 29 - 30 - t_2- 23 - 18 - 21 - 16 - t_2- 17 - t_2

t_2- 22 - 36 - 14 - 19 - 3 - t_2

t_4- 20 - 13 - 27 - 37 - 26 - t_4- 7 - 5 - 6 - 28 - t_4

t_4- 0 - 1 - 34 - 24 - 35 - 33 - 25 - t_4

p03-l40-v2t2.txt: Optimal Cost = 6639.

t_0- 15 - 8 - 4 - 3 - 36 - t_0- 9 - 12 - 11 - 38 - 32 - t_0- 39 - 10 - 31 - t_0

t_2- 23 - 30 - 29 - t_2- 18 - 21 - 16 - t_2- 17 - 2 - 22 - t_2

t_4- 20 - 13 - 19 - 14 - 27 - t_4- 37 - 26 - t_4

t_4- 0 - 1 - 34 - 24 - 35 - 33 - 25 - t_4- 7 - 5 - 28 - 6 - t_4

p07-f40-v1t1.txt: Optimal Cost = 7167.

t_0- 16 - 37 - 13 - 36 - 5 - t_0- 4 - 15 - t_0- 17 - t_0

t_2- 11 - 2 - 28 - 23 - 24 - t_2- 1 - 14 - 12 - t_2

t_2- 22 - 21 - t_2- 38 - 3 - t_2- 39 - 25 - 20 - t_2

t_3- 10 - 18 - 35 - 7 - 6 - t_3- 19 - 31 - 9 - t_3

t_3- 32 - 33 - 34 - 8 - t_3- 26 - 27 - 0 - t_3- 30 - 29 - t_3

p07-f40-v1t2.txt: Optimal Cost = 7001.

t_0- 16 - 37 - 13 - 36 - 5 - t_0- 4 - 15 - t_0- 17 - 7 - t_0

t_2- 24 - 23 - 28 - 2 - 11 - t_2- 22 - 21 - t_2

t_2- 3 - 38 - t_2- 39 - 25 - 20 - t_2- 1 - 14 - 12 - t_2

t_4- 6 - 35 - 18 - 10 - t_4- 0 - 27 - 26 - t_4- 30 - 29 - t_4

t_4- 32 - 33 - 34 - 8 - t_4- 19 - 31 - 9 - t_4

p07-f40-v2t1.txt: Optimal Cost = 6881.

t_0- 16 - 15 - 37 - 13 - 36 - 5 - t_0- 4 - t_0

t_2- 20 - 21 - 1 - t_2- 11 - 2 - 28 - 23 - 24 - 3 - t_2

t_2- 38 - 22 - 14 - t_2- 39 - 25 - 27 - 12 - t_2

t_4- 0 - 32 - 33 - 34 - 8 - 19 - t_4- 30 - 26 - t_4
t_4- 17 - 7 - 35 - 18 - 10 - 6 - t_4- 29 - 31 - 9 - t_4

p07-f40-v2t2.txt: Cost = 6845.
t_1- 3 - 38 - 24 - 23 - 28 - t_1- 33 - 34 - 8 - 32 - 2 - t_1- 25 - 11 - t_1
t_2- 1 - 14 - 37 - 13 - 36 - t_2- 39 - 20 - 22 - 21 - t_2- 12 - t_2
t_4- 5 - 15 - 16 - 4 - 17 - t_4- 29 - 19 - 31 - 9 - t_4
t_4- 7 - 35 - 18 - 10 - 6 - t_4- 30 - 0 - 27 - 26 - t_4

p07-s40-v1t1.txt: Optimal Cost = 7343.
t_0- 32 - 1 - 20 - 38 - t_0- 17 - t_0
t_0- 3 - 4 - 2 - 19 - t_0- 24 - 36 - 23 - t_0
t_1- 13 - 7 - 12 - 11 - 29 - t_1- 16 - 14 - t_1
t_1- 34 - 35 - 39 - t_1- 0 - 31 - t_1- 21 - t_1
t_4- 33 - 18 - 5 - 15 - t_4- 28 - 6 - 30 - t_4
t_4- 8 - 25 - 10 - 26 - t_4- 22 - 27 - 9 - t_4- 37 - t_4

p07-s40-v1t2.txt: Optimal Cost = 7117.
t_0- 13 - 7 - 12 - 11 - 29 - t_0- 28 - 6 - 30 - t_0- 17 - t_0
t_1- 8 - 25 - 10 - 26 - t_1- 36 - 24 - 23 - t_1- 37 - t_1
t_1- 34 - 4 - 3 - t_1- 2 - 20 - 39 - 38 - 19 - t_1
t_4- 18 - 33 - 32 - t_4- 35 - 1 - t_4
t_4- 22 - 9 - 27 - 21 - t_4- 16 - 5 - 15 - 14 - t_4- 31 - 0 - t_4

p07-s40-v2t1.txt: Optimal Cost = 7000.
t_0- 13 - 7 - 12 - 26 - 11 - 6 - t_0- 29 - 28 - 30 - t_0
t_1- 2 - 20 - 34 - 4 - 3 - t_1- 9 - 27 - 22 - t_1
t_1- 24 - 36 - 8 - 25 - 10 - t_1- 37 - 19 - 23 - t_1
t_4- 33 - 18 - 5 - 15 - 14 - t_4- 17 - 16 - t_4
t_4- 32 - 1 - 35 - 39 - 38 - t_4- 31 - 0 - 21 - t_4

p07-s40-v2t2.txt: Optimal Cost = 6944.
t_0- 16 - 18 - 5 - 15 - t_0- 29 - 28 - 30 - 14 - t_0
t_2- 22 - 37 - 23 - 24 - t_2- 25 - 8 - 36 - 10 - t_2
t_2- 9 - 27 - t_2- 26 - 11 - 12 - 7 - 13 - 6 - t_2
t_4- 31 - 33 - 32 - 1 - 39 - t_4- 0 - 20 - 2 - 19 - 38 - t_4
t_4- 3 - 4 - 34 - 35 - t_4- 21 - 17 - t_4

p07-t40-v1t1.txt: Optimal Cost = 7012.

t_0- 13 - 28 - t_0- 16 - 15 - 30 - t_0- 4 - 29 - 17 - 7 - t_0

t_1- 2 - 8 - 19 - 20 - t_1- 36 - 25 - 3 - t_1

t_1- 35 - 34 - t_1- 23 - 24 - 11 - t_1- 37 - t_1

t_4- 32 - 33 - 10 - 18 - t_4- 27 - 22 - 0 - 38 - t_4

t_4- 12 - 1 - 26 - 14 - 5 - t_4- 6 - 31 - 9 - 39 - t_4- 21 - t_4

p07-t40-v1t2.txt: Cost = 6972.

t_0- 7 - 16 - 15 - 30 - t_0- 29 - 28 - 5 - 17 - t_0- 4 - t_0

t_2- 11 - 23 - 24 - t_2- 37 - 2 - t_2- 22 - 27 - t_2

t_2- 3 - 36 - 25 - t_2- 13 - 14 - 26 - 1 - t_2- 12 - t_2

t_4- 34 - 35 - 6 - t_4- 31 - 9 - 38 - 21 - t_4

t_4- 0 - 20 - 8 - 19 - 39 - t_4- 18 - 10 - 33 - 32 - t_4

p07-t40-v2t1.txt: Cost = 6665.

t_0- 13 - 14 - 26 - 1 - 28 - t_0- 4 - 30 - 15 - 16 - 7 - t_0- 29 - t_0

t_1- 23 - 24 - 36 - 25 - 3 - t_1- 11 - 20 - 8 - 2 - t_1- 37 - t_1

t_4- 18 - 10 - 33 - 32 - 9 - 39 - t_4- 22 - 27 - 12 - 5 - 17 - t_4

t_4- 34 - 35 - 19 - 0 - 38 - t_4- 21 - 31 - 6 - t_4

Best Solution we got 6663.

t_0- 28 - 1 - 26 - 14 - 13 - t_0- 4 - 30 - 15 - 16 - 7 - t_0- 29 - t_0

t_1- 23 - 3 - 25 - 36 - 24 - t_1- 2 - 8 - 20 - 11 - t_1- 37 - t_1

t_4- 38 - 0 - 34 - 35 - 19 - t_4- 6 - 31 - 21 - t_4

t_4- 18 - 10 - 33 - 32 - 9 - 39 - t_4- 22 - 27 - 12 - 5 - 17 - t_4

p07-t40-v2t2.txt: Cost = 6658.

t_0- 29 - 28 - 30 - 15 - 16 - t_0- 7 - 18 - 10 - 33 - 32 - 17 - t_0- 4 - t_0

t_2- 12 - 5 - 13 - 14 - 26 - 1 - t_2- 3 - 23 - 24 - 36 - 25 - t_2

t_4- 21 - 22 - 27 - 11 - 0 - t_4- 38 - 37 - 2 - 20 - 39 - t_4

t_4- 8 - 34 - 35 - 19 - t_4- 9 - 31 - 6 - t_4

Best Solution we got 6657.

t_0- 16 - 15 - 13 - 14 - 26 - 1 - 5 - 29 - t_0- 4 - 28 - 30 - t_0

t_1- 23 - 24 - 36 - 25 - 3 - t_1- 2 - 8 - 20 - 0 - 11 - t_1- 37 - t_1

t_4- 9 - 32 - 33 - 7 - 17 - t_4- 21 - 12 - 27 - 22 - t_4

t_4- 38 - 34 - 35 - 19 - 39 - t_4- 6 - 18 - 10 - 31 - t_4

Bibliography

Logistics Costs and U.S. Gross Domestic Product. Technical report, MacroSys Research and Technology, Washington, DC, August 2005. Prepared for Federal Highway Administration Department of Transportation. Available at http://ops.fhwa.dot.gov/freight/freight_analysis/econ_methods/lcdp_rep/index.htm.

K Aardal. Capacitated Facility Location: Separation Algorithms and Computational Experience. *Mathematical Programming*, 81(2):149–175, 1998.

K Aardal, Y. Pochet, and L. A. Wolsey. Capacitated Facility Location: Valid Inequalities and Facets. *Mathematics of Operations Research*, 20:562–582, 1995.

Y. Agarwal, K. Mathur, and H.M. Salkin. A Set-Partitioning-Based Exact Algorithm for the Vehicle Routing Problem. *Networks*, 19:731–749, 1989.

R. K. Ahuja, J.B. Orlin, S. Pallottino, M. P. Scaparra, and M.G. Scutellá. A Multi-exchange Heuristic for the Single-source Capacitated Facility Location Problem. *Management Science*, 50(6): 749–760, 2004.

R. Anbil, J.J. Forrest, and W.R. Pulleyblank. Column Generation and the Airline Crew Pairing Problem. *Proceedings of the International Congress of Mathematicians Berlin*, Extra Volume, III:667–686, 1998.

J. A. Appleget and R. K. Wood. Explicit-Constraint-Branching for Solving Mixed-Integer Programs. In M. Laguna and J.L. González-Velarde, editors, *Computing Tools for Modeling, Optimization and Simulation*, pages 245–261. Kluwer Academic Publishers, Boston, 2000.

J. R. Araque. Lots of Combs of Different Size for Vehicle Routing, 1990. CORE Discussion Paper.

J. R. Araque, L. A. Hall, and T. L. Magnanti. Capacitated Trees, Capacitated Routing and Associated Polyhedra, 1990. CORE Discussion Paper.

P. Augerat, J. M. Belenguer, E. Benavent, A. Corberán, and D. Naddef. Separating Capacity Inequalities in the CVRP Using Tabu Search. *European Journal of Operational Research*, 106: 546–557, 1999.

B. M. Baker. A Partial Dual Algorithm for the Capacitated Warehouse Location Problem. *European Journal of Operational Research*, 23:48–56, 1986.

E. Balas. Facets of the Knapsack Polytope. *Mathematical Programming*, 8:146–164, 1975.

E. Balas, S. Ceria, and G. Cornuéjols. A Lift-and-Project Cutting Plane Algorithm for Mixed 0-1 Programs. *Mathematical Programming*, 58:295–324, 1993.

E. Balas, S. Ceria, and G. Cornuéjols. Mixed 0-1 Programming by Lift-and-Project in a Branch-and-Cut Framework. *Management Science*, 42(9):1229–1246, 1996.

E. Balas and M. PAdberg. Set Partitioning: A Survey. *SIAM Review*, 18:710–760, 1976.

E. Balas and M. Perregaard. Disjunctive Programming. *Annals of Discrete Mathematics*, 5:3–51, 1979.

E. Balas and M. Perregaard. A Precise Correspondence Between Lift-and-Project Cuts, Simple Disjunctive Cuts. In *Mathematical Programming B*, pages 221–245, 2003.

E. Balas and A. Saxena. Optimizing Over the Split Closure. *Mathematical Programming*, 113(2): 219–240, 2008.

M. Balinski and R. Quandt. On an Integer Program for a Delivery Problem. *Operations Research*, 12:300–304, 1964.

C. Barnhart, E. L. Johnson, G.L. Nemhauser, M.W.P. Savelsbergh, and P.H. Vance. Branch and Price: Column Generation for Solving Huge Integer Programs. *Operations Research*, 46(3): 316–329, 1998.

S. Barreto, c. Ferreira, J. Paixao, and B. S. Santos. Using Clustering Analysis in a Capacitated Location-routing Problem. *European Journal of Operational Research*, 127(3):968–977, 2007.

J. E. Beasley. An Algorithm for Solving Large Capacitated Warehouse Location Problems. *European Journal of Operational Research*, 33:314–325, 1988.

J. E. Beasley and N. Christofides. An Algorithm for the Resource Constrained Shortest Path Problem. *Networks*, 19:379–394, 1989.

J. M. Belenguer, E. Benavent, C. Prins, C. Prodhon, and R. Wolfler-Calvo. A Branch and Cut Method for the Capacitated Location-Routing Problem. *Service Systems and Service Management, 2006 International Conference on*, 2:1541–1546, 2006.

R. T. Berger. *Location-Routing Models for Distribution System Design*. PhD thesis, Department of Industrial Engineering and Management Sciences, Northwestern University, 1997.

R. T. Berger, C. R. Coullard, and M. S. Daskin. Location-Routing Problems with Distance Constraints. *Transportation Science*, 41:29–43, 2007.

A. A. Bertossi, P. Carraresi, and G. Gallo. On Some Matching Problems Arising in Vehicle Scheduling Models. *Networks*, 17:271–281, 1987.

L. Bianco, A. Mingozzi, and S. Ricciardelli. A Set Partitioning Approach to the Multiple Depot Vehicle Scheduling Problem. *Optimization Methods and Software*, 3:163–194, 1994.

O. Bilde and J. Krarup. Sharp Lower Bounds and Efficient Algorithms for the Simple Plant Location Problem. *Discrete Mathematics*, 1:79–97, 1977.

A. Bixby. *Polyhedral Analysis and Effective Algorithms for the Capacitated Vehicle Routing Problem*. PhD thesis, Department of Industrial Engineering and Management Sciences, Northwestern University, 1999.

U. Blasum and W. Hochstättler. Application of the Branch and Cut Method to the Vehicle Routing Problem. Technical Report ZPR2000-386, Universität zu Köln, 2000.

L. Bodin and B. Golden. Classification in Vehicle Routing and Scheduling. *Networks*, 11(2):97–108, 1981.

L. Bodin, B. Golden, A. Assad, and M. Ball. Routing and Scheduling of Vehicles and Crews: The State of the Art. *Computers and Operations Research*, 10, 1983.

J. Brandao and A. Mercer. A Tabu Search Algorithm for the Multi-Trip Vehicle Routing and Scheduling Problem. *European Journal of Operational Research*, 100:180–191, 1997.

J. Brandao and A. Mercer. The Multi-Trip Vehicle Routing Problem. *Journal of the Operational Research Society*, 49:799–805, 1998.

L. Burlett. Regional Operating Expenses: A Case Study, 2002. Available at http://www.sb-d.com/issues/winter2002/features/save.asp.

A. Caprara and A. N. Letchford. On the Separation of Split Cuts and Related Inequalities. *Mathematical Programming*, 94:279–294, 2003.

G. Carpaneto, M. Dell'Amico, M. Fischetti, and P. Toth. A Branch and Bound Algorithm for the Multiple Depot Vehicle Scheduling Problem. *Networks*, 19:531–548, 1989.

P. Carraresi and G. Gallo. Network Models for Vehicle and Crew Scheduling. *European Journal of Operational Research*, 16:139–151, 1984.

I.M. Chao, B.L. Golden, and E.A. Wasil. A New Heuristic for the Multi-Depot Vehicle Routing Problem that Improves Upon Best-Known Solutions. *American Journal of Mathematical and Management Sciences*, 13:371–406, 1993.

D. C. Cho, E. L Johnson, M. W. Padberg, and M. R. Rao. On the Uncapacitated Plant Location Problem. I. Valid Inequalities and Facets. *Mathematics of Operations Research*, 8:579–589, 1983 a.

D. C. Cho, M. W. Padberg, and M. R. Rao. On the Uncapacitated Plant Location Problem. II. Facets and Lifting Theorems. *Mathematics of Operations Research*, 8:590–612, 1983 b.

N. Christofides. Vehicle Routing. In E. L. Lawler, J. F. Lenstra, A.H.G. Rinnooy Kan, and D. B. Shmoys, editors, *The Traveling Salesman Problem*, pages 431–448. Wiley, Chichester, 1985.

G. Clarke and J. Wright. Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*, 12(4):568–581, 1964.

Kenworth Truck Company. White Paper On Life Cycle Cost, 2003.

W. Cook, , R. Kannan, and A. Schrijver. Chva´tal Closures For Mixed Integer Programming Problems. *Mathematical Programming*, 47(2):155–174, 1990.

J.F. Cordeau, M. Gendreau, and G. Laporte. A Tabu Search Heuristic for Periodic and Multi-Depot Vehicle Routing Problems. *Networks*, 30:105–119, 1997.

G. Cornuejols. Valid Inequalities For Mixed Integer Linear Programs. *Mathematical Programming B*, 112:2008, 2006.

G. Cornuejols, M. L. Fisher, and G. L. Nemhauser. Location of Bank Accounts to Optimize Float: An Analytic Study of Exact and Approximate Algorithms. *Management Science*, 23:789–810, 1977.

G. Cornuejols and F. Harche. Polyhedral Study of the Capacitated Vehicle Routing Problem. *Mathematical Programming*, 60:21–52, 1993.

G. Cornuejols and Y. Li. Elementary Closures For Integer Programs. *Operations Research Letters*, 28:1–8, 2001.

G. Cornuejols, G. L. Nemhauser, and L.A. Wolsey. The Uncapacitated Facility Location Problems. In P. B. Mirchandani R. L. Francis, editor, *Discrete Location Theory*, pages 119–171. Wiley-Interscience, New York, 1990.

G. Cornuejols and J. M. Thizy. Some Facets of the Simple Plant Location Polytope. *Mathematical Programming*, 23:50–74, 1982.

H. Crowder, E. L. Johnson, and M. Padberg. Solving Large-Scale Zero-One Programming Problems. *Operations Research*, 31(5):803–834, 1983.

F. Cullen, J. Jarvis, and D. Ratliff. Set Partitioning Based Heuristics for Interactive Routing. *Networks*, 11:125–144, 1981.

M. S. Daskin. *Network and Discrete Location: Models, Algorithms and Applications*. John Wiley and Sons, Inc., New York, 1995.

I. R. de Farias Jr. and G. L. Nemhauser. A Family of Inequalities for the Generalized Assignment Polytope. *Operations Research Letters*, 29:49–55, 2001.

M. Dell'Amico, M. Fischetti, and P. Toth. Heuristic Algorithms for the Multiple Depot Vehicle Scheduling Problem. *Management Science*, 39(1):115–125, 1993.

H. Delmaire, J.A. Diaz, E. Fernandez, and M.Ortega. Comparing New Heuristics for the Pure Integer Capacitated Location Problem. Research Report DR97/10, Universidad Politecnica de Catalunya, Spain, 1997.

M. Desrochers. An Algorithm for the Shortest Path Problem with Resource Constraints. Technical Report G-88-27, GERAD, 1988.

M. Desrochers, J. Desrosiers, and M. M. Solomon. A New Optimization Algorithm for the Vehicle Routing Problem with Time Windows. *Operations Research*, 40(2):342–354, 1992.

M. Desrochers and G. Laporte. Iprovements and Extensions to Miller-Tucker-Zemlin Subtour Elimination Constraints. *Operations Research Letters*, 10:27–36, 1991.

M. Desrochers and F. Soumis. A Column Generation Approach to the Urban Transit Crew Scheduling Problem. *Transportation Science*, 23(1):1–13, 1989.

J. Desrosiers, Y. Dumas, M. Desrochers, F. Soumis, F. Sanso, and P. Trudeau. A Breakthrough in Airline Crew Scheduling. Technical Report G-91-11, GERAD, 1991.

M. Dror. Note on the Complexity of the Shortest Path Models for Column Generation in vrptw. *Operations Research*, 42(5):977–978, 1994.

I. Dumitrescu. *Constrained Path and Cycle Problems*. PhD thesis, Department of Mathematics and Statistics, University of Melbourne, 2002.

S. Eilon. Management Perspectives in Physical Distribution. *Omega*, 5(4):437–462, 1977.

D. Erlenkotter. A Dual Based Procedure for Uncapacitated Facility Location. *Operations Research*, 26:992–1009, 1978.

M. Eso. *Parallel Branch-and-cut for Set Partitioning*. Phd thesis, Cornell University, Ithaca, NY, 1999.

D. Feillet, P. Dejax, M. Gendreau, and C. Gueguen. An Exact Algorithm for the Elementary Shortest Path Problem with Resource Constraints: Application to Some Vehicle Routing Problems. *Networks*, 44(3):216–229, 2004.

M. Fischetti, A. Lodi, S. Martello, and P. Toth. A Polyhedral Approach to Simplified Crew Scheduling and Vehicle Scheduling Problems. *Management Science*, 47(6):833–850, 2001.

M. A. Forbes, J. N. Holt, and A. M. Watts. An Exact Algorithm for Multiple Depot Bus Scheduling. *European Journal of Operational Research*, 72:115–124, 1994.

R. Fukasawa, H. Longo, J. Lysgaard, M. Poggi de Aragão, M. Reis, E. Uchoa, and R. F. Werneck. Robust Branch-and-Cut-and-Price for the Capacitated Vehicle Routing Problem. *Mathematical Programming*, 106(3):491–511, 2006.

R. Galvao. The Use of Lagrangean Relaxation in the Solution of Uncapacitated Facility Location Problems. *Location Science*, 1:57–79, 1993.

W. M. Garvin, H. W. Crandall, J. b. John, and R. A. Spellman. Applications of Linear Programming in the Oil Industry. *Management Science*, 3:407–430, 1957.

B. Gavish and S. C. Graves. The Traveling Salesman Problem and Related Problems. Working Paper 7905, 1979.

M. Gendreau, A. Hertz, and G. Laporte. New Insertion and Post Optimization Procedures for the Travelling Salesman Problem. *Operations Research*, 40:1086–1094, 1992.

B.E. Gillett and J.G. Johnson. Multi-Terminal Vehicle-Dispatch Algorithm. *Omega*, 4:711–718, 1976.

R. E. Gomory. An Algorithm For Integer Solutions to Linear Programming. In R. L. Graves and P. Wolfe, editors, *Recent Advances in Mathematical Programming*, pages 269–302, New York, 1963. McGraw-Hill.

E. S. Gottlieb and M. R. Rao. The Generalized Assignment Problem: Valid Inequalities and Facets. *Mathematical Programming*, 46:31–52, 1990.

L. Gouveia. A Result On Projection for the Vehicle Routing Problem. *European Journal of Operational Research*, 85(3):610–624, 1995.

Z. Gu, G. L. Nemhauser, and M. W. P. Savelsbergh. Lifted cover inequalities for 0–1 integer programs: computation. *INFORMS Journal on Computing*, 10:427–438, 1998.

M. Guignard. Fractional Vertices, Cuts and Facets of the Simple Plant Location Problem. *Mathematical Programming Study*, 12:150–162, 1980.

E. Hadjiconstantinou, N. Christofides, and A. Mingozzi. A New Exact Algorithm for the Vehicle Routing Problem Based on q-Paths and k-Shortest Paths Relaxations. *Annals of Operations Research*, 61:21–43, 1995.

S.L. Hakimi. Optimal Locations of Switching Centers and the Absolute Centers and Medians of a Graph. *Operations Research*, 12:450–459, 1964.

S.L. Hakimi. Optimal Distribution of Switching Centers in a Communication Network and Some Related Theoretic Graph Problems. *Operations Research*, 13:462–475, 1965.

P. H. Hansen, B. Hegedahl, S. Hjortkjaer, and B. Obel. A Heuristic Solution to the Warehouse Location-Routing Problem. *European Journal of Operational Research*, 76:111–127, 1994.

K. Holmberg, M. Rönnqvist, and D. Yuan. An Exact Algorithm for the Capacitated Facility Location Problem with Single Sourcing. *European Journal of Operational Research*, 113:544–559, 1999.

M. Jepsen, B. Petersen, S. Spoorendonk, and D. Pisinger. Subset-Row Inequalities Applied to the Vehicle-Routing Problem with Time Windows. *Operations Research*, 56:497–511, 2008.

O. Kariv and S.L. Hakimi. An Algorithmic Approach to Network Location Problems. Part ii: The p-median. *SIAM Journal of Applied Mathematics*, 37:539–560, 1979.

J. Klincewicz and H. Luss. A Lagrangean Relaxation for Capacitated Facility Location with Single-source Constraints. *Journal of the Operational Research Society*, 37:495–500, 1986.

A. Klose and A. Drexl. Facility Location Models for Distribution System Design. *European Journal of Operational Research*, 162:4–29, 2005.

M. Körkel. On the Exact Solution of Large Scale Simple Plant Location Problems. *European Journal of Operational Research*, 39:157–173, 1989.

M. Labbé and F.V. Louveaux. Location problems. In S. Martello M. Dell'Amico, F. Maffioli, editor, *Annotated Bibliographies in Combinatorial Optimization*. J. Wiley & Sons, New York, 1997.

G. Laporte. Location Routing Problems. In B.L. Golden and A.A. Assad, editors, *Vehicle Routing: Methods and Studies*, pages 293–318. North-Holland, Amsterdam, 1988.

G. Laporte. The Vehicle Routing Problem: An Overview of Exact and Approximate Algorithms. *European Journal of Operational Research*, 59:345–358, 1992.

G. Laporte and Y. Nobert. An Exact Algorithm for Minimizing Routing and Operating Cost in Depot Location. *European Journal of Operational Research*, 6:224–226, 1981a.

G. Laporte and Y. Nobert. Comb Inequalities for the Vehicle Routing Problem. *Methods of Operations Research*, 51:271–, 1981b.

G. Laporte and Y. Nobert. Exact Algorithms for the Vehicle Routing Problem. *Annals of Discrete Mathematics*, 31:147–184, 1987.

G. Laporte, Y. Nobert, and D. Arpin. Optimal Solutions to Capacitated Multidepot Vehicle Routing Problems. *Congressus Numerantium*, 44:283–292, 1984.

G. Laporte, Y. Nobert, and D. Arpin. An Exact Algorithm for Solving a Capacitated Location-Routing Problem. *Annals of Operations Research*, 6:293–310, 1986.

G. Laporte, Y. Nobert, and M. Desrouchers. Optimal Routing with Capacity and Distance Restrictions. *Operations Research*, 33:1050–1073, 1985.

G. Laporte, Y. Nobert, and Y. Pelletier. Hamiltonian Location Problems. *European Journal of Operational Research*, 12:82–89, 1983.

G. Laporte, Y. Nobert, and S. Taillefer. Solving a Family of Multi-Depot Vehicle Routing and Location-Routing Problems. *Transportation Science*, 22:161–172, 1988.

J. Lenstra and A. Rinnooy Kan. Complexity of Vehicle Routing and Scheduling Problems. *Networks*, 11:221–228, 1981.

A. N. Letchford. On Disjunctive Cuts For Combinatorial Optimization. *Journal of Combinatorial Optimization*, 5:299–315, 2001.

A. N. Letchford and J. José S. González. Projection Results for Vehicle Routing. *Mathematical Programming*, 105(2-3):251–274, 2006.

J. M. Y. Leung and T. L. Magnanti. Valid Inequalities and Facets of the Capacitated Plant Location Problems. *Mathematical Programming*, 44:271–291, 1989.

C.K. Lin, C.K. Chew, and A. Chen. A Location-Routing-Loading Problem for Bill Delivery Services. *Computers and Industrial Engineering*, 43:5–25, 2002.

C.K. Lin and R.C.W. Kwok. Multi-Objective Metaheuristics for a Location-Routing Problem with-Multiple Use of Vehicles on Real Data and Simulated Date. *European Journal of Operational Research*, In Press, 2005.

M. E. Lübbecke and J. Desrosiers. Selected Topics in Column Generation. *Operations Research*, 53(6):1007–1023, 2005.

F. Maranzana. On the Location of Supply Points to Minimize Transport Cost. *Operations Research Quarterly*, 15:261–270, 1964.

T. McCormick, M. R. Rao, and G. Rinaldi. When is Min Cut with Negative Edges Easy to Solve? Easy and Difficult Objective Functions for Max Cut. Research report, IASI-CNR, 2002.

C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer Programming Formulations and Travelling Salesman Problems. *Journal of ACM*, 7(1):326–329, 1960.

H. Min. Consolidation Terminal Location-allocation and Consolidated Routing Problems. *Journal of Business Logistics*, 17(2):235–263, 1996.

H. Min, V. Jayaraman, and R. Srivastava. Combined Location-Routing Problems: A Synthesis and Future Research Directions. *European Journal of Operational Research*, 108(1):1–15, 1998.

J. M. Nambiar, L.F.Gelders, and L.N. Van Wassenhove. A Large Scale Location Allocation Problem in the Natural Rubber Industry. *European Journal of Operational Research*, 6:183–189, 1981.

G. L. Nemhauser and L. A. Wolsey. *Integer and Combinatorial Optimization*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, New York, 1989.

G. L. Nemhauser and L. A. Wolsey. A Recursive Procedure to Generate All Cuts For 0-1 Mixed Integer Programs. *Mathematical Programming*, 46(3):379–390, 1990.

A. Olivera and O. Viera. Adaptive Memory Programming for the Vehicle Routing Problem with Multiple Trips. submitted to Elsevier Science, 2004.

M. W. Padberg. (1,k)- Configurations and Facets for Packing Problems. *Mathematical Programming*, 18:94–99, 1980.

M. W. Padberg, T. J. van Roy, and L. A. Wolsey. Valid Inequalities for Fixed Charge Problems. *Operations Research*, 33:842–861, 1985.

J. Perl and M. S. Daskin. A Warehouse Location-Routing Model. *Transportation Research*, 19(B): 381396, 1985.

R. J. Petch and S. Salhi. A Multi-Phase Constructive Heuristic for the Vehicle Routing Problem with Multiple Trips. *Discrete Applied Mathematics*, 133:69–92, 2004.

H. Pirkul. Efficient Algorithms for the Capacitated Concentrator Location Problem. *Computers and Operations Research*, 14:197–208, 1987.

T. K. R. Ralphs, L. Kopman, W. R. Pulleyblank, and L.E. Trotter. On the Capacitated Vehicle Routing Problem. *Mathematical Programming*, 94:343–, 2003.

J. Renaud, G. Laporte, and F. F. Boctor. A Tabu Search Heuristic for the Multi-Depot Vehicle Routing Problem. *Computers and Operations Research*, 23(3):229–235, 1996.

C. S. ReVelle and H. A. Eiselt. Location Analysis: A Synthesis and Survey. *European Journal of Operational Research*, 165:1–19, 2005.

C. S. ReVelle and R. W. Swain. Central Facilities Location. *Geographical Analysis*, 2:30–42, 1970.

C. Ribeiro and F. Soumis. A Column Generation Approach to the Multiple Depot Vehicle Scheduling Problem. *Operations Research*, 42:41–52, 1994.

Y. Rochat and E. Taillard. Probabilistic Diversification and Intensification in Local Search for Vehicle Routing. *Heuristics*, 1:147–167, 1995.

S. Ropke. Disjunctive Cuts in Branch-and-Cut-and-Price Algorithms. Application to the Capacitated Vehicle Routing Problem. In *International Workshop on Column Generation*, June 2008.

S. Ropke and J. F. Cordeau. Branch-and-Cut-and-Price for Pickup and Delivery Problems With Time Windows. Technical report, November 2007.

K. Rosing and C. S. ReVelle. Heuristic Concentration: Two Stage Solution Construction. *European Journal of Operational Research*, 97:75–86, 1997.

T. J. Van Roy. A Cross Decomposition Algorithm for Capacitated Facility Location. *Operations Research*, 34:145–163, 1986.

D. Ryan and B. Foster. An Integer Programming Approach to Scheduling. In A. Wren, editor, *Computer Scheduling of Public Transport Urban Passenger Vehicle and Crew Scheduling*, pages 269–280. North-Holland, 1981.

S. Salhi and G. K. Rand. The Effect of Ignoring Routes When Locating Depots. *European Journal of Operational Research*, 39(2):150–156, 1989.

M.W.P. Savelsbergh and G. Nemhauser. *Functional Description of MINTO, A Mixed INTeger Optimizer*. Georgia Institute of Technology, 1996.

H. P. Simao and J. M. Thizy. A Dual Simplex Algorithm for the Canonical Representation of the Uncapaciated Facility Location Problem. *Operations Research Letters*, 8:279–286, 1989.

R. Sridharan. A Lagrangean Heuristic for the Capacitated Plant Location Problem with Single Source Constraints. *European Journal of Operational Research*, 66:305–312, 1993.

R. Srivastava. Alternate Solution Procedures for the Location-routing Problem. *Omega*, 21(4): 497–506, 1993.

E. D. Taillard, G. Laporte, and M. Gendreau. Vehicle Routing with Multiple Use of Vehicles. *The Journal of Operational Research Society*, 47(8):1065–1070, 1996.

M. Teitz and P. Bart. Heuristic Methods for Estimating the Generalized Vertex Median of a Weighted Graph. *Operations Research*, 16:955–961, 1968.

F. A. Tillman. The Multiple Terminal Delivery Problem with Probabilistic Demands. *Transportation Science*, 3:192–204, 1969.

F. A. Tillman and T.M. Cain. An Upper Bound Algorithm for the Single and Multiple Terminal Delivery Problem. *Management Science*, 18:664–682, 1972.

P. Toth and D. Vigo. *The Vehicle Routing Problem*. SIAM Monographs on Discrete Mathematics and Applications, 2002.

H. P. Vance, C. Barnhart, E. Johnson, and G.L. Nemhauser. Airline Crew Scheduling: A New Formulation and Decomposition Algorithm. *Operations Research*, 45(2):188–199, 1997.

M. H. Webb. Cost Functions in the Location of Depots for Multiple-Delivery Journeys. *Operational Research Quarterly*, 19:311–320, 1968.

R. Weismantel. On the $0\backslash 1$ Knapsack Polytope. *Mathematical Programming*, 77:49–68, 1997.